

基于变物性与收缩效应的圆柱药材热湿迁移模型

摘要

针对圆柱形药材烘干中内部干燥滞后、非线性物性与几何收缩共同影响热量和湿度传递的问题，本文基于药材的变物性构建**径向热湿耦合有限体积模型**，按恒定热物性、变物性和移动边界逐步扩展。根据附件数据计算时变环境与半径曲线，采用全隐式有限体积法求解径向温湿分布，以全截面首次达到判断依据确定干燥终点，并通过同物性对照，定量分析几何收缩的影响。

针对问题一：在固定半径与给定热物性下，建立 **Fourier–Fick 热湿传递基准模型**，以轴线对称条件和**表面双 Robin 边界**描述内部扩散及环境交换，采用表面加密网格求解预热过程。1800 s 时中心与表面温度分别为**33.577 °C 和 36.786 °C**，水分浓度分别为**2.550 和 1.510 kg/kg**。中心水分仍接近初值，表明预热初期失水主要发生在表层，内部干燥存在明显滞后。

针对问题二：引入热物性对水分、扩散系数对温度与水分的依赖，构建**变物性非线性热湿耦合模型**，通过 Picard 迭代交替更新场量与物性，反映温度升高与含水量降低对水分扩散的相反作用。3 h 时中心与表面温度分别达到**49.849 °C 和 49.967 °C**，水分浓度分别降至**1.766 和 1.008 kg/kg**。温度趋于均匀而水分仍有明显径向差异，说明水分均匀化滞后于升温过程，温度接近环境值不能作为干燥完成的依据。

针对问题三：将问题二模型扩展为**基于全截面首达判据的长期干燥模型**。依据末段观测，假设 4 h 后环境保持为 $T_e = 50\text{ °C}$ 、 $C_e = 0.05\text{ kg/kg}$ ，采用相邻时间层线性插值，定位全截面水分最大值降至 0.15 kg/kg 的临界时刻，其后全截面严格达标。得到**全截面临界达标时间为 57.472 h**，此时中心与表面水分浓度分别为**0.150 和 0.053 kg/kg**；**表面首达时间为 12.552 h**，早于全截面达标约**44.919 h**。计算表明，药材中心的水分浓度最高、达标最晚；考虑圆柱两端面的传热和失水后，这一结论仍然成立。因此，表面达标并不代表内部已完成干燥，仅依据表面水分判断会导致过早停机。

针对问题四：建立**收缩移动边界热湿耦合模型**，在均匀材料收缩与干骨架守恒假设下，通过 $\xi = r/R(t)$ 将变化的物理区域映射至固定材料坐标，使材料与网格速度一致，统一处理扩散距离缩短与表面交换变化。得到**临界干燥时间为 51.088 h**，**达标半径为 1.200 cm**。保持附件物性与环境条件一致，固定初始半径的对照模型需**129.845 h**；在给定物性与收缩轨迹下，**收缩使干燥时长缩短 60.7%**。该对照用于分析收缩效应。

网格与步长加密检验支持数值收敛；**移动网格验证**中，两种坐标实现的最大水分差为 $2.61 \times 10^{-13}\text{ kg/kg}$ ，干燥时间差为 $1.07 \times 10^{-7}\text{ s}$ ；**COMSOL 有限元复核**在六个采样时刻的**最大水分差为 $4.54 \times 10^{-5}\text{ kg/kg}$** 。在收缩轨迹不变的所设温度与环境水分扰动试验中，**干燥时长对温度的响应更明显**。模型可确定圆柱药材的干燥终点。

关键字： 药材烘干 Fourier–Fick 模型 有限体积法 热湿耦合 移动边界

一、问题重述

1.1 问题背景

药材烘干过程同时包含内部传热和水分迁移。外部温度变化首先作用于药材表面，热量向内部传递；水分则在浓度梯度和表面传质作用下由内部向外迁移。由于热扩散和水分扩散的时间尺度不同，温度场和水分场通常具有明显的径向非均匀性，表面状态不能代表整个截面的状态。因此，需要建立描述药材内部温度和水分浓度时空分布的非稳态模型，并以全截面状态作为干燥完成的判断依据。

本题研究一根长 25 cm、初始半径 2 cm 的近似圆柱形药材，初始温度和水分浓度分别为 28 °C 和 2.55 kg/kg。已知条件依次引入时变环境、状态相关物性、全截面含水率约束以及半径收缩。因而四个问题分别对应恒定热物性、变物性耦合、全域达标判定和移动边界四个层次；其中问题一仍保留水分扩散系数 $D(C)$ 的非线性，模型在统一热湿传递框架下逐步增加相应的物理因素。本节题意整理与文字表述经工具辅助检查，问题界定与建模决策由参赛队负责。

1.2 问题提出

根据题目给定的几何尺寸、初始状态、环境条件及物性关系，本文需要获得药材内部温度和水分浓度的时空分布，并确定全截面达到含水率要求的时间。四问的区别在于逐步增加物性变化、全域约束和几何收缩等因素，因此采用由简到繁的统一建模路线。

问题一：首先需要刻画预热阶段药材内部热量与水分的径向迁移规律。由于药材内部存在明显的温度和水分梯度，应建立圆柱坐标下的非稳态传热传质模型，并结合时变环境边界求得温湿场，为后续各问提供统一的基础模型和数值求解框架。

问题二：在基础模型上进一步考虑物性随温度和水分状态变化的影响，使导热、储热和水分扩散过程相互耦合。由此需将问题一的基准模型扩展为变物性的非线性热湿迁移模型，以分析整个烘干过程中温度场与水分场的动态演化及其控制机制。

问题三：在获得长期水分迁移规律后，需要进一步解决“何时完成烘干”的判定问题。将“药材各处水分均低于规定阈值”转化为空间最大水分浓度的首达条件，并据此确定控制位置及最终烘干时间，从而避免仅依据表面或平均水分作出过早停机判断。

问题四：实际烘干过程中药材半径持续收缩，使计算区域由固定域转化为移动域。为此需在变物性热湿模型中进一步引入时变几何边界，通过固定域映射处理收缩过程，并分析扩散距离缩短对烘干效率的影响；同时采用独立移动网格计算以及 COMSOL 物理仿真对移动边界结果进行交叉验证。

二、问题分析

四问共享“附件数据重构—内部热湿传递—表面对流交换—数值求解—结果验证”的主链条，但各问新增困难不同。问题一解决基础的时变边界扩散；问题二处理系数随解变化引起的非线性；问题三引入全域阈值首达事件；问题四再处理随时间收缩的空间区域。按照这一依赖关系建立同一套守恒离散程序，可以避免各问采用不同数值口径造成的结果不可比。总体技术路线如图 1 所示。

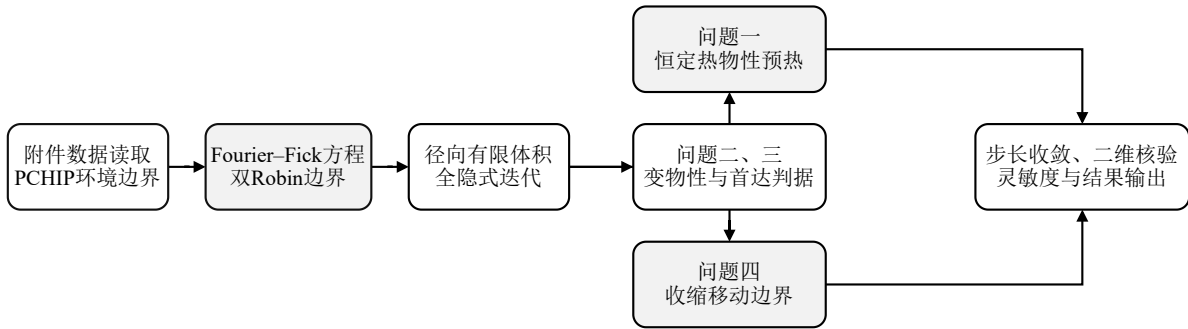


图 1 四问递进建模与验证技术路线

2.1 问题一的分析

问题一的半径和主要热物性给定为常数，但热量和水分仍需在药材内部传递。热、质扩散特征时间不同，且表面对流边界不能忽略，因此不能用截面平均量描述预热过程。本文建立一维径向热湿传递模型，保留水分扩散系数 $D(C)$ 的非线性；由附件重构时变环境，在中心和表面分别设置对称与 Robin 边界，并采用有限体积法求解预热阶段的温湿分布，为后续各问提供统一的离散框架。

2.2 问题二的分析

问题一的恒定热物性假设不能反映干燥过程中密度、比热和导热系数随状态变化的影响，水分扩散系数则已保留 $D(C)$ 的非线性。问题二进一步引入状态相关热物性及 $D(T, C)$ ，形成温度场与水分场相互影响的非线性耦合模型。由于系数依赖待求状态，采用分步 Picard 迭代更新温度、水分与物性，获得同一时间层的自洽解，再比较中心与表面的响应。

2.3 问题三的分析

问题三要求所有位置的水分浓度均低于 0.15 kg/kg ，因此表面浓度或截面平均浓度都不能作为停机依据。本文在问题二模型上延长计算时间，以全域最大水分浓度越过阈

值作为事件判据；通过全过程剖面检查确定控制位置，并在阈值附近插值定位临界时刻，区分临界等于与其后的严格低于。

2.4 问题四的分析

问题四中药材半径随时间变化，固定空间区域和固定网格不能直接描述材料边界的移动。本文假设药材均匀径向收缩，利用 $\xi = r/R(t)$ 将移动域映射至固定区间，并在问题二的变物性热湿模型上更新几何尺度。为区分收缩本身的影响，另设使用相同物性和环境的固定半径对照模型；再用物理坐标移动网格与 COMSOL 有限元计算进行交叉验证。

三、模型假设

依据题给几何、物性和边界数据，采用以下等效化与闭合假设；各项假设的依据及检验范围在相应模型中说明。

1. **介质与几何。**用题给标量物性表征均匀、各向同性的等效连续介质，初始温湿场均匀且轴对称。主模型保留中截面径向梯度；圆柱几何为降维提供依据，端面影响由后文二维对照检验。
2. **传递机制。**以导热和 Fick 扩散描述内部热湿迁移，耦合通过题给状态相关物性体现。不另设内热源、辐射、内部宏观对流、重力渗流及托盘接触传热通道，以题给物性对应的扩散过程为研究范围。
3. **表面交换与浓度口径。**表面采用 Robin 边界；问题一取题给的 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ，后续三问假设沿用。将附件环境水分量的数值作为与药材干基含水率可比较的等效边界值 C_e ，这一映射由传质边界的闭合假设确定。
4. **蒸发潜热。**采用基于题给热物性的导热方程，不单列蒸发潜热项，也不将其视为已包含在 ρc_p 中。蒸发冷却对温度和达标时长的影响尚未量化，相关结果解释为这一热过程简化下的条件预测。
5. **环境延拓。**附件一末段观测接近稳定水平，故假设 14400 s 后 $T_e = 50.000 \text{ }^\circ\text{C}$ 、 $C_e = 0.05000 \text{ kg/kg}$ 。该设定代表后续工况保持不变的情景，合理性由末段统计与环境扰动分析共同评估。
6. **收缩闭合。**问题四保持长度不变，材料点沿径向同比例收缩；无干物质流失，且干骨架密度在截面内均匀，满足 $\rho_d R^2 = \rho_{d0} R_0^2$ 。这是由给定半径历史构造内部运动的简化闭合； ρ_d 与热方程采用的题给 $\rho(C)$ 分别定义。

四、符号说明

本文主要符号及其单位见表 1。温度函数中的绝对温度统一写为 $T_K = T + 273.15$ ，其余温差仍可直接用摄氏度计算。

表 1 主要符号及含义

符号	含义	单位	备注
r, z	径向、轴向物理坐标	m	主模型只保留 r
$R(t), L$	瞬时半径、圆柱长度	m	$L = 0.25$
T, T_e	药材温度、环境温度	°C	扩散系数中换为 K
C	药材干基含水率	kg/kg	水/干物质质量比
C_e	等效环境边界含水率	kg/kg	采用附件环境量的数值
ρ, c_p	密度、比热容	kg/m ³ , J/(kg K)	状态相关
ρ_d	干骨架密度	kg/m ³	区别于题给 ρ
k, D	导热系数、水分扩散系数	W/(m K), m ² /s	状态相关
h_T, h_m	表面传热、传质系数	W/(m ² K), m/s	Robin 边界
ξ	固定域材料坐标	1	$\xi = r/R(t)$
V_i, G_f	控制体体积、界面导通系数	m ² , 依场量而定	按单位轴向长度
$\Delta r_i, \Delta t$	局部空间、时间步长	m, s	表面加密网格, 1 s
n, m	时间层、Picard 迭代编号	1	非负整数
t_d	全截面达标临界时刻	s 或 h	$\max C = 0.15$

五、模型准备

已有研究对吸湿性毛细多孔材料的对流干燥进行了数值模拟 [3]，并将温湿场计算扩展至热风、红外与微波联合干燥 [4]。本文以题给热风边界和物性关系为依据建立模型，不引入辐射或微波热源。

5.1 几何降维与初始条件

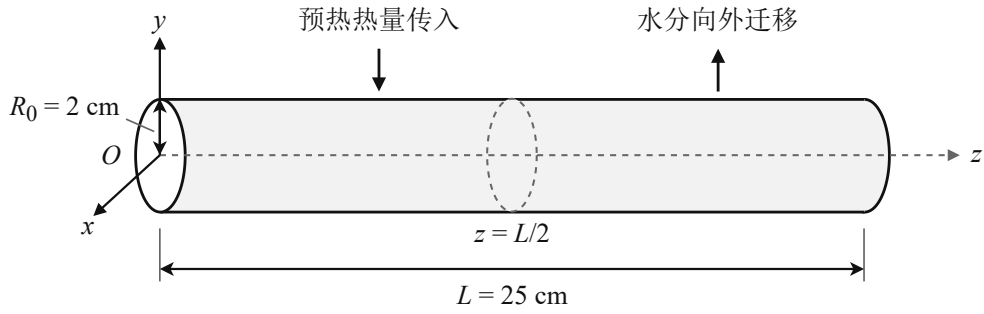
药材的初始几何及坐标约定见图 2。取一端面中心为原点 O ， z 轴沿药材轴线， x 、 y 轴位于端面内；圆柱长 $L = 25$ cm，初始半径 $R_0 = 2$ cm。中截面中心记为 O_m ，径向距离 r 从该点起算，周向角 θ 从正 x 轴起算。图中热量传入箭头示意预热阶段，水分箭头表示向外迁移。

在圆柱坐标 (r, θ, z) 中，轴对称条件使 $\partial/\partial\theta = 0$ 。主模型进一步取中截面 $z = L/2$ ，忽略该处的轴向梯度，计算域为 $0 \leq r \leq R(t)$ 。固定半径问题中 $R = R_0 = 0.02$ m；移

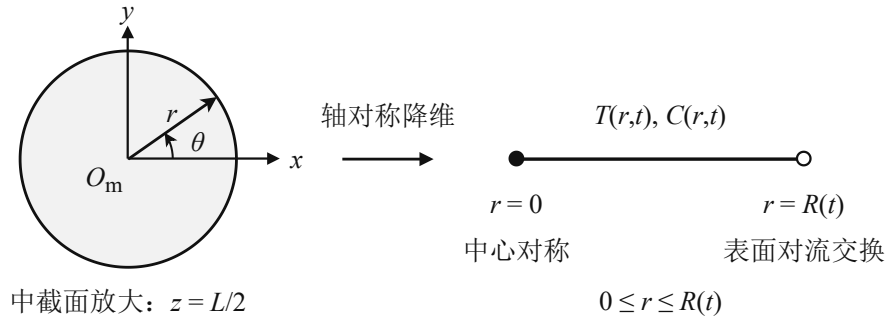
动边界问题稍后映射到 $0 \leq \xi \leq 1$ 。初始条件为

$$T(r, 0) = 28\text{ }^{\circ}\text{C}, \quad C(r, 0) = 2.55\text{ kg/kg}. \quad (1)$$

初始中截面至端面的距离 $L/2$ 为半径的 6.25 倍，较长的轴向传递路径为中部径向近似提供几何依据。后文保持相同物性和表面交换条件，以含端面传递的二维模型检查达标时间及控制位置；该对照检验降维误差，均匀、各向同性仍是依据题给标量物性采用的等效化假设。



(a) 初始圆柱几何与坐标系



(b) 中截面圆柱坐标与一维径向计算域

图 2 药材初始几何、坐标系与径向计算域（中截面单独放大）

5.2 附件数据检查与边界重构

附件数据的结构与采用方式汇总于表 2。附件一、二分别含 241 个和 145 个等间隔采样时刻，采样间隔分别为 60 s 和 1800 s，均不同于 1 s 的数值积分步长，因此需通过插值获得各积分时刻的边界值。

采用分段三次 Hermite 保形插值 (PCHIP)，节点斜率按相邻割线斜率确定，使单调数据区间不产生新的极值。该方法避免半径在快速收缩段出现非物理回弹，并保持一阶连续，便于稳定求取问题四所需的 $\dot{R}(t)$ 。

附件一存在小幅波动，故环境量保留实测序列的 PCHIP 重构，不强制单调化。最后 1 h 的温度和环境水分均值分别为 $49.9989\text{ }^{\circ}\text{C}$ 和 0.049988 kg/kg ，据此假设 14400 s 后工况保持为 $50.000\text{ }^{\circ}\text{C}$ 与 0.05000 kg/kg ，避免无限沿用末个采样点的波动。末段稳定性

表 2 附件时序数据及模型处理方式

数据	覆盖区间	采样间隔	模型处理
环境温度 T_e	0–14400 s	60 s	PCHIP, 区间外取 50 °C
环境水分 C_e	0–14400 s	60 s	PCHIP, 区间外取 0.05
药材半径 R	0–259200 s	1800 s	单调 PCHIP 并同步求导

支持这一情景的选取，但不能确定后续数十小时的实际边界；后文第9.3.8节进一步量化环境偏离设定值对达标时间的影响。

5.3 径向热湿控制方程

基于 Fourier 导热定律和 Fick 扩散定律 [1]，固定半径域内的控制方程为

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right], \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(T, C)\frac{\partial C}{\partial r}\right]. \quad (3)$$

式中 ρc_p 是单位体积热容量，决定温度对净热通量的响应速度； D 是以题设浓度变量定义的等效扩散系数。问题一中除 $D(C)$ 外各物性为常数，问题二至四中的密度、比热容、导热系数和水分扩散系数均随状态更新。

轴线处没有净通量，故满足

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0. \quad (4)$$

侧表面采用“向外法向通量等于表面对流通量”的约定：

$$-k\left.\frac{\partial T}{\partial r}\right|_{r=R} = h_T[T(R, t) - T_e(t)], \quad (5)$$

$$-D\left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m[C(R, t) - C_e(t)]. \quad (6)$$

当 $T_e > T(R, t)$ 时，(5) 给出向内热流；当 $C(R, t) > C_e$ 时，(6) 给出向外水分通量。附录 2 给出了问题一的交换系数，后续附录仅更新材料物性。本文假设同一烘房内后续过程仍可沿用这组系数；问题四显式更新面积和几何尺度，系数本身的流动及几何依赖不另行标定。

C 为药材水质量与干物质质量之比。附件环境水分量虽也以 kg/kg 表示，但题面未明确其质量基准及与材料平衡含水率的转换关系。为闭合 (6)，本文假设附件数值可直接用作材料侧的等效环境边界值 C_e ， h_m 则作为该变量下的有效交换系数。若原始环境

量以干空气质量为基准，须结合吸湿平衡关系重新换算；现有预测以这一等效映射成立为条件，环境水分扰动检验不能替代该映射的标定。

5.4 物性关系及耦合路径

问题一采用题面附录 2 给出的热物性常数与水分扩散系数公式

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7.0 \times 10^{-9} \exp(-0.89/C). \quad (7)$$

问题二、三采用题面附录 3 给出的物性关系

$$\begin{aligned} \rho(C) &= 650 + 128C, \\ c_p(C) &= 1450 + 2736 \frac{C}{C+1}, \\ k(C) &= 0.21 + 0.38 \frac{C}{C+1}, \\ D(T, C) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T+273.15}\right]. \end{aligned} \quad (8)$$

问题四采用题面附录 4 给出的物性关系

$$\begin{aligned} \rho(C) &= 760 + 90C, \\ c_p(C) &= 1850 + 2150 \frac{C}{C+1}, \\ k(C) &= 0.12 + 0.20 \frac{C}{C+1}, \\ D(T, C) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left[-\frac{3850}{T+273.15}\right]. \end{aligned} \quad (9)$$

由 (8) 和 (9) 可知，水分影响储热与导热，温度和水分共同影响 D ，由此形成状态相关物性耦合。采用题给函数可在统一口径下比较四问的温湿迁移，但这一耦合不包含蒸发耗热；Purlis 的食品对流干燥多相模型则考虑了吸附热，并讨论蒸发速率项及方程闭合问题 [5]。本文未建立液汽相分配及局部汽化速率关系，也未完成潜热与输入热量的量级对照，故不将缺少相变项解释为其影响已足够小；后文收敛与交叉求解仅检验既定方程的数值实现。

5.5 节点中心有限体积离散

空间节点取 $r_i = R[1 - (1 - i/N)^2]$, $i = 0, 1, \dots, N$, 基准取 $N = 160$, 对低扩散率表层加密；输出位置仍为每隔 0.1 cm, 由计算解插值得到。内部界面位于 $r_{i+1/2} = (r_i + r_{i+1})/2$ 。

按单位轴向长度积分后，第 i 个环形控制体体积和界面面积分别为

$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}. \quad (10)$$

中心控制体下边界直接取 0，表面控制体上边界取 R 。这种几何构造自动消除了连续方程在 $r = 0$ 处形式上的 $1/r$ 奇点。

以 ϕ 统一表示 T 或 C ，储存系数记为 s ，扩散系数记为 Γ 。在时间层 $n+1$ 采用全隐式格式，内部节点守恒式为

$$\frac{s_i V_i}{\Delta t} (\phi_i^{n+1} - \phi_i^n) = G_{i+1/2} (\phi_{i+1}^{n+1} - \phi_i^{n+1}) - G_{i-1/2} (\phi_i^{n+1} - \phi_{i-1}^{n+1}), \quad (11)$$

其中

$$G_{i+1/2} = \frac{A_{i+1/2} \Gamma_{i+1/2}}{r_{i+1} - r_i}, \quad \Gamma_{i+1/2} = \frac{\Gamma_i + \Gamma_{i+1}}{2}. \quad (12)$$

温度场取 $s = \rho c_p$, $\Gamma = k$ ，水分场取 $s = 1$, $\Gamma = D$ 。将 (11) 整理为

$$-a_W \phi_{i-1}^{n+1} + a_P \phi_i^{n+1} - a_E \phi_{i+1}^{n+1} = b_i, \quad (13)$$

其中

$$a_W = G_{i-1/2}, \quad a_E = G_{i+1/2}, \quad a_P = \frac{s_i V_i}{\Delta t} + a_W + a_E, \quad b_i = \frac{s_i V_i}{\Delta t} \phi_i^n.$$

矩阵主对角元为正，非对角元非正并满足对角占优，既保留局部守恒，也有利于保持温度、水分的离散最大值性质。

中心节点令 $a_W = 0$ ，自然实现零通量条件。表面节点将 Robin 通量直接并入控制体：

$$a_P \leftarrow a_P + A_R h, \quad b_N \leftarrow b_N + A_R h \phi_e, \quad A_R = 2\pi R, \quad (14)$$

其中热场取 $(h, \phi_e) = (h_T, T_e)$ ，水分场取 (h_m, C_e) 。最终每一场只需求解一个三对角线性系统，计算量随径向节点数线性增长。

5.6 非线性 Picard 迭代与输出控制

在每个新时间层，令初始迭代值等于上一时间层结果，并执行：

1. 由当前 $C^{(m)}$ 计算 ρ, c_p, k ，组装并求解温度三对角系统，得到 $T^{(m+1)}$ ；
2. 由 $T^{(m+1)}, C^{(m)}$ 计算 D ，组装并求解水分三对角系统，得到 $C^{(m+1)}$ ；
3. 检查

$$\|T^{(m+1)} - T^{(m)}\|_\infty < 10^{-8}, \quad \|C^{(m+1)} - C^{(m)}\|_\infty < 10^{-10};$$

若未同时满足则返回步骤 1，最多允许 30 次迭代。

四问终算均采用 $\Delta t = 1 \text{ s}$ ；问题二、三共享从初态至达标的同一条轨迹，问题三按 60 s 抽取，不再独立积分。程序在每一步检查有限值、 $0 \leq C \leq 2.55$ 、径向单调性、Picard 收敛及方程平衡残差；阈值跨越时另存插值得到的首达剖面。空间网格、时间步长与输出采样分别检验，四位小数仅表示结果展示精度。

六、问题一：固定几何与给定热物性的预热模型建立与求解

6.1 恒定热物性预热过程模型

6.1.1 模型特化与特征尺度判断

问题一中半径固定为 $R_0 = 0.02 \text{ m}$ ，密度、比热容和导热系数分别为 820 kg/m^3 、 2600 J/(kgK) 和 0.36 W/(mK) 。因此热扩散率为

$$\alpha = \frac{k}{\rho c_p} = 1.689 \times 10^{-7} \text{ m}^2/\text{s}. \quad (15)$$

初始水分浓度下，由 (7) 得到

$$D_0 = 7.0 \times 10^{-9} \exp(-0.89/2.55) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}. \quad (16)$$

据此可估计径向热、质扩散特征时间

$$\tau_T = \frac{R_0^2}{\alpha} = 2369 \text{ s}, \quad \tau_C = \frac{R_0^2}{D_0} = 8.101 \times 10^4 \text{ s}. \quad (17)$$

水分扩散特征时间约为热扩散特征时间的 34 倍，预示 1800 s 内温度扰动能够明显进入中心，而水分变化主要局限在外层。

进一步计算热、质 Biot 数

$$Bi_T = \frac{h_T R_0}{k} = 1.389, \quad Bi_C = \frac{h_m R_0}{D_0} = 3.240. \quad (18)$$

二者均明显大于集中参数法通常要求的 0.1，说明内部阻力不可忽略。尤其 $Bi_C > Bi_T$ ，表面对流传质并非唯一瓶颈，水分到达表面前必须克服更显著的内部扩散阻力。因此，本问不能只解两个平均状态常微分方程，而必须保留径向偏微分方程。

6.2 模型的求解

采用第五节的统一有限体积方法，以 161 个表面加密节点和 1 s 步长计算 $[0, 1800] \text{ s}$ 的预热过程。控制体、界面导通系数及三对角系统分别按 (10)、(12) 和 (13) 构造，中心零通量和表面 Robin 通量按 (14) 处理。

初始场取 (1), 每步环境量由 PCHIP 在 t_{n+1} 处求值。温度方程为线性, 水分扩散系数 $D(C)$ 仍需 Picard 更新; 迭代右端的历史状态始终固定为上一时间层。沿用第五节的双重容差及 30 次迭代上限, 本问实际最多 4 次迭代即收敛。

每步检查水分范围、径向单调性和通量平衡, 完成 1800 个时间步后结束。温湿场线性插值至 0, 0.1, $\dots$, 2.0 cm 的 21 个规定物理位置, 计算网格与输出位置分别设置。

6.3 结果的分析 and 验证

6.3.1 温度场计算结果与分析

问题一典型时刻的温度结果见表 3。100 s 时热扰动尚集中在表面附近, 中心温度只比初值高 $6.4 \times 10^{-5} \text{ }^\circ\text{C}$; 随着热量持续输入, 600 s 后整个截面开始明显升温。到 1800 s, 中心与表面分别达到 $33.5770 \text{ }^\circ\text{C}$ 和 $36.7865 \text{ }^\circ\text{C}$ 。

表 3 问题一典型时刻温度分布 ($^\circ\text{C}$)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0004	28.0041	28.0324	28.1802
300	28.0414	28.0642	28.1522	28.3683	28.8495
600	28.4549	28.5377	28.8053	29.3168	30.1650
900	29.3262	29.4602	29.8774	30.6177	31.7305
1200	30.5447	30.7118	31.2241	32.1143	33.4304
1500	31.9976	32.1887	32.7681	33.7486	35.1218
1800	33.5770	33.7736	34.3654	35.3631	36.7865

温度剖面由中心向表面平滑升高, 1800 s 时表面—中心温差为 $3.2095 \text{ }^\circ\text{C}$, 与中心升温滞后的预期一致。

6.3.2 水分场计算结果与分析

典型时刻水分浓度列于表 4。100 s 时表面水分已由 2.55 降至 2.2475 kg/kg , 而 $r \leq 1.0 \text{ cm}$ 区域在四位小数精度下仍保持初值。到 1800 s, 表面进一步降至 1.5104 kg/kg , 中心仍为 2.5500 kg/kg , 形成 1.0396 kg/kg 的中心—表面差。

温度和水分的中心—表面差见表 5, 预热阶段两者均增大。

图 3(a)、(b) 分别给出温度 and 水分径向剖面。表层失水使 $D(C)$ 降低, 内部水分迁移较慢, 因此表面水分不能代表中心状态。

表 4 问题一典型时刻水分浓度 (kg/kg)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8772
900	2.5500	2.5500	2.5497	2.5045	1.7549
1200	2.5500	2.5500	2.5482	2.4646	1.6587
1500	2.5500	2.5499	2.5445	2.4206	1.5789
1800	2.5500	2.5497	2.5383	2.3755	1.5104

表 5 问题一中心—表面差的演化

t/s	$T(R, t) - T(0, t)/^{\circ}\text{C}$	$C(0, t) - C(R, t)/(\text{kg/kg})$
100	0.1801	0.3025
300	0.8081	0.4989
600	1.7101	0.6728
900	2.4043	0.7951
1200	2.8857	0.8913
1500	3.1242	0.9711
1800	3.2095	1.0396

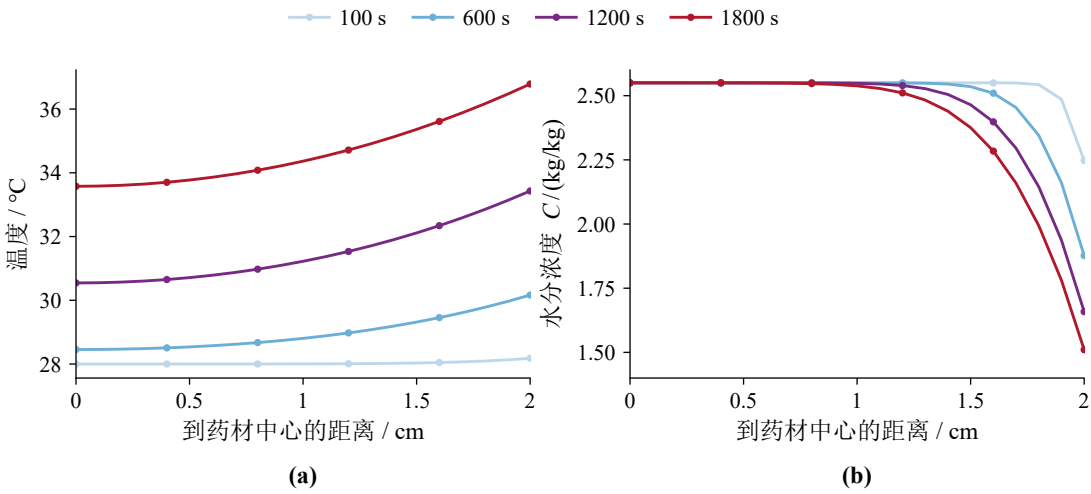


图 3 问题一温度与水分径向剖面

6.3.3 二维轴对称检验

为检验端面传递对中截面结果的影响，另建立半长圆柱轴对称有限体积模型。检验坐标取 $z = 0$ 为中截面、 $z = L/2$ 为端面，轴线与中截面零通量，侧表面和端面均施加热、质 Robin 边界。在 41 个径向、21 个半轴向节点及 10 s 步长下计算至 1800 s，与配对一维结果比较，全时段中截面最大温差为 $7.49 \times 10^{-7} \text{ }^\circ\text{C}$ 、最大水分差为 $7.35 \times 10^{-14} \text{ kg/kg}$ ，支持本问预热阶段的中截面径向简化。

6.3.4 预热温度的连续时空演化

图 4 展示 0–30 min 的温度时空分布及等温线，热量由表面逐渐传向中心。

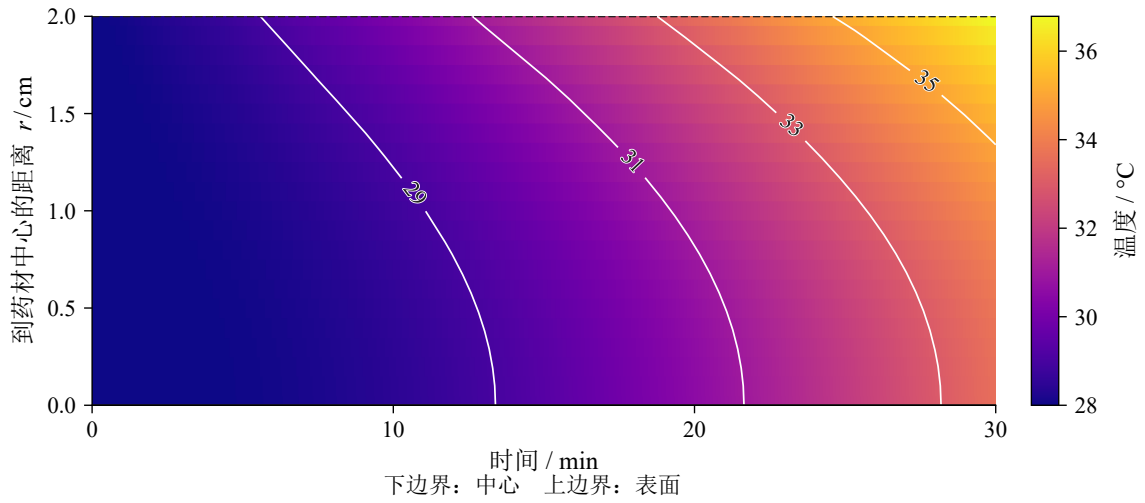


图 4 问题一预热温度的时空分布及等温线 (0–30 min)

七、问题二：变物性热湿迁移模型的建立与求解

7.1 变物性热湿迁移模型

7.1.1 非线性模型及耦合关系

问题二保持 $R = 0.02 \text{ m}$ ，并按表面交换假设沿用 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ ，将 (8) 代入 (2) 和 (3)。温度方程的储存项和扩散项依赖 C ，水分扩散系数同时依赖 T, C ，形成双向耦合；每次 Picard 迭代先解温度、再更新水分，求得同一时间层的自洽状态。

初始物性见表 6，与问题一所用常数不同，故相同几何和表面交换系数下仍有不同响应；两问的初段解不能混用。

7.1.2 物性随状态的演化

为展示非线性系数的实际变化，将初态以及 3 h 的中心、表面状态代入 (8)，结果见表 6。随着水分降低， ρ, c_p, k 均减小；这意味着单位体积储热能力降低，同时内部导热能力也有所下降。二者对升温速度具有相反影响，需要由所采用的热方程共同决定。

表 6 问题二代表状态下的物性参数

状态	$\rho/(\text{kg/m}^3)$	$c_p/[\text{J}/(\text{kg K})]$	$k/[\text{W}/(\text{m K})]$	$D/(\text{m}^2/\text{s})$
初始全域	976.4	3415.3	0.4830	5.642×10^{-9}
3 h 中心	876.1	3196.9	0.4526	1.239×10^{-8}
3 h 表面	779.0	2823.5	0.4008	1.027×10^{-8}

扩散系数同时受温度和水分影响。升温使 Arrhenius 因子 $\exp[-3850/(T + 273.15)]$ 增大，促进水分扩散；干燥使 $\exp(-0.45/C)$ 减小，抑制扩散。3 h 时两种效应叠加后，中心和表面 D 都高于初值；尽管表面温度略高，其扩散系数 D 却小于中心值。这说明低含水表层已开始形成扩散阻力，水分迁移并非随温度升高而无限加速。

7.2 模型的求解

7.2.1 变物性迭代与轨迹衔接

沿用 161 个表面加密节点和 1 s 步长，从 (1) 重新出发，采用附录 3 物性，不能以问题一的预热结果替代初段解。前 3 h 用于本问分析，随后沿同一轨迹继续积分，为问题三确定达标时间；4 h 后采用第五节的常值环境延拓。

按 (13) 分别组装温度、水分三对角矩阵 A_T, A_C ，界面平均及边界处理沿用第五节。以 $T^{(0)} = T^n, C^{(0)} = C^n$ 为初值，一次交替迭代为

$$A_T(C^{(m)})T^{(m+1)} = \text{diag}\left(\frac{\rho^{(m)}c_p^{(m)}V}{\Delta t}\right)T^n + b_T,$$

$$A_C(T^{(m+1)}, C^{(m)})C^{(m+1)} = \text{diag}\left(\frac{V}{\Delta t}\right)C^n + b_C.$$

其中 b_T, b_C 为环境交换项。先由当前水分更新热物性，再以新温度和当前水分更新 D ，温度指数统一采用开尔文温度；历史状态 T^n, C^n 保持不变，迭代收敛后才推进时间。

7.2.2 精度与计算检查

采用第五节的双重容差和逐步检查，实际最多 5 次 Picard 迭代即收敛，未触及 30 次上限。全隐式格式减轻稳定性限制，离散精度由问题三的长期收敛检验统一评估

(第8.3.3节)。

7.3 结果的分析 and 验证

7.3.1 温湿分布及内部迁移特征

典型时刻温度见表 7。0.5 h 时中心和表面分别为 32.1911 °C、35.4141 °C，仍有 3.2230 °C 的明显差异；1.5 h 时，温差降为 1.2928 °C；到 3 h 仅剩 0.1173 °C。这表明温度先经历表面快速升温 and 内部滞后，随后在环境逐渐稳定时趋于径向均匀。

表 7 问题二温度分布 (°C)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.1911	32.3840	32.9674	33.9620	35.4141
1	40.3822	40.5543	41.0609	41.8784	43.0001
1.5000	45.8464	45.9345	46.1927	46.6049	47.1393
2	48.4498	48.4877	48.5980	48.7732	49.0030
2.5000	49.4667	49.4790	49.5133	49.5645	49.6617
3	49.8493	49.8552	49.8746	49.9100	49.9666

水分结果见表 8。0.5 h 时中心仍为 2.5499 kg/kg，表面已降至 1.6487 kg/kg；3 h 时中心和表面分别降至 1.7662 和 1.0081 kg/kg。中心—表面差由 0.5 h 的 0.9012 降至 3 h 的 0.7581 kg/kg，说明水分扰动已经传播到中心，但内部尚未达到近似均匀状态。

表 8 问题二水分浓度分布 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5489	2.5255	2.3257	1.6487
1	2.5257	2.4947	2.3578	2.0230	1.4712
1.5000	2.3860	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6260	1.2311
2.5000	1.9567	1.9006	1.7360	1.4720	1.1167
3	1.7662	1.7165	1.5702	1.3333	1.0081

图 5(a)、(b) 分别给出温度 and 水分径向剖面。3 h 时温度已接近均匀，而中心水分仍为表面的约 1.75 倍，说明温度趋于均匀不能作为烘干结束的依据。

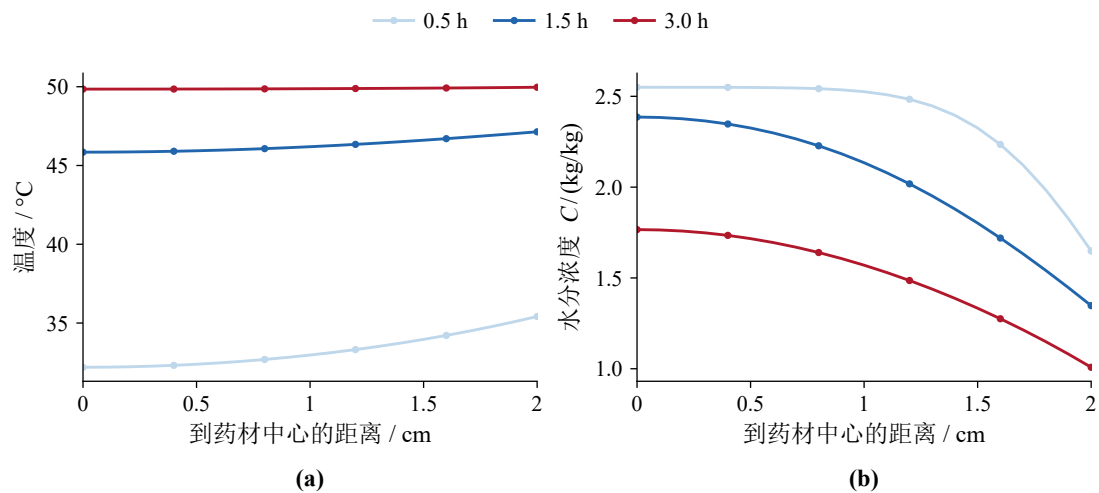


图 5 问题二变物性温度与水分剖面

7.3.2 水分迁移与扩散系数的时空证据

图 6 展示前 3 h 水分时空分布，失水影响由表面向中心传播。

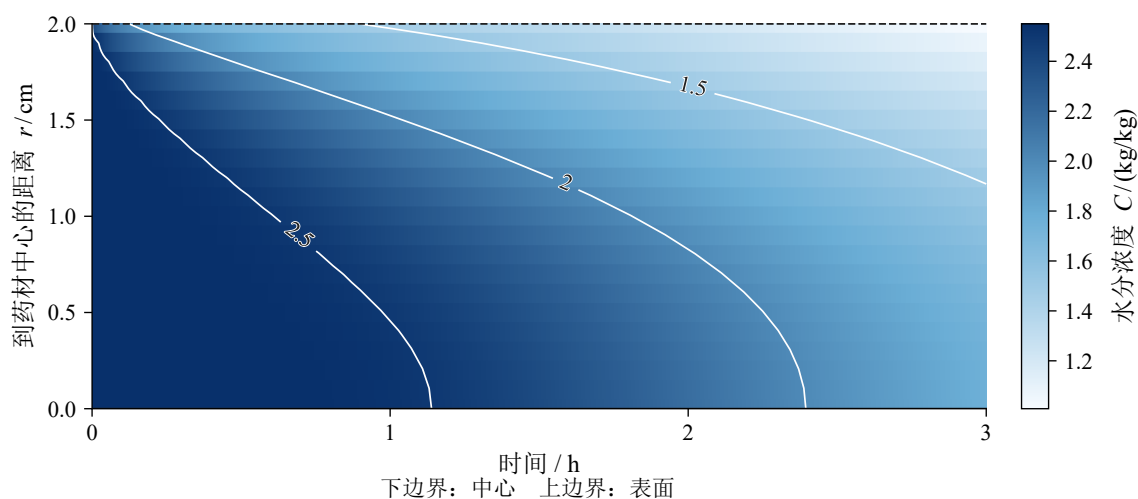


图 6 问题二前 3 h 水分浓度的时空分布及等浓度线

图 7 展示 $D(T, C)$ 的时空变化：升温增强扩散，但表层失水使其扩散系数低于内部。

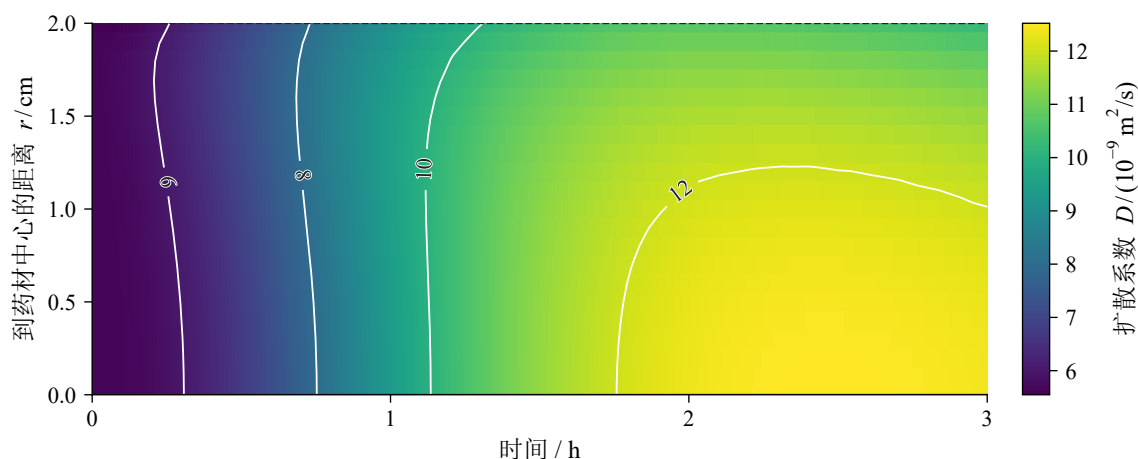


图 7 问题二状态相关扩散系数的时空云图及等值线

7.3.3 共享轨迹一致性检验

问题二、三采用相同物性、网格和边界连续计算。第二问逐秒温湿场按 60 s 抽取后，与第三问全部共同时间层逐元素一致，最大差为零。这一检查确认短期温湿响应与长期达标预测采用相同数值轨迹。

八、问题三：全截面达标时间模型的建立与求解

8.1 全截面达标时间模型

8.1.1 全域首达判据

问题三要求“所有位置”的水分浓度均低于 0.15 kg/kg，因此不能用截面平均值或表面值代替。定义空间最大水分浓度

$$M(t) = \max_{0 \leq r \leq R_0} C(r, t), \quad (19)$$

将严格达标时间集合的下确界定义为临界时刻

$$t_d = \inf\{t \geq 0 : M(t) < 0.15\}. \quad (20)$$

在本题连续下降的临界段， $M(t_d) = 0.15$ ，越过 t_d 后才满足严格低于；此前至少有一个位置未达标。本文报告该临界时刻，并另核对首个严格达标时间层，避免将输出间隔或四舍五入值作为停机判据。

初始水分场均匀，表面环境水分始终显著低于药材内部，且有限体积矩阵是对角占优的 M 型结构。在这种向外失水的条件下，数值解应保持由中心向表面非增。程序对

每一个内部时间层检查

$$C_i^n - C_{i+1}^n \geq -2 \times 10^{-9}, \quad i = 0, 1, \dots, N-1, \quad (21)$$

全部时间层均通过。因此

$$M(t) = C(0, t), \quad (22)$$

即轴线是本问的控制位置。这里并非预先假定中心最大，而是先保留全域最大值判据，再由计算结果确认其可化为中心事件。

8.2 模型的求解

8.2.1 长期积分与阈值事件定位

沿用问题二的 161 节点网格、1 s 步长及附录 3 物性，从初态连续推进，不在 3 h 处重新初始化。附件区间内采用 PCHIP 环境值，4 h 后取 $T_e = 50^\circ\text{C}$ 、 $C_e = 0.05 \text{ kg/kg}$ 。每步完成 Picard 迭代并检查 (21)，以全域最大水分搜索首达区间。

首次出现 $M^n > 0.15$ 且 $M^{n+1} \leq 0.15$ 时，在相邻时间层间定义

$$\lambda = \frac{0.15 - M^n}{M^{n+1} - M^n}, \quad t_d = t_n + \lambda \Delta t, \quad (23)$$

并同步插值得到临界剖面

$$C(r, t_d) = C^n(r) + \lambda[C^{n+1}(r) - C^n(r)]. \quad (24)$$

由于 $0 \leq \lambda \leq 1$ ，所得时刻位于跨越阈值的 1 s 区间内。时间精度由积分步长和事件插值决定，60 s 取值间隔只用于结果展示。

8.2.2 达标剖面与输出核验

以同一权重插值温湿剖面，最大水分等于阈值；首个严格达标整数秒为 206899 s。每 60 s 的分布由同一轨迹取值，并覆盖临界时刻后的首个整分钟；末行显示 0.1500 不代表未舍入值等于阈值。

8.3 结果的分析 and 验证

8.3.1 干燥时间与径向分布

由 (23) 得到

$$t_d = 206898.8710 \text{ s} = 57.4719 \text{ h}. \quad (25)$$

典型时刻及临界时刻的水分浓度见表 9。中心从 6 h 的 1.0170 降至 12 h 的 0.4562，随后下降速度逐渐减慢；临界时刻中心恰为 0.1500，而表面已降至 0.0527 kg/kg，接近模型采用的稳定环境边界值。

表 9 问题三每 6 h 及达标时的水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4562	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4719	0.1500	0.1477	0.1402	0.1249	0.0527

中心、表面轨迹及径向剖面见图 8。左图用水平虚线标出全域达标阈值，右图给出 6、18、36、48 h 及达标瞬间的径向剖面。表面于 12.5525 h 达标，比中心提前 44.9194 h，应以内部最大水分决定停机时刻。

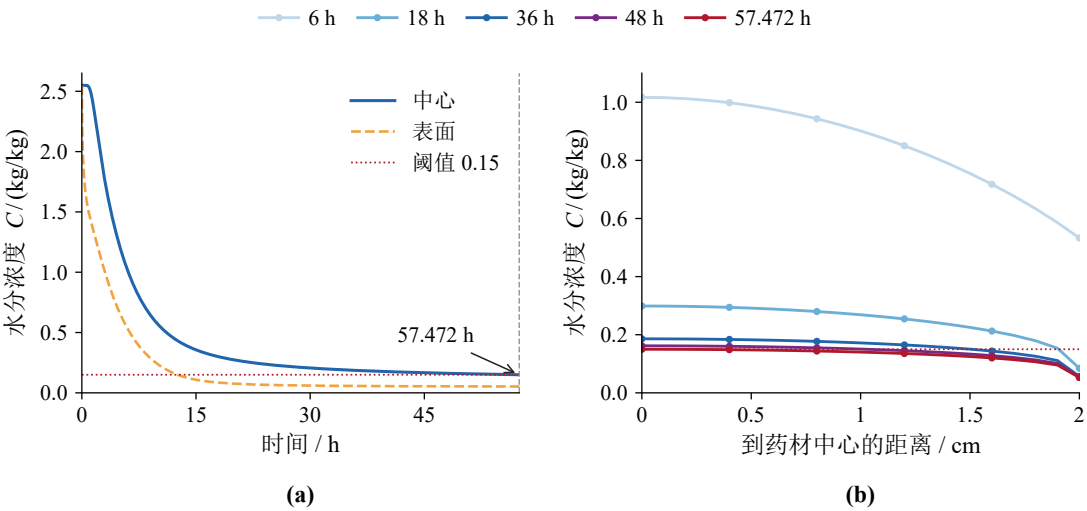


图 8 问题三中心、表面水分轨迹及典型时刻径向分布

低含水状态下扩散系数减小，后期干燥明显变慢：48 h 中心水分为 0.1618，仍需约 9.47 h 才能达到阈值。

8.3.2 达标区域由表面向中心的推进

图 9 显示严格达标区域由表面向中心推进，未达标核心最终于 57.472 h 消失。红线为浓度阈值边界，不是相变界面。

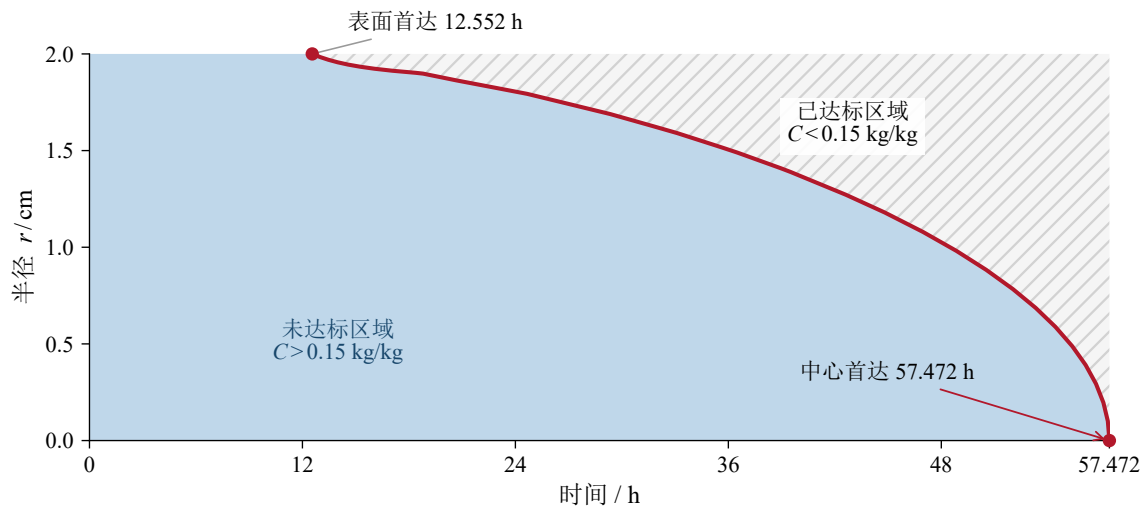


图 9 问题三已达标区域与未达标区域的演化；红线为含水率阈值边界

8.3.3 空间网格与时间步长检验

先固定 10 s 步长，比较 41、81、161、321 个径向节点，再固定 161 个节点将步长由 2 s 减为 1 s，结果见表 10。空间与时间加密的达标时间相对变化分别为 1.65×10^{-5} 、 1.52×10^{-5} ，均小于预设空间 10^{-3} 、时间 10^{-4} 的相对容差。这些差值表示离散加密的经验变化量，不是严格误差上界。

表 10 问题三网格与步长收敛检验：达标时间（h）

计算设置	问题三
41 节点，10 s	57.461591
81 节点，10 s	57.476003
161 节点，10 s	57.479757
321 节点，10 s	57.480706
161 节点，2 s	57.472781
161 节点，1 s（终算）	57.471909

8.3.4 端面影响的二维检验

沿用问题一的半长圆柱二维边界设置，将计算延伸至全二维区域达标，统一采用 30 s 步长，并与相同径向网格和步长的一维计算配对。41×21 节点下，二维达标时间

为 57.476411 h，配对相对差为 4.55×10^{-5} ；81×41 节点下为 57.491233 h，相对差为 3.84×10^{-5} 。两种网格的最大水分均位于轴线与中截面交点。

41×21 网格下，中截面最大温差为 0.001775 °C，最大水分差为 2.41×10^{-5} kg/kg；端面与中截面最大水分差约为 0.970 kg/kg。因此中部径向模型能够描述控制达标的位置，但不能代替端面局部状态。二维检验步长与 1 s 终算不同，其时长不用于替换主模型答案。

九、问题四：收缩移动边界模型的建立与求解

9.1 收缩移动边界模型

9.1.1 半径数据重构与固定域映射

附件二给出的半径由初始 2 cm 持续减小至 1.2 cm。经单调 PCHIP 重构后，在本问主要时刻的半径见表 11。前 6 h 半径已经缩小到 1.374 cm，完成总收缩量的 78.3%；此后收缩速度逐渐减小，从 36 h 至本问达标时刻，重构半径保持为 1.200 cm。由此可见，几何变化主要集中在干燥早期，但它对随后全部扩散过程都会持续产生影响。

表 11 问题四典型时刻的重构半径

t/h	0	6	12	18	24	30	36	48	51.088
R/cm	2.000	1.374	1.248	1.214	1.204	1.201	1.200	1.200	1.200

若直接在物理域 $0 \leq r \leq R(t)$ 上使用固定的 0.1 cm 网格，节点数会随时间减少，旧节点场量还需频繁插值到新网格，不利于保持守恒。为此定义无量纲固定域坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (26)$$

并令 $\psi(\xi, t) = \phi(r, t)$ 。半径收缩时，固定 ξ 的节点随材料截面同步向内移动，始终保留 161 个表面加密计算节点，不需要改变矩阵维数。

9.1.2 收缩模型方程

食品等温干燥中的收缩效应可采用移动边界模型描述 [8]。

移动边界模型需同时指定内部速度与边界运动 [2, 6, 7]。半径数据只能确定外边界，本问另假设长度不变、径向同比例收缩以闭合速度场，材料速度 v_s 与网格速度 w 相同，故

$$v_s = w = \frac{\dot{R}}{R}r, \quad D_t = \frac{\partial}{\partial t} \Big|_r + v_s \frac{\partial}{\partial r} = \frac{\partial}{\partial t} \Big|_\xi. \quad (27)$$

其中 D_t 为材料导数。令 ρ_d 表示单位现体积的干骨架质量，它不同于题给物性中的 $\rho(C)$ 。干骨架连续性与干基含水率 C 的水分平衡为

$$\begin{aligned}\frac{\partial \rho_d}{\partial t} + \frac{1}{r} \frac{\partial}{\partial r} (r \rho_d v_s) &= 0, \\ \frac{\partial(\rho_d C)}{\partial t} + \frac{1}{r} \frac{\partial}{\partial r} (r \rho_d C v_s) &= \frac{1}{r} \frac{\partial}{\partial r} \left(r \rho_d D \frac{\partial C}{\partial r} \right).\end{aligned}$$

截面内 ρ_d 均匀时，第一式给出 $\rho_d R^2 = \rho_{d0} R_0^2$ ；以其消去第二式的体积压缩项，再约去仅随时间变化的 ρ_d ，得到材料坐标下的水分式。故收缩不直接使干基含水率增大，也无需另加相对对流项。温度沿用本文的热过程简化，以材料导数描述随材料运动的温度变化。记 T, C 为映射后的场，两式写为

$$\begin{aligned}\rho(C) c_p(C) \frac{\partial T}{\partial t} &= \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k(C) \frac{\partial T}{\partial \xi} \right), \\ \frac{\partial C}{\partial t} &= \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(T, C) \frac{\partial C}{\partial \xi} \right).\end{aligned}\tag{28}$$

温度式中 $\rho(C) c_p(C)$ 仅作为储热系数，未要求题给 $\rho(C)$ 满足上述干骨架连续性；该式不含蒸发潜热及水分迁移携带的焓，不能视为完整焓平衡。质量守恒推导说明水分方程与既定收缩闭合相容，不构成对均匀变形的实验验证。

表面边界也必须同步变换。由 $\partial/\partial r = R^{-1} \partial/\partial \xi$ 得

$$-\frac{k}{R} \frac{\partial T}{\partial \xi} \Big|_{\xi=1} = h_T (T_s - T_e),\tag{29}$$

$$-\frac{D}{R} \frac{\partial C}{\partial \xi} \Big|_{\xi=1} = h_m (C_s - C_e).\tag{30}$$

因此在无量纲控制体方程中，内部扩散导通系数乘 R^{-2} ，表面对流导通系数乘 R^{-1} 。若只缩放网格而遗漏这些尺度因子，模型将不能正确反映半径变化带来的通量放大。

9.2 模型的求解

9.2.1 材料坐标离散与迭代流程

在 $\xi_i = 1 - (1 - i/160)^2$ 上采用 161 个表面加密节点，时间步长取 1 s。每个时间层先由 PCHIP 更新半径和环境边界，再更新 R^{-2} 内部扩散项及 R^{-1} 表面交换项；随后采用 Picard 迭代依次求解温度场和水分场。全域最大水分浓度跨越 0.15 kg/kg 时停止搜索，并对相邻时间层线性插值定位临界时刻；首个严格低于阈值的整数秒另行核对。计算流程见图 10。

为分离几何收缩作用，另保持题面附录 4 的物性和环境不变，将半径固定为 2 cm，

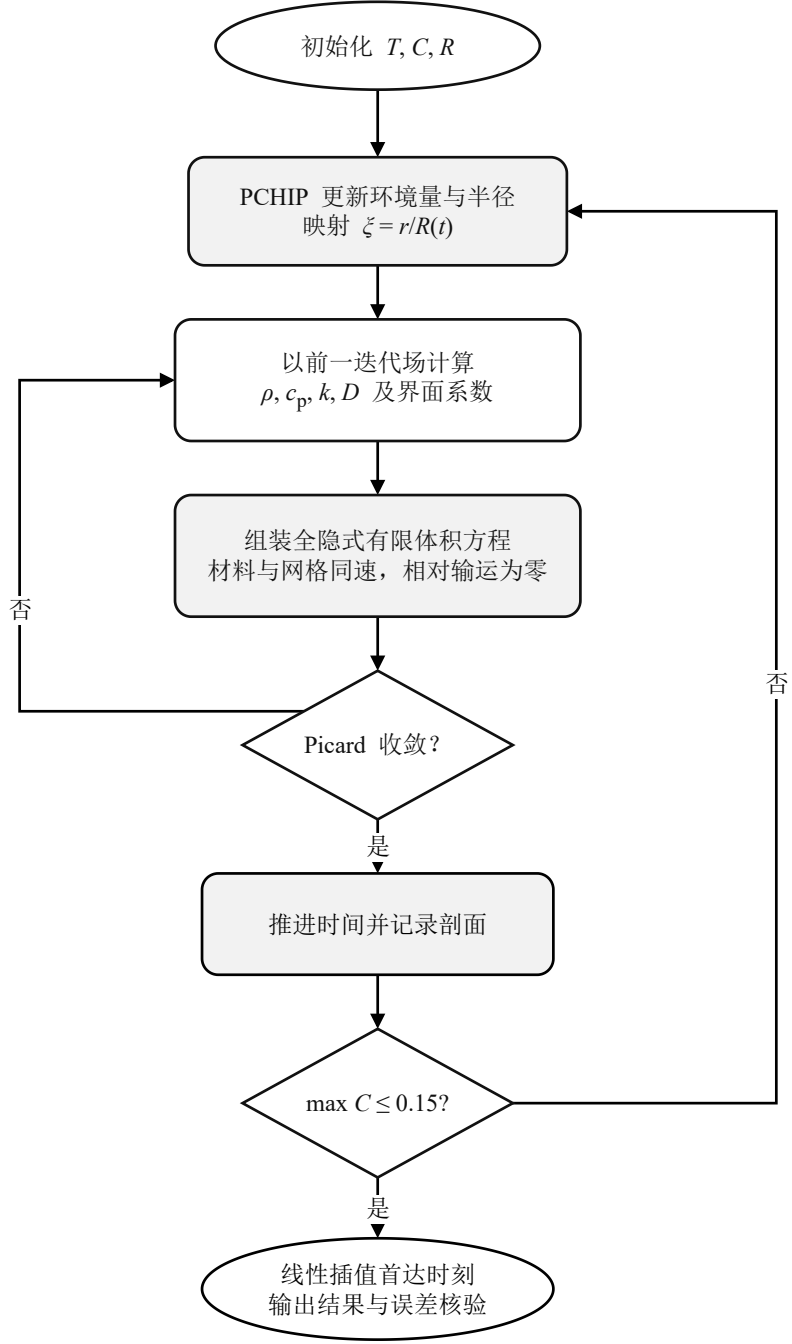


图 10 问题四移动边界全隐式迭代流程

按相同步长与首达判据求解对照方案。

9.2.2 物理位置输出规则

计算解首先定义在固定材料坐标 ξ 上, 而题目模板给出的列对应固定物理位置 $r_j = 0, 0.1, \dots, 1.9 \text{ cm}$ 。对任一时刻, 仅当 $r_j \leq R(t)$ 时计算

$$C(r_j, t) = \mathcal{I}_\xi[C(\xi, t)]|_{\xi=r_j/R(t)}, \quad (31)$$

其中 $\mathcal{I}_\xi$ 直接使用全部 161 个计算节点作线性插值；当 $r_j > R(t)$ 时，该位置已经处于药材外部，单元格留空而不是填 0。最后一列始终记录 $C(\xi = 1, t)$ ，即随半径移动的实时表面。

该规则区分固定物理位置和随材料移动的表面位置，避免将药材外部的环境量误认为材料状态。

9.3 结果的分析 and 验证

9.3.1 达标时间与浓度分布

问题四继续以全域最大水分浓度作为终止量。全部剖面保持中心最大，故事件定位仍可用 (23) 和 (24)。计算得到

$$t_d = 183915.7175 \text{ s} = 51.0877 \text{ h}, \quad R(t_d) = 1.2000 \text{ cm}. \quad (32)$$

该值为最大水分等于阈值的临界时刻；首个严格达标整数秒为 183916 s。典型时刻的固定物理位置与动态表面水分见表 12。

表 12 问题四典型时刻水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	动态表面
6	1.7188	1.5370	1.0221	0.4205
12	0.7374	0.6544	0.4082	0.1669
18	0.4087	0.3686	0.2397	0.0892
24	0.2851	0.2610	0.1790	0.0673
30	0.2263	0.2094	0.1492	0.0595
36	0.1928	0.1796	0.1316	0.0560
42	0.1712	0.1604	0.1200	0.0541
48	0.1561	0.1468	0.1116	0.0530
51.0877	0.1500	0.1413	0.1081	0.0526

6 h 时中心水分仍为 1.7188，动态表面已降至 0.4205 kg/kg；24 h 时中心为 0.2851，表面只有 0.0673；临界时刻中心为 0.1500，表面为 0.0526。

动态表面在 12.7848 h 已达到 0.15，而中心需要 51.0877 h，两者仍相差 38.3029 h。收缩虽然显著缩短了内部扩散距离，却没有消除“外干内湿”的过程特征，因此实际控制仍应关注中心或内部代理量。

9.3.2 收缩效应的反事实分解

问题三和问题四的物性函数不同，直接比较 57.4719 h 与 51.0877 h 不能将全部差异归因于收缩。为保持物性和环境相同，另作两组对照：

1. 使用题面附录 4 的物性和相同环境，但保持 $R = 2 \text{ cm}$ ，得到固定半径反事实时长 129.8445 h；
2. 保留真实 $R(t)$ ，采用原固定空间标量方程，在 ξ 坐标下保留系数为 $-\dot{R}\xi/R$ 的对流项，得到 52.6375 h，仅用作模型结构比较。

第一组在本文材料收缩闭合下给出

$$\Delta t_R = 129.8445 - 51.0877 = 78.7568 \text{ h}, \quad \eta_R = \frac{\Delta t_R}{129.8445} = 60.65\%. \quad (33)$$

第二组与材料模型相差 1.5498 h，表明速度描述影响长期预测。该差异不能解释为独立“坐标移动效应”，也不能据此认定任一方程已被实验验证。固定半径与均匀收缩模型的中心水分轨迹及半径变化见图 11。

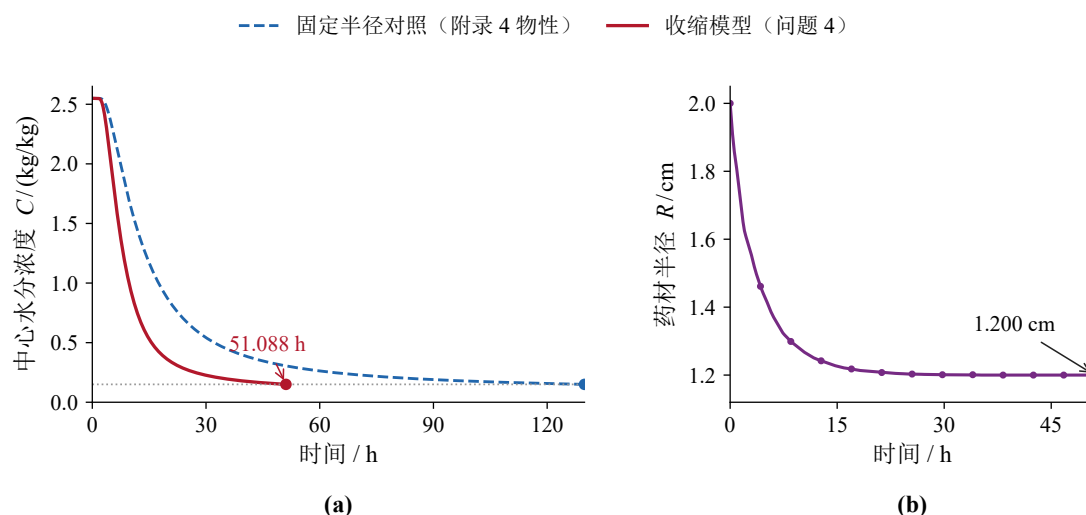


图 11 问题四收缩与固定半径反事实比较

9.3.3 收缩材料域与截面水分的联合展示

图 12(a) 给出收缩边界与水分场；(b) 为径向解的轴对称截面重建，各截面共用长度比例和色标。几何收缩先于深部水分达标，重建截面不属于独立二维计算。

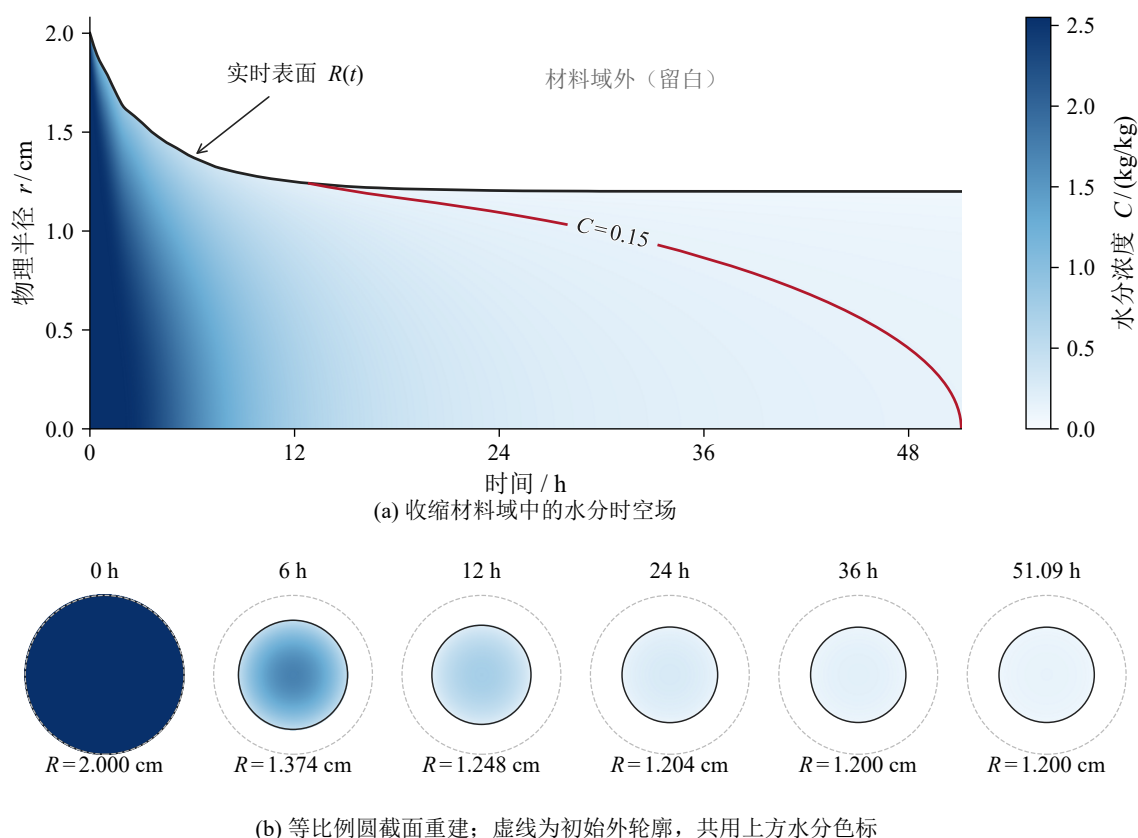


图 12 问题四收缩材料域的水分时空场及等比例截面重建

9.3.4 COMSOL 有限元复核

COMSOL 采用相同物性、环境与半径历史，并设置轴向均匀、端面绝热。在 6、12、24、36、48 h 及主模型首达时刻，分别取 1001 个共同径向位置，将有限元解与由原始 161 节点线性插值得到的有限体积解比较。表 13 统一列出这一采样口径下的网格与容差检验，表中加密单元数指径向单元数。初始自动网格下最大水分差约为 $1.17 \times 10^{-2} \text{ kg/kg}$ ；径向加密至 320 单元、相对求解容差收紧至 10^{-7} 后，六个对照时刻的最大水分差为 $4.54 \times 10^{-5} \text{ kg/kg}$ 。图 13 采用该加密设置，展示径向剖面、采样点对照及半径历程。主模型首达时刻 51.0877 h 对应的 COMSOL 中心水分 0.1499888 kg/kg，该值不是 COMSOL 独立首达时间。上述检验支持既定模型的数值一致性，不替代对物理假设的实验验证。

表 13 COMSOL 网格与求解容差检验

网格设置	相对求解容差	最大水分差 (kg/kg)
初始自动网格	1.00×10^{-4}	1.17×10^{-2}
初始自动网格, 收紧容差	1.00×10^{-6}	1.17×10^{-2}
表层加密, 80 单元	1.00×10^{-6}	4.52×10^{-5}
表层加密, 160 单元	1.00×10^{-6}	4.53×10^{-5}
表层加密, 320 单元	1.00×10^{-6}	4.19×10^{-5}
320 单元, 收紧容差	1.00×10^{-7}	4.54×10^{-5}

COMSOL 物理场剖面、独立数值校核与收缩历程

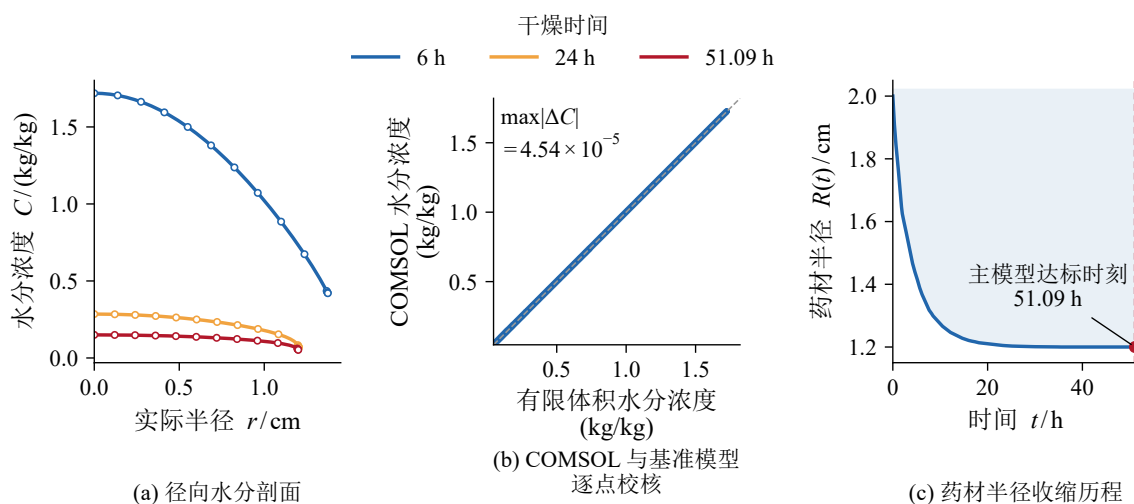


图 13 加密 COMSOL 有限元与有限体积解的剖面对照及半径收缩历程

9.3.5 移动网格独立验证

另在物理坐标中建立移动控制体模型, 以相同物性、环境、时间步长和首达判据独立计算, 与固定材料坐标解比较, 结果见表 14 和图 14。

表 14 移动网格与固定域映射的数值比较

指标	固定域映射法	移动网格法	相对差
t_d/h	51.087699300	51.087699300	5.81×10^{-13}
6 h, 中心	1.718798385	1.718798385	1.68×10^{-14}
6 h, 动态表面	0.420513845	0.420513845	2.92×10^{-13}
24 h, 中心	0.285074422	0.285074422	1.29×10^{-13}
24 h, 动态表面	0.067330674	0.067330674	4.91×10^{-14}
48 h, 中心	0.156140474	0.156140474	2.83×10^{-13}
48 h, 动态表面	0.053014141	0.053014141	1.11×10^{-13}
t_d^{map} , 中心	0.150000000	0.150000000	3.66×10^{-13}
t_d^{map} , 动态表面	0.052619962	0.052619962	8.68×10^{-14}

水分单位为 kg/kg；相对差以固定域结果为分母。末两行为同一固定域达标时刻的比较。

两法达标时间相对差为 5.81×10^{-13} ，全水分场最大差为 2.61×10^{-13} kg/kg、温度场最大差为 1.16×10^{-10} °C，并通过几何守恒检查。

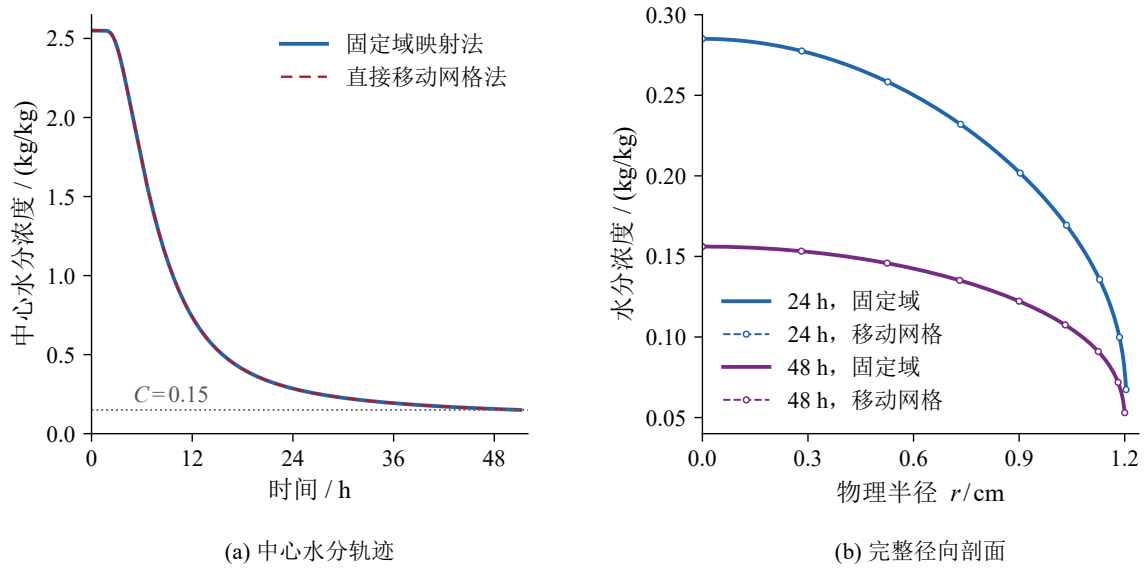


图 14 两种坐标实现的中心水分轨迹及物理半径剖面对比；曲线在图示精度下重合，圆点仅用于区分移动网格曲线

两种坐标实现的差异处于浮点舍入量级，支持程序实现的一致性；该比较不替代对收缩假设的实验验证。

9.3.6 空间网格与时间步长检验

按问题三的加密方案对收缩模型复算，结果见表 15。该检验用于评估主模型的空间和时间离散稳定性；161 至 321 节点的达标时间相对变化为 5.77×10^{-7} ，2 s 至 1 s 的变化为 1.29×10^{-5} ，满足预设容差。累计归一化水分方程残差为 1.77×10^{-13} ，有效热方程最大单步残差为 3.21×10^{-14} ；后者仅针对不含蒸发潜热的既定温度方程。表中数值与正文的一致性以计算记录为准。

表 15 问题四网格与步长收敛检验：达标时间 (h)

计算设置	问题四
41 节点, 10 s	51.093100
81 节点, 10 s	51.093511
161 节点, 10 s	51.093626
321 节点, 10 s	51.093656
161 节点, 2 s	51.088358
161 节点, 1 s (终算)	51.087699

9.3.7 端面影响的二维检验

沿用问题一的半长圆柱轴对称模型，在侧表面与端面施加热、质 Robin 边界，并加入给定半径历史与均匀径向收缩。采用 30 s 步长，41×21 节点下二维达标时间为 51.106259 h，与配对一维计算相对差为 1.86×10^{-7} ；加密至 81×41 节点后为 51.106671 h，相对差为 1.59×10^{-7} 。

全二维最大水分均位于轴线与中截面交点。41×21 网格下，全过程中截面最大温差为 0.000108 °C，最大水分差为 1.38×10^{-6} kg/kg；最大端面与中截面水分差约为 1.707 kg/kg。该检验支持当前端面条件下的中心控制，主模型仍采用 1 s 终算时长。

9.3.8 环境边界灵敏度分析

保持前 4 h 实测边界不变，分别对后续 $T_{e,\infty} = 50$ °C 施加 ± 2.5 °C 扰动、对 $C_{e,\infty} = 0.05$ kg/kg 施加 $\pm 5\%$ 扰动。问题三、四均采用 161 节点、10 s 步长；问题四保持给定半径历史 $R(t)$ 不变，结果见表 16。

用温度对称差分导数与水分弹性衡量影响：

$$K_T = \frac{t_d(T_e + \delta T) - t_d(T_e - \delta T)}{2\delta T}, \quad S_C = \frac{t_d(1.05C_e) - t_d(0.95C_e)}{0.10t_d(C_e)}. \quad (34)$$

表 16 问题三、四环境边界扰动下的干燥时长 (h)

情形	问题三	问题四
基准 (10 s)	57.4798	51.0936
恒温阶段温度 -2.5°C	62.3098	55.3270
恒温阶段温度 $+2.5^{\circ}\text{C}$	53.1605	47.3025
恒温阶段环境水分 -5%	57.4320	51.0413
恒温阶段环境水分 $+5\%$	57.5322	51.1531

问题三、四的 K_T 分别为 -1.8299 、 $-1.6049 \text{ h}^{\circ}\text{C}$ ， S_C 分别为 0.0174 、 0.0219 ；以开尔文温度定义 $S_T = (T_{e,K}/t_d)K_T$ ，分别得 -10.2874 、 -10.1504 。两模型均表现为升温缩短干燥时间、提高环境水分延长时间，且温度影响更明显。

温度扰动下，问题三、四的时长范围分别为 $53.16\text{--}62.31 \text{ h}$ 、 $47.30\text{--}55.33 \text{ h}$ 。这是给定边界扰动下的情景范围，不是统计置信区间；问题四未计入环境变化对收缩历史的反馈。常值延拓仍是预测的重要条件，数值收敛不等于实际时长具有同等确定性。

十、模型的评价与推广

10.1 模型的评价

模型的优点：

1. 四问采用统一的热湿迁移框架，依次引入状态相关物性、全域首达判据与材料收缩，能够衔接短期预热和长期干燥预测。
2. 有限体积法直接平衡控制体通量，全隐式与 Picard 迭代适应非线性物性；网格、步长、二维及交叉求解检验支持数值结果的可靠性。
3. 全域阈值判据识别中心控制位置，同物性固定半径对照分离几何收缩效应，便于解释干燥时间差异。

模型的局限：

1. 中截面近似获得同物性、同边界二维对照的支持，但材料各向异性及蒸发潜热的影响尚未量化；既有数值验证不能替代对这些物理简化的检验。
2. 环境水分采用等效映射，交换系数沿用题给值，长期边界取常值延拓；实际应用需标定浓度转换与表面交换关系，并将后续工况变化纳入预测范围。
3. 收缩模型以给定半径历史和均匀径向缩放闭合内部运动。水分守恒和坐标实现对照支持这一闭合的数值自洽性，实际局部变形、长度变化及完整能量平衡仍需另行验证。

10.2 模型的推广

结合少量中心温湿测量，可标定扩散与表面交换参数，利用模型估计内部达标时刻，并进一步加入能耗、品质和设备约束以优化干燥温度策略。

对短柱或端面交换显著的材料，可采用二维轴对称模型；获得潜热和吸湿数据后，可扩展为 Luikov 型热湿耦合模型 [1]。经物性与边界重新标定，该框架可用于果蔬、木材等多孔材料的干燥预测。

AI 工具使用声明

本文使用 ChatGPT 等 AI 工具辅助思路讨论、局部公式检查、程序实现与调试、结果核对及图文排版。数值结果由程序计算获得，参赛队负责模型方案的确定、建议的取舍和结果审核，并对论文内容承担最终责任。具体情况见支撑材料《AI 工具使用详情》。

参考文献

- [1] Luikov A V. Systems of differential equations of heat and mass transfer in capillary-porous bodies (review)[J]. International Journal of Heat and Mass Transfer, 1975, 18(1): 1–14. DOI: 10.1016/0017-9310(75)90002-2.
- [2] Adrover A, Brasiello A, Ponso G. A moving boundary model for food isothermal drying and shrinkage: General setting[J]. Journal of Food Engineering, 2019, 244: 178–191. DOI: 10.1016/j.jfoodeng.2018.09.018.
- [3] Dietl C, Winter E R F, Viskanta R. An efficient simulation of the heat and mass transfer processes during drying of capillary porous, hygroscopic materials[J]. International Journal of Heat and Mass Transfer, 1998, 41(22): 3611–3625. DOI: 10.1016/S0017-9310(97)00357-8.
- [4] Salagnac P, Glouannec P, Lecharpentier D. Numerical modeling of heat and mass transfer in porous medium during combined hot air, infrared and microwaves drying[J]. International Journal of Heat and Mass Transfer, 2004, 47(19–20): 4479–4489. DOI: 10.1016/j.ijheatmasstransfer.2004.04.015.
- [5] Purlis E. Modelling convective drying of foods: A multiphase porous media model considering heat of sorption[J]. Journal of Food Engineering, 2019, 263: 132–146. DOI: 10.1016/j.jfoodeng.2019.05.028.
- [6] Lentzou D, Boudouvis A G, Karathanos V T, Xanthopoulos G. A moving boundary model for fruit isothermal drying and shrinkage: An optimization method for water diffusivity and peel resistance estimation[J]. Journal of Food Engineering, 2019, 263: 299–310. DOI: 10.1016/j.jfoodeng.2019.07.010.
- [7] Zhu Y, Wang P, Sun D, Qu Z, Yu B. Multiphase porous media model with thermo-hydro and mechanical bidirectional coupling for food convective drying[J]. International Journal of Heat and Mass Transfer, 2021, 175: 121356. DOI: 10.1016/j.ijheatmasstransfer.2021.121356.
- [8] Adrover A, Brasiello A. The Role of Shrinkage on Food Isothermal Drying: a Moving Boundary Model[J]. Chemical Engineering Transactions, 2019, 74: 1111–1116. DOI: 10.3303/CET1974186.

附录 A 支撑材料文件列表

以下路径均相对于支撑材料压缩包的解压根目录。程序运行环境、执行顺序及可重建文件见包内运行说明。

表 17 支撑材料及作用

文件（相对支撑包根目录）	作用
<code>code/solve_drying.py</code>	四问热湿迁移求解、首达判定及收缩模型
<code>code/write_results.py</code>	四份结果工作簿导出与逐项回读核验
<code>code/check_numerics.py</code>	空间网格与时间步长收敛检验
<code>code/validate_2d.py</code>	问题一轴对称二维端面影响检验入口
<code>code/validate_long_2d.py</code>	长期二维与一维配对计算
<code>code/verify_revision.py</code>	轨迹一致性、场量与离散平衡检查
<code>code/run_sensitivity.py</code>	长期环境边界灵敏度计算
<code>code/run_q4_counterfactual.py</code>	同物性固定半径反事实计算
<code>code/validate_moving_mesh.py</code>	独立移动控制体模型与几何守恒检验
<code>code/check_moving_mesh_delivery.py</code>	移动网格验证结果一致性检查
<code>code/figure_style.py</code>	绘图字体、数学排版与缺字检查
<code>code/make_figures.py</code>	四问主要数值结果图绘制
<code>code/make_additional_figures.py</code>	时空场、扩散系数和收缩截面图绘制
<code>code/plot_moving_mesh_validation.py</code>	移动网格与固定域结果对照图绘制
<code>code/make_geometry_diagram.py</code>	药材几何与坐标示意图绘制
<code>code/run_figures.sh</code>	绘图批处理入口及字体环境检查
<code>inspect_inputs.py</code>	题目附件与结果模板结构检查
<code>comsol_q1/prepare_inputs.py</code>	问题一有限元输入准备与结果比较
<code>comsol_q1/Q1Automation.java</code>	问题一有限元建模、求解和剖面导出
<code>comsol_q1/run_comsol_q1.ps1</code>	问题一有限元编译、运行与比较入口
<code>comsol_q4/prepare_q4_inputs.py</code>	问题四有限元物性与边界输入准备
<code>comsol_q4/Q4Automation.java</code>	问题四收缩有限元建模、求解和剖面导出
<code>comsol_q4/analyze_q4_comsol.py</code>	有限元与有限体积数值结果比较
<code>comsol_q4/plot_q4_physical.py</code>	有限元物理剖面和水分场绘制
<code>comsol_q4/near_surface_check/analyze_refinement.py</code>	有限元近表面加密结果分析
<code>comsol_q4/near_surface_check/write_report.py</code>	有限元网格与容差检验结果汇总
<code>comsol_q4/near_surface_check/run_cases.ps1</code>	六组有限元网格与容差算例运行入口
<code>comsol_q4/near_surface_check/original_dense/Q4Automation.java</code>	原始网格加密输出检验
<code>comsol_q4/near_surface_check/original_tight/Q4Automation.java</code>	原始网格收紧求解容差检验

续下页

表 17 支撑材料及作用（续）

文件（相对支撑包根目录）	作用
comsol_q4/near_surface_check/graded80/ Q4Automation.java	近表面分级网格 80 单元检验
comsol_q4/near_surface_check/graded160/ Q4Automation.java	近表面分级网格 160 单元检验
comsol_q4/near_surface_check/graded320/ Q4Automation.java	近表面分级网格 320 单元检验
comsol_q4/near_surface_check/graded320_tight/ Q4Automation.java	近表面分级网格 320 单元及严格容差检验
support_manifest.json	逐文件来源、哈希及可重建资料清单
AI 工具使用详情.pdf	AI 工具使用情况说明
README.md	支撑材料运行及文件核验说明
comsol_q1/ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q1/ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q1/comparison.json	COMSOL 输入、导出数据或数值比较记录
comsol_q1/input_metadata.json	COMSOL 输入、导出数据或数值比较记录
comsol_q1/q1_comparison.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q1/q1_comsol_profiles.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q4/ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/comparison.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/comparison_ fields.npz	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded160/ ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded160/ ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded160/radius_ history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320/ ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320/ ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320/radius_ history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320_tight/ ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320_tight/ ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录

续下页

表 17 支撑材料及作用（续）

文件（相对支撑包根目录）	作用
comsol_q4/near_surface_check/graded320_tight/ q4_comsol_profiles.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded320_tight/ radius_history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded80/ambient_ moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded80/ambient_ temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/graded80/radius_ history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/manifest.json	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_dense/ ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_dense/ ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_dense/ radius_history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_tight/ ambient_moisture.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_tight/ ambient_temperature.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/original_tight/ radius_history.txt	COMSOL 输入、导出数据或数值比较记录
comsol_q4/near_surface_check/summary.json	COMSOL 输入、导出数据或数值比较记录
comsol_q4/q4_comsol_comparison.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q4/q4_comsol_fields.npz	COMSOL 输入、导出数据或数值比较记录
comsol_q4/q4_comsol_profiles.csv	COMSOL 输入、导出数据或数值比较记录
comsol_q4/q4_comsol_validation.json	COMSOL 输入、导出数据或数值比较记录
comsol_q4/radius_history.txt	COMSOL 输入、导出数据或数值比较记录
paper/cumcm-1.1.0/main.pdf	当前完整论文；供附录与结果核对
results/data_audit.json	已保存的计算场、汇总结果或验证记录
results/long_2d_verification.json	已保存的计算场、汇总结果或验证记录
results/numerical_verification.json	已保存的计算场、汇总结果或验证记录
results/paper_revision_verification.json	已保存的计算场、汇总结果或验证记录
results/paper_table_audit.json	已保存的计算场、汇总结果或验证记录
results/q1/fields.npz	已保存的计算场、汇总结果或验证记录
results/q1/summary.json	已保存的计算场、汇总结果或验证记录
results/q1/validation_2d.json	已保存的计算场、汇总结果或验证记录

续下页

表 17 支撑材料及作用（续）

文件（相对支撑包根目录）	作用
results/q2/summary.json	已保存的计算场、汇总结果或验证记录
results/q3/fields.npz	已保存的计算场、汇总结果或验证记录
results/q3/summary.json	已保存的计算场、汇总结果或验证记录
results/q4/fields.npz	已保存的计算场、汇总结果或验证记录
results/q4/output_verification.json	已保存的计算场、汇总结果或验证记录
results/q4/summary.json	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/comparison.csv	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/delivery_checks.json	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/profiles.csv	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/protected_files_sha256.json	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/reference_reproduction.json	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/summary.json	已保存的计算场、汇总结果或验证记录
results/q4/validation_moving_mesh/unit_checks.json	已保存的计算场、汇总结果或验证记录
results/result1.xlsx	题目要求的正式结果工作簿
results/result2.xlsx	题目要求的正式结果工作簿
results/result3.xlsx	题目要求的正式结果工作簿
results/result4.xlsx	题目要求的正式结果工作簿
results/revision_checks.json	已保存的计算场、汇总结果或验证记录
results/revision_scope_verification.json	已保存的计算场、汇总结果或验证记录
results/sensitivity.json	已保存的计算场、汇总结果或验证记录
results/workbook_verification.json	已保存的计算场、汇总结果或验证记录
附件/ 附件1.xlsx	题目原始输入数据或结果工作簿模板
附件/ 附件2.xlsx	题目原始输入数据或结果工作簿模板
附件/ 附件3/result1.xlsx	题目原始输入数据或结果工作簿模板
附件/ 附件3/result2.xlsx	题目原始输入数据或结果工作簿模板
附件/ 附件3/result3.xlsx	题目原始输入数据或结果工作簿模板
附件/ 附件3/result4.xlsx	题目原始输入数据或结果工作簿模板

附录 B 完整源程序

以下收录求解、结果导出、数值验证与绘图所用源程序。除首行文件路径注释外，内容与项目对应源码一致；可执行文件及运行说明见支撑材料。

2.1 solve_drying.py

```
# File: code/solve_drying.py
"""Numerical solution for CUMCM 2026 Problem A.

The model uses a node-centred finite-volume discretisation in the radial
coordinate, fully implicit diffusion, Picard iteration for nonlinear
properties, and an ALE-style fixed-domain transform for shrinkage.
"""

from __future__ import annotations

import argparse
import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator
from scipy.linalg import solve_banded

ROOT = Path(__file__).resolve().parents[1]
INPUT_DIR = ROOT / "附件"
RESULTS_DIR = ROOT / "results"
H_T = 25.0
H_M = 8.0e-7
R_FIXED = 0.02
NODES = 161
OUTPUT_NODES = 21
GRID_POWER = 2.0
INITIAL_T = 28.0
INITIAL_C = 2.55
THRESHOLD_C = 0.15
TERMINAL_TEMPERATURE_C = 50.0
TERMINAL_MOISTURE = 0.05
```

```

@dataclass
class Environment:
    time_s: np.ndarray
    temperature_c: np.ndarray
    moisture: np.ndarray
    temp_interp: PchipInterpolator
    moisture_interp: PchipInterpolator

    def values(
        self,
        time_s: float,
        terminal_temp_scale: float = 1.0,
        terminal_moisture_scale: float = 1.0,
    ) -> tuple[float, float]:
        cutoff = float(self.time_s[-1])
        if time_s <= cutoff:
            return float(self.temp_interp(time_s)), float(self.moisture_interp(time_s))
        return (
            float(TERMINAL_TEMPERATURE_C * terminal_temp_scale),
            float(TERMINAL_MOISTURE * terminal_moisture_scale),
        )

```

```

@dataclass
class RadiusHistory:
    time_s: np.ndarray
    radius_m: np.ndarray
    interp: PchipInterpolator
    derivative: PchipInterpolator
    monotone_adjustments: int

    def values(self, time_s: float) -> tuple[float, float]:
        bounded = min(max(time_s, float(self.time_s[0])), float(self.time_s[-1]))
        return float(self.interp(bounded)), float(self.derivative(bounded))

```

```

def _read_numeric_sheet(path: Path) -> np.ndarray:
    book = load_workbook(path, read_only=True, data_only=True)
    sheet = book.active
    rows: list[list[float]] = []
    for row in sheet.iter_rows(min_row=2, values_only=True):
        if row[0] is None:
            continue
        rows.append([float(value) for value in row])
    return np.asarray(rows, dtype=float)

```

```

def load_environment() -> Environment:
    data = _read_numeric_sheet(INPUT_DIR / "附件1.xlsx")
    time_s, temperature_c, moisture = data.T
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件1时间列不是严格递增。")
    if not np.all(np.isfinite(data)):
        raise ValueError("附件1含非有限数。")
    return Environment(
        time_s=time_s,
        temperature_c=temperature_c,
        moisture=moisture,
        temp_interp=PchipInterpolator(time_s, temperature_c),
        moisture_interp=PchipInterpolator(time_s, moisture),
    )

def load_radius_history() -> RadiusHistory:
    data = _read_numeric_sheet(INPUT_DIR / "附件2.xlsx")
    time_s = data[:, 0]
    raw_radius_m = data[:, 1] / 100.0
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件2时间列不是严格递增。")
    monotone_radius_m = np.minimum.accumulate(raw_radius_m)
    adjustments = int(np.count_nonzero(np.abs(monotone_radius_m - raw_radius_m) > 1e-12))
    interp = PchipInterpolator(time_s, monotone_radius_m, extrapolate=False)
    return RadiusHistory(
        time_s=time_s,
        radius_m=monotone_radius_m,
        interp=interp,
        derivative=interp.derivative(),
        monotone_adjustments=adjustments,
    )

def property_q1(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    del temperature_c
    rho = np.full_like(moisture, 820.0)
    cp = np.full_like(moisture, 2600.0)
    k = np.full_like(moisture, 0.36)
    safe_c = np.maximum(moisture, 1e-8)
    diffusivity = 7.0e-9 * np.exp(-0.89 / safe_c)
    return rho, cp, k, diffusivity

def property_q23(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:

```

```

    safe_c = np.maximum(moisture, 1e-8)
    rho = 650.0 + 128.0 * safe_c
    cp = 1450.0 + 2736.0 * safe_c / (safe_c + 1.0)
    k = 0.21 + 0.38 * safe_c / (safe_c + 1.0)
    diffusivity = (
        2.4e-3
        * np.exp(-0.45 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def property_q4(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 760.0 + 90.0 * safe_c
    cp = 1850.0 + 2150.0 * safe_c / (safe_c + 1.0)
    k = 0.12 + 0.20 * safe_c / (safe_c + 1.0)
    diffusivity = (
        4.2e-4
        * np.exp(-0.30 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def face_interpolate(left: np.ndarray, right: np.ndarray) -> np.ndarray:
    """Linear face interpolation for a smooth state-dependent coefficient."""
    return 0.5 * (left + right)

def radial_grid(count: int, radius: float = 1.0) -> np.ndarray:
    """Surface-refined computational mesh; output positions are independent."""
    s = np.linspace(0.0, 1.0, count)
    return radius * (1.0 - (1.0 - s) ** GRID_POWER)

def node_control_volumes(coordinate: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return dimensionless/physical radial CV volumes and internal face radii."""
    faces = 0.5 * (coordinate[:-1] + coordinate[1:])
    lower = np.concatenate(([0.0], faces))
    upper = np.concatenate((faces, [coordinate[-1]]))
    volumes = math.pi * (upper**2 - lower**2)
    return volumes, faces

def implicit_radial_step(
    old: np.ndarray,

```

```

storage: np.ndarray,
conductivity: np.ndarray,
dt: float,
coordinate: np.ndarray,
diffusion_scale: float,
boundary_conductance: float,
boundary_value: float,
advection_velocity: np.ndarray | None = None,
) -> np.ndarray:
    """One implicit node-centred radial FVM step on coordinate [0, 1] or [0, R]."""
    count = len(old)
    volumes, faces = node_control_volumes(coordinate)
    delta = np.diff(coordinate)
    face_property = face_interpolate(conductivity[:-1], conductivity[1:])
    conductance = 2.0 * math.pi * faces * face_property * diffusion_scale / delta

    capacity = storage * volumes / dt
    diagonal = capacity.copy()
    lower = np.zeros(count - 1)
    upper = np.zeros(count - 1)
    rhs = capacity * old

    diagonal[:-1] += conductance
    diagonal[1:] += conductance
    upper -= conductance
    lower -= conductance

    diagonal[-1] += boundary_conductance
    rhs[-1] += boundary_conductance * boundary_value

    if advection_velocity is not None:
        # PDE form:  $\phi_t + u(x_i) \phi_{xi} = \text{diffusion}$ ;  $u \geq 0$  under shrinkage.
        advective = storage[1:] * volumes[1:] * np.maximum(advection_velocity[1:], 0.0) /
        delta
        diagonal[1:] += advective
        lower -= advective

    matrix = np.zeros((3, count))
    matrix[0, 1:] = upper
    matrix[1, :] = diagonal
    matrix[2, :-1] = lower
    return solve_banded((1, 1), matrix, rhs, check_finite=False)

def advance_coupled(
    temperature_old: np.ndarray,
    moisture_old: np.ndarray,

```

```

dt: float,
radius_m: float,
radius_rate_m_s: float,
ambient_temperature_c: float,
ambient_moisture: float,
property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
moving_domain: bool,
tolerance_temperature: float = 1e-8,
tolerance_moisture: float = 1e-10,
max_iterations: int = 30,
balance: dict | None = None,
) -> tuple[np.ndarray, np.ndarray, int]:
    temperature = temperature_old.copy()
    moisture = moisture_old.copy()
    if moving_domain:
        coordinate = radial_grid(len(temperature_old))
        diffusion_scale = 1.0 / radius_m**2
        boundary_scale = 1.0 / radius_m
        shrink_velocity = -(radius_rate_m_s / radius_m) * coordinate
    else:
        coordinate = radial_grid(len(temperature_old), radius_m)
        diffusion_scale = 1.0
        boundary_scale = 1.0
        shrink_velocity = None

    for iteration in range(1, max_iterations + 1):
        rho, cp, k, _ = property_model(temperature, moisture)
        heat_storage = rho * cp
        heat_boundary = 2.0 * math.pi * coordinate[-1] * H_T * boundary_scale
        next_temperature = implicit_radial_step(
            old=temperature_old,
            storage=heat_storage,
            conductivity=k,
            dt=dt,
            coordinate=coordinate,
            diffusion_scale=diffusion_scale,
            boundary_conductance=heat_boundary,
            boundary_value=ambient_temperature_c,
            advection_velocity=shrink_velocity,
        )

    _, _, _ , diffusivity = property_model(next_temperature, moisture)
    mass_boundary = 2.0 * math.pi * coordinate[-1] * H_M * boundary_scale
    next_moisture = implicit_radial_step(
        old=moisture_old,
        storage=np.ones_like(moisture),
        conductivity=diffusivity,

```

```

        dt=dt,
        coordinate=coordinate,
        diffusion_scale=diffusion_scale,
        boundary_conductance=mass_boundary,
        boundary_value=ambient_moisture,
        advection_velocity=shrink_velocity,
    )

    error_t = float(np.max(np.abs(next_temperature - temperature)))
    error_c = float(np.max(np.abs(next_moisture - moisture)))
    temperature = next_temperature
    moisture = next_moisture
    if error_t < tolerance_temperature and error_c < tolerance_moisture:
        break
    else:
        raise RuntimeError(
            f"Picard iteration failed: dT={error_t:.3e}, dC={error_c:.3e}, dt={dt}"
        )

if not np.all(np.isfinite(temperature)) or not np.all(np.isfinite(moisture)):
    raise FloatingPointError("Non-finite state generated.")
if np.min(moisture) < -1e-10 or np.max(moisture) > INITIAL_C + 1e-8:
    raise FloatingPointError("Moisture left physical bounds.")
if balance is not None:
    volumes, _ = node_control_volumes(coordinate)
    adv_c = adv_t = 0.0
    if shrink_velocity is not None:
        weight = volumes[1:] * np.maximum(shrink_velocity[1:], 0.0) / np.diff(
coordinate)
        adv_c = dt * float(np.dot(weight, np.diff(moisture)))
        adv_t = dt * float(np.dot(weight * heat_storage[1:], np.diff(temperature)))
    change = float(np.dot(volumes, moisture - moisture_old))
    flux = dt * mass_boundary * (float(moisture[-1]) - ambient_moisture)
    reference = INITIAL_C * float(volumes.sum())
    balance['moisture_change'] += change / reference
    balance['boundary_loss'] += flux / reference
    balance['relative_transport'] += adv_c / reference
    residual = change + flux + adv_c
    balance['maximum_step_mass_residual'] = max(balance['maximum_step_mass_residual'],
abs(residual) / reference)
    heat_change = float(np.dot(heat_storage * volumes, temperature - temperature_old))
    heat_flux = dt * heat_boundary * (float(temperature[-1]) - ambient_temperature_c)
    heat_reference = INITIAL_T * float(np.dot(heat_storage, volumes))
    heat_residual = (heat_change + heat_flux + adv_t) / heat_reference
    balance['maximum_step_heat_equation_residual'] = max(balance['
maximum_step_heat_equation_residual'], abs(heat_residual))
return temperature, moisture, iteration

```

```

def new_balance() -> dict:
    return dict(moisture_change=0.0, boundary_loss=0.0, relative_transport=0.0,
                maximum_step_mass_residual=0.0, maximum_step_heat_equation_residual=0.0)

def simulate_fixed(
    environment: Environment,
    property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
    end_time_s: float | None,
    dt_s: float,
    output_interval_s: float,
    stop_at_threshold: bool,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
    stop_interval_s: float | None = None,
) -> dict[str, np.ndarray | float | int | bool]:
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    radial_m = radial_grid(NODES, R_FIXED)
    output_r = np.linspace(0.0, R_FIXED, OUTPUT_NODES)
    times = [0.0]
    temperatures = [np.interp(output_r, radial_m, temperature)]
    moistures = [np.interp(output_r, radial_m, moisture)]
    iterations: list[int] = []
    monotone_profiles = True
    center_is_max = True
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    stop_output_time = math.inf
    balance = new_balance()
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(end_time_s if end_time_s is not None else 10.0 * 24.0 * 3600.0)

    while time_s < maximum_time - 1e-9:
        step = min(dt_s, maximum_time - time_s, next_output - time_s)
        new_time = time_s + step
        temperature_before = temperature.copy()
        moisture_before = moisture.copy()
        ambient_t, ambient_c = environment.values(
            new_time, terminal_temp_scale, terminal_moisture_scale
        )

```

```

temperature, moisture, count = advance_coupled(
    temperature,
    moisture,
    step,
    R_FIXED,
    0.0,
    ambient_t,
    ambient_c,
    property_model,
    moving_domain=False,
    balance=balance,
)
iterations.append(count)
time_s = new_time
current_max = float(np.max(moisture))
monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
center_is_max = center_is_max and bool(
    moisture[0] >= float(np.max(moisture)) - 1e-9
)

if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
    fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e
-15)
    threshold_time = previous_time + fraction * step
    threshold_temperature = (
        temperature_before + fraction * (temperature - temperature_before)
    )
    threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
    stop_spacing = output_interval_s if stop_interval_s is None else
stop_interval_s
    stop_output_time = math.ceil(threshold_time / stop_spacing) * stop_spacing

if time_s + 1e-8 >= next_output:
    times.append(time_s)
    temperatures.append(np.interp(output_r, radial_m, temperature))
    moistures.append(np.interp(output_r, radial_m, moisture))
    next_output += output_interval_s

if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
    break
previous_max = current_max
previous_time = time_s

return {
    "times_s": np.asarray(times),
    "radius_m": output_r,
    "temperature_c": np.asarray(temperatures),

```

```

        "moisture": np.asarray(moistures),
        "threshold_time_s": threshold_time,
        "threshold_temperature_c": None if threshold_temperature is None else np.interp(
output_r, radial_m, threshold_temperature),
        "threshold_moisture": None if threshold_moisture is None else np.interp(output_r,
radial_m, threshold_moisture),
        "last_time_s": time_s,
        "picard_max": int(max(iterations, default=0)),
        "picard_mean": float(np.mean(iterations) if iterations else 0.0),
        "monotone_profiles": monotone_profiles,
        "center_is_max": center_is_max,
        "balance": balance,
    }

def simulate_moving(
    environment: Environment,
    radius_history: RadiusHistory,
    dt_s: float,
    output_interval_s: float,
    stop_at_threshold: bool = True,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
    include_advection: bool = False,
) -> dict[str, np.ndarray | float | int | bool]:
    # Default: homogeneous solid shrinkage, material velocity equals mesh velocity.
    # True retains the former stationary-spatial scalar model for structural comparison.
    xi = radial_grid(NODES)
    # Preserve native nodes; interpolate to physical positions only at export.
    output_xi = xi.copy()
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    times = [0.0]
    radii = [R_FIXED]
    temperatures = [np.interp(output_xi, xi, temperature)]
    moistures = [np.interp(output_xi, xi, moisture)]
    iterations: list[int] = []
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    threshold_radius_m = math.nan
    stop_output_time = math.inf
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(radius_history.time_s[-1])

```

```

monotone_profiles = True
center_is_max = True
maximum_cfl = 0.0
balance = new_balance()

while time_s < maximum_time - 1e-9:
    step = min(dt_s, maximum_time - time_s, next_output - time_s)
    new_time = time_s + step
    temperature_before = temperature.copy()
    moisture_before = moisture.copy()
    radius_m, radius_rate_m_s = radius_history.values(new_time)
    if not include_advection:
        radius_rate_m_s = 0.0
    ambient_t, ambient_c = environment.values(
        new_time, terminal_temp_scale, terminal_moisture_scale
    )
    temperature, moisture, count = advance_coupled(
        temperature,
        moisture,
        step,
        radius_m,
        radius_rate_m_s,
        ambient_t,
        ambient_c,
        property_q4,
        moving_domain=True,
        balance=balance,
    )
    iterations.append(count)
    time_s = new_time
    current_max = float(np.max(moisture))
    maximum_cfl = max(
        maximum_cfl,
        float(np.max(abs(radius_rate_m_s / radius_m) * xi[1:] * step / np.diff(xi))),
    )
    monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
    center_is_max = center_is_max and bool(
        moisture[0] >= float(np.max(moisture)) - 1e-9
    )

    if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
        fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e
-15)
        threshold_time = previous_time + fraction * step
        threshold_temperature = (
            temperature_before + fraction * (temperature - temperature_before)
        )

```

```

        threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
        threshold_radius_m = radius_history.values(threshold_time)[0]
        stop_output_time = (
            math.ceil(threshold_time / output_interval_s) * output_interval_s
        )

    if time_s + 1e-8 >= next_output:
        times.append(time_s)
        radii.append(radius_m)
        temperatures.append(np.interp(output_xi, xi, temperature))
        moistures.append(np.interp(output_xi, xi, moisture))
        next_output += output_interval_s

    if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
        break

    previous_max = current_max
    previous_time = time_s

    return {
        "times_s": np.asarray(times),
        "xi": output_xi,
        "radii_m": np.asarray(radii),
        "temperature_c": np.asarray(temperatures),
        "moisture": np.asarray(moistures),
        "threshold_time_s": threshold_time,
        "threshold_temperature_c": None if threshold_temperature is None else np.interp(
            output_xi, xi, threshold_temperature),
        "threshold_moisture": None if threshold_moisture is None else np.interp(output_xi,
            xi, threshold_moisture),
        "threshold_radius_m": threshold_radius_m,
        "last_time_s": time_s,
        "picard_max": int(max(iterations, default=0)),
        "picard_mean": float(np.mean(iterations) if iterations else 0.0),
        "monotone_profiles": monotone_profiles,
        "center_is_max": center_is_max,
        "maximum_advection_cfl": maximum_cfl,
        "balance": balance,
        "velocity_model": "stationary_spatial_comparison" if include_advection else "
        uniform_material_shrinkage",
    }

def extract_fixed_table(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_times_s: list[float],
    requested_radii_cm: list[float],
    variable: str,

```

```

) -> list[list[float]]:
    times = np.asarray(simulation["times_s"])
    radii_m = np.asarray(simulation["radius_m"])
    values = np.asarray(simulation[variable])
    table: list[list[float]] = []
    for requested_time in requested_times_s:
        threshold_time = float(simulation.get("threshold_time_s", math.nan))
        threshold_key = (
            "threshold_temperature_c" if variable == "temperature_c" else "
threshold_moisture"
        )
        threshold_profile = simulation.get(threshold_key)
        if (
            threshold_profile is not None
            and math.isfinite(threshold_time)
            and abs(requested_time - threshold_time) < 1e-5
        ):
            profile = np.asarray(threshold_profile)
        else:
            time_index = int(np.argmin(np.abs(times - requested_time)))
            profile = values[time_index]
        row = [
            float(np.interp(radius_cm / 100.0, radii_m, profile))
            for radius_cm in requested_radii_cm
        ]
        table.append(row)
    return table

def extract_moving_profile(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_time_s: float,
    requested_radii_cm: list[float],
) -> list[float | None]:
    times = np.asarray(simulation["times_s"])
    xi = np.asarray(simulation["xi"])
    radii_m = np.asarray(simulation["radii_m"])
    moisture = np.asarray(simulation["moisture"])
    threshold_time = float(simulation.get("threshold_time_s", math.nan))
    threshold_profile = simulation.get("threshold_moisture")
    if (
        threshold_profile is not None
        and math.isfinite(threshold_time)
        and abs(requested_time_s - threshold_time) < 1e-5
    ):
        profile = np.asarray(threshold_profile)
        radius = float(simulation["threshold_radius_m"])

```

```

else:
    index = int(np.argmin(np.abs(times - requested_time_s)))
    profile = moisture[index]
    radius = float(radii_m[index])
values: list[float | None] = []
for radius_cm in requested_radii_cm:
    position_m = radius_cm / 100.0
    if position_m > radius + 1e-12:
        values.append(None)
    else:
        values.append(float(np.interp(position_m / radius, xi, profile)))
values.append(float(profile[-1]))
return values

def save_npz(path: Path, simulation: dict[str, np.ndarray | float | int | bool]) -> None:
    arrays = {key: value for key, value in simulation.items() if isinstance(value, np.
ndarray)}
    np.savez_compressed(path, **arrays)

def json_ready(value):
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, float) and math.isnan(value):
        return None
    if isinstance(value, dict):
        return {key: json_ready(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [json_ready(item) for item in value]
    return value

def write_summary(path: Path, payload: dict) -> None:
    path.write_text(
        json.dumps(json_ready(payload), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def run_all(run_convergence: bool = True) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    for name in ("q1", "q2", "q3", "q4"):
        (RESULTS_DIR / name).mkdir(parents=True, exist_ok=True)

```

```

environment = load_environment()
radius_history = load_radius_history()
data_audit = {
    "environment_rows": len(environment.time_s),
    "environment_time_range_s": [environment.time_s[0], environment.time_s[-1]],
    "environment_sampling_s": np.unique(np.diff(environment.time_s)),
    "temperature_range_c": [
        float(np.min(environment.temperature_c)),
        float(np.max(environment.temperature_c)),
    ],
    "ambient_moisture_range": [
        float(np.min(environment.moisture)),
        float(np.max(environment.moisture)),
    ],
    "terminal_boundary_after_14400_s": {
        "temperature_c": TERMINAL_TEMPERATURE_C,
        "moisture": TERMINAL_MOISTURE,
        "basis": "constant-stage setpoints; final-hour attachment means are 49.9989 C
and 0.049988",
    },
    "radius_rows": len(radius_history.time_s),
    "radius_time_range_s": [radius_history.time_s[0], radius_history.time_s[-1]],
    "radius_range_cm": [
        float(np.min(radius_history.radius_m) * 100.0),
        float(np.max(radius_history.radius_m) * 100.0),
    ],
    "radius_monotone_adjustments": radius_history.monotone_adjustments,
}
write_summary(RESULTS_DIR / "data_audit.json", data_audit)

q1 = simulate_fixed(
    environment,
    property_q1,
    end_time_s=1800.0,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=False,
)
save_npz(RESULTS_DIR / "q1" / "fields.npz", q1)
q1_times = [100, 300, 600, 900, 1200, 1500, 1800]
table_radai = [0.0, 0.5, 1.0, 1.5, 2.0]
q1_summary = {
    "table_times_s": q1_times,
    "table_radai_cm": table_radai,
    "temperature_table_c": extract_fixed_table(q1, q1_times, table_radai, "
temperature_c"),
    "moisture_table": extract_fixed_table(q1, q1_times, table_radai, "moisture"),
}

```

```

    "biot_heat": H_T * R_FIXED / 0.36,
    "initial_diffusivity_m2_s": float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "biot_mass_initial": H_M * R_FIXED / float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "picard_max": q1["picard_max"],
    "picard_mean": q1["picard_mean"],
    "temperature_bounds_c": [
        float(np.min(q1["temperature_c"])),
        float(np.max(q1["temperature_c"])),
    ],
    "moisture_bounds": [
        float(np.min(q1["moisture"])),
        float(np.max(q1["moisture"])),
    ],
}
write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)

q2 = simulate_fixed(
    environment,
    property_q23,
    end_time_s=None,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=True,
    stop_interval_s=60.0,
)
save_npz(RESULTS_DIR / "q2" / "fields.npz", q2)
q2_times = [1800, 3600, 5400, 7200, 9000, 10800]
q2_summary = {
    "table_times_h": [time / 3600.0 for time in q2_times],
    "table_radai_cm": table_radai,
    "temperature_table_c": extract_fixed_table(q2, q2_times, table_radai, "
temperature_c"),
    "moisture_table": extract_fixed_table(q2, q2_times, table_radai, "moisture"),
    "picard_max": q2["picard_max"],
    "picard_mean": q2["picard_mean"],
    "monotone_profiles": q2["monotone_profiles"],
    "center_is_max": q2["center_is_max"],
}
write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)

q3 = dict(q2)
for key in ("times_s", "temperature_c", "moisture"):
    q3[key] = q2[key][:60].copy()

```

```

save_npz(RESULTS_DIR / "q3" / "fields.npz", q3)
q3_end = float(q3["threshold_time_s"])
q3_report_times = list(np.arange(6 * 3600, math.floor(q3_end / (6 * 3600)) * 6 * 3600
+ 1, 6 * 3600))
if not q3_report_times or abs(q3_report_times[-1] - q3_end) > 1e-6:
    q3_report_times.append(q3_end)
q3_summary = {
    "drying_time_s": q3_end,
    "drying_time_h": q3_end / 3600.0,
    "output_end_s": q3["last_time_s"],
    "table_times_h": [time / 3600.0 for time in q3_report_times],
    "table_radai_cm": table_radai,
    "moisture_table": extract_fixed_table(q3, q3_report_times, table_radai, "moisture")
,
    "center_is_max": q3["center_is_max"],
    "monotone_profiles": q3["monotone_profiles"],
    "picard_max": q3["picard_max"],
    "picard_mean": q3["picard_mean"],
}

q4 = simulate_moving(
    environment,
    radius_history,
    dt_s=1.0,
    output_interval_s=60.0,
    stop_at_threshold=True,
)
save_npz(RESULTS_DIR / "q4" / "fields.npz", q4)
q4_end = float(q4["threshold_time_s"])
q4_report_times = list(np.arange(6 * 3600, math.floor(q4_end / (6 * 3600)) * 6 * 3600
+ 1, 6 * 3600))
if not q4_report_times or abs(q4_report_times[-1] - q4_end) > 1e-6:
    q4_report_times.append(q4_end)
q4_table_radai = [0.0, 0.5, 1.0]
q4_summary = {
    "drying_time_s": q4_end,
    "drying_time_h": q4_end / 3600.0,
    "output_end_s": q4["last_time_s"],
    "table_times_h": [time / 3600.0 for time in q4_report_times],
    "table_radai_cm": q4_table_radai,
    "columns": q4_table_radai + ["药材表面"],
    "moisture_table": [
        extract_moving_profile(q4, time, q4_table_radai) for time in q4_report_times
    ],
    "radius_at_end_cm": float(radius_history.values(q4_end)[0] * 100.0),
    "center_is_max": q4["center_is_max"],
    "monotone_profiles": q4["monotone_profiles"],
}

```

```

    "maximum_advection_cfl": q4["maximum_advection_cfl"],
    "picard_max": q4["picard_max"],
    "picard_mean": q4["picard_mean"],
}

if run_convergence:
    q3_dt2 = simulate_fixed(
        environment,
        property_q23,
        end_time_s=None,
        dt_s=2.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    q4_dt2 = simulate_moving(
        environment,
        radius_history,
        dt_s=2.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    q4_stationary = simulate_moving(
        environment,
        radius_history,
        dt_s=1.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
        include_advection=True,
    )
    q3_summary["time_step_check"] = {
        "dt1_s_time_h": q3_end / 3600.0,
        "dt2_s_time_h": float(q3_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q3_dt2["threshold_time_s"]) - q3_end
        ) / float(q3_dt2["threshold_time_s"]),
    }
    q4_summary["time_step_check"] = {
        "dt1_s_time_h": q4_end / 3600.0,
        "dt2_s_time_h": float(q4_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q4_dt2["threshold_time_s"]) - q4_end
        ) / float(q4_dt2["threshold_time_s"]),
    }
    q4_summary["moving_term_comparison"] = {
        "material_model_h": q4_end / 3600.0,
        "stationary_spatial_model_h": float(q4_stationary["threshold_time_s"]) /
3600.0,

```

```

        "difference_h": (
            float(q4_stationary["threshold_time_s"]) - q4_end
        ) / 3600.0,
    }

for summary, simulation in [(q1_summary, q1), (q2_summary, q2), (q3_summary, q3), (
q4_summary, q4)]:
    summary['numerics'] = {'nodes': NODES, 'grid_power': GRID_POWER, 'dt_s': 1.0, '
output_nodes': simulation['moisture'].shape[1]}
    summary['balance'] = simulation['balance']
    summary['balance']['cumulative_mass_equation_residual'] = abs(sum(simulation['
balance'][key] for key in ['moisture_change', 'boundary_loss', 'relative_transport']))
    q2_summary['output_end_s'] = q2['last_time_s']
    q2_summary['drying_time_h'] = q2['threshold_time_s'] / 3600.0
    q3_summary['shared_trajectory_with_q2'] = bool(np.array_equal(q3['moisture'], q2['
moisture'][:60]))
    q4_summary['velocity_model'] = q4['velocity_model']
    write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)
    write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)
    write_summary(RESULTS_DIR / "q3" / "summary.json", q3_summary)
    write_summary(RESULTS_DIR / "q4" / "summary.json", q4_summary)

print(json.dumps(
    {
        "q1_surface_temperature_1800_c": q1_summary["temperature_table_c"][-1][-1],
        "q1_center_moisture_1800": q1_summary["moisture_table"][-1][0],
        "q2_center_temperature_3h_c": q2_summary["temperature_table_c"][-1][0],
        "q2_center_moisture_3h": q2_summary["moisture_table"][-1][0],
        "q3_drying_time_h": q3_summary["drying_time_h"],
        "q4_drying_time_h": q4_summary["drying_time_h"],
        "q4_radius_at_end_cm": q4_summary["radius_at_end_cm"],
    },
    ensure_ascii=False,
    indent=2,
))

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-convergence", action="store_true")
    args = parser.parse_args()
    run_all(run_convergence=not args.skip_convergence)

if __name__ == "__main__":
    main()

```

2.2 write_results.py

```
# File: code/write_results.py
"""Write the four official-format Excel result workbooks."""

from __future__ import annotations

import json
from copy import copy
import hashlib
import shutil
import sys
from pathlib import Path

import numpy as np
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parents[1]
TEMPLATE_DIR = ROOT / "附件" / "附件3"
RESULTS_DIR = ROOT / "results"
def reset_sheet(sheet) -> dict:
    """Clear template placeholders while retaining the official sheet settings."""
    last_column = sheet.max_column
    template = {
        "corner": sheet.cell(1, 1).value,
        "styles": {(row, column): copy(sheet.cell(row, column)._style)
                    for row in (1, 2) for column in range(1, last_column + 1)},
        "last_column": last_column,
        "column_a": copy(sheet.column_dimensions["A"]),
        "column_data": copy(sheet.column_dimensions["B"]),
        "row_header": copy(sheet.row_dimensions[1]),
        "row_data": copy(sheet.row_dimensions[2]),
    }
    for row in sheet.iter_rows():
        for cell in row:
            cell.value = None
    return template

def style_sheet(sheet, row_count: int, column_count: int, template: dict) -> None:
    """Extend template styles; only result cells use the required four decimals."""
    last_template_column = template["last_column"]
```

```

for row in sheet.iter_rows(min_row=1, max_row=row_count,
                           min_col=1, max_col=column_count):
    for cell in row:
        prototype_column = (last_template_column if cell.column == column_count
                             else min(cell.column, last_template_column - 2))
        cell._style = copy(template["styles"][(1 if cell.row == 1 else 2, prototype_column)])
        if cell.row > 1 and cell.column > 1:
            cell.number_format = "0.0000"
sheet.column_dimensions.clear()
for column in range(1, column_count + 1):
    letter = get_column_letter(column)
    dim = copy(template["column_a" if column == 1 else "column_data"])
    dim.index = letter
    dim.min = dim.max = column
    sheet.column_dimensions[letter] = dim
sheet.row_dimensions.clear()
for row in range(1, row_count + 1):
    dim = copy(template["row_header" if row == 1 else "row_data"])
    dim.index = row
    sheet.row_dimensions[row] = dim

def anonymize_workbook(book) -> None:
    """Remove personal document properties from the submitted result copy."""
    book.properties.creator = ""
    book.properties.lastModifiedBy = ""

def write_fixed_workbook(
    template_name: str,
    output_name: str,
    fields_path: Path,
    include_temperature: bool,
    end_time_s: float | None = None,
) -> dict:
    output_path = RESULTS_DIR / output_name
    shutil.copy2(TEMPLATE_DIR / template_name, output_path)
    book = load_workbook(output_path)
    fields = np.load(fields_path)
    times = fields["times_s"]
    radii_cm = fields["radius_m"] * 100.0
    start = 1 if abs(times[0]) < 1e-12 else 0
    stop = len(times) if end_time_s is None else int(np.searchsorted(times, end_time_s,
                              side="right"))
    written_rows = stop - start + 1
    sheet_variables = (

```

```

[("温度", "temperature_c"), ("水分浓度", "moisture")]
if include_temperature
else [(book.sheetnames[0], "moisture")]
)
for sheet_name, key in sheet_variables:
    sheet = book[sheet_name]
    template = reset_sheet(sheet)
    sheet.cell(1, 1, template["corner"])
    for column, radius in enumerate(radii_cm, 2):
        sheet.cell(1, column, round(float(radius), 1))
    values = fields[key]
    for index in range(start, stop):
        row_number = index - start + 2
        sheet.cell(row_number, 1, int(round(float(times[index])), 1))
        for column, value in enumerate(values[index], 2):
            sheet.cell(row_number, column, round(float(value), 4))
    style_sheet(sheet, written_rows, len(radii_cm) + 1, template)
anonymize_workbook(book)
book.save(output_path)
book.close()
fields.close()
return {
    "path": str(output_path),
    "rows_including_header": written_rows,
    "columns": len(radii_cm) + 1,
    "sheets": [name for name, _ in sheet_variables],
}

def write_moving_workbook() -> dict:
    fields = np.load(RESULTS_DIR / "q4" / "fields.npz")
    numerics = json.loads((RESULTS_DIR / "q4" / "summary.json").read_text(encoding="utf-8"))
    ["numerics"]
    native_xi = 1.0 - (1.0 - np.linspace(0.0, 1.0, numerics["nodes"])) ** numerics["
    grid_power"]
    if fields["xi"].shape != native_xi.shape or not np.allclose(fields["xi"], native_xi,
    rtol=0, atol=1e-14):
        fields.close()
        raise ValueError("Q4 output requires native calculation nodes; regenerate q4/fields
        .npz before exporting.")
    output_path = RESULTS_DIR / "result4.xlsx"
    shutil.copy2(TEMPLATE_DIR / "result4.xlsx", output_path)
    book = load_workbook(output_path)
    sheet = book[book.sheetnames[0]]
    template = reset_sheet(sheet)
    times = fields["times_s"]
    xi = fields["xi"]

```

```

radii = fields["radii_m"]
moisture = fields["moisture"]
fixed_radii_cm = np.arange(0.0, 2.0, 0.1)
headers = ([template["corner"]]
            + [round(float(radius), 1) for radius in fixed_radii_cm]
            + ["药材表面"])
for column, value in enumerate(headers, 1):
    sheet.cell(1, column, value)
start = 1 if abs(times[0]) < 1e-12 else 0
for index in range(start, len(times)):
    current_radius_m = float(radii[index])
    profile = moisture[index]
    row: list[float | int | None] = [int(round(float(times[index])))]
    for radius_cm in fixed_radii_cm:
        position_m = radius_cm / 100.0
        if position_m > current_radius_m + 1e-12:
            row.append(None)
        else:
            value = np.interp(position_m / current_radius_m, xi, profile)
            row.append(round(float(value), 4))
    row.append(round(float(profile[-1]), 4))
    for column, value in enumerate(row, 1):
        sheet.cell(index - start + 2, column, value)
rows = len(times) - start + 1
columns = len(fixed_radii_cm) + 2
style_sheet(sheet, rows, columns, template)
anonymize_workbook(book)
book.save(output_path)
book.close()
fields.close()
return {
    "path": str(output_path),
    "rows_including_header": rows,
    "columns": columns,
    "sheets": book.sheetnames,
    "blank_rule": "固定物理位置超过当时药材半径时留空；末列始终为实时表面。",
}

```

```

def verify_workbook(item: dict) -> dict:
    path = Path(item["path"])
    book = load_workbook(path, read_only=True, data_only=False)
    verification = {
        "path": str(path),
        "file_size_bytes": path.stat().st_size,
        "sheets": {},
        "formula_errors": [],
    }

```

```

}
error_tokens = {"#REF!", "#DIV/0!", "#VALUE!", "#NAME?", "#N/A", "#NUM!", "#NULL!"}
for sheet in book.worksheets:
    first_row = [cell.value for cell in next(sheet.iter_rows(min_row=1, max_row=1))]
    last_row = [
        cell.value
        for cell in next(
            sheet.iter_rows(
                min_row=sheet.max_row,
                max_row=sheet.max_row,
                min_col=1,
                max_col=sheet.max_column,
            )
        )
    ]
    for row in sheet.iter_rows(values_only=True):
        for value in row:
            if isinstance(value, str) and value in error_tokens:
                verification["formula_errors"].append(
                    {"sheet": sheet.title, "value": value}
                )
    verification["sheets"][sheet.title] = {
        "rows": sheet.max_row,
        "columns": sheet.max_column,
        "first_row": first_row,
        "last_row": last_row,
    }
verification["pass"] = (
    not verification["formula_errors"]
    and all(
        details["rows"] == item["rows_including_header"]
        and details["columns"] == item["columns"]
        for details in verification["sheets"].values()
    )
)
question = int(path.stem[-1])
fields_path = RESULTS_DIR / f"q{question}" / "fields.npz"
with np.load(fields_path) as archive:
    # Decode each compressed array once, rather than once per Excel row.
    fields = {key: archive[key] for key in archive.files}
times = fields["times_s"]
end_time = {1: 1800.0, 2: 10800.0}.get(question, float(times[-1]))
indexes = np.flatnonzero((times > 0) & (times <= end_time))
interval = 1 if question <= 2 else 60
expected_times = np.arange(interval, end_time + 0.5, interval)
schedule_valid = np.array_equal(times[indexes], expected_times)
mismatches = 0

```

```

for sheet in book.worksheets:
    key = "temperature_c" if sheet.title == "温度" else "moisture"
    if sheet.max_row != len(indexes) + 1:
        mismatches += 1
    for index, row in zip(indexes, sheet.iter_rows(min_row=2, values_only=True)):
        if row[0] != int(times[index]):
            mismatches += 1
        if question != 4:
            values = np.interp(np.linspace(0, 0.02, 21), fields["radius_m"], fields[key][index])
            expected = [round(float(v), 4) for v in values]
        else:
            radius = float(fields["radii_m"][index])
            expected = [None if r > radius + 1e-12 else
                        round(float(np.interp(r / radius, fields["xi"], fields["moisture"][index])), 4)
                        for r in np.arange(20) * 0.001]
            expected.append(round(float(fields["moisture"][index, -1]), 4))
        mismatches += sum(actual != target for actual, target in zip(row[1:], expected))
    )
verification["source_value_mismatches"] = mismatches
verification["time_schedule_valid"] = schedule_valid
verification["source_fields_sha256"] = hashlib.sha256(fields_path.read_bytes()).hexdigest()
verification["workbook_sha256"] = hashlib.sha256(path.read_bytes()).hexdigest()
verification["pass"] = verification["pass"] and schedule_valid and mismatches == 0
book.close()
return verification

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    outputs = [
        write_fixed_workbook(
            "result1.xlsx",
            "result1.xlsx",
            RESULTS_DIR / "q1" / "fields.npz",
            include_temperature=True,
            end_time_s=1800.0,
        ),
        write_fixed_workbook(
            "result2.xlsx",
            "result2.xlsx",
            RESULTS_DIR / "q2" / "fields.npz",
            include_temperature=True,
            end_time_s=10800.0,
        ),
    ]

```

```

        write_fixed_workbook(
            "result3.xlsx",
            "result3.xlsx",
            RESULTS_DIR / "q3" / "fields.npz",
            include_temperature=False,
        ),
        write_moving_workbook(),
    ]
    report = {
        "authoring_backend": "existing openpyxl result exporter",
        "template_policy": "保留附件模板表头、工作表名称和样式；展开省略号，结果显示四位小数。",

        "outputs": outputs,
        "verification": [verify_workbook(item) for item in outputs],
    }
    report_path = RESULTS_DIR / "workbook_verification.json"
    report_path.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    if not all(item["pass"] for item in report["verification"]):
        raise RuntimeError("Workbook verification failed; see workbook_verification.json")
    print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

2.3 check_numerics.py

```

# File: code/check_numerics.py
"""Independent spatial/time refinement checks; does not overwrite production fields."""
from pathlib import Path
import json
import sys
import numpy as np
import solve_drying as model

def run():
    environment=model.load_environment(); radius=model.load_radius_history()
    saved_nodes=model.NODES
    report={'grid_power':model.GRID_POWER, 'grid':[], 'time':[]}
    for nodes in [41,81,161,321]:
        model.NODES=nodes
        q3=model.simulate_fixed(environment,model.property_q23,None,10,60,True)
        q4=model.simulate_moving(environment,radius,10,60)

```

```

        record={'nodes':nodes,'q3_h':q3['threshold_time_s']/3600,'q4_h':q4['
threshold_time_s']/3600}
        report['grid'].append(record);print(record,flush=True)
        (model.RESULTS_DIR/'numerical_verification.json').write_text(json.dumps(report,
indent=2))
model.NODES=saved_nodes
for dt in [2.,1.]:
    q3=model.simulate_fixed(environment,model.property_q23,None,dt,60,True)
    q4=model.simulate_moving(environment,radius,dt,60)
    record={'dt_s':dt,'nodes':model.NODES,'q3_h':q3['threshold_time_s']/3600,'q4_h':q4[
'threshold_time_s']/3600}
    report['time'].append(record);print(record,flush=True)
report['spatial_relative_change_161_to_321']={q:abs(report['grid'][-1][q]-report['grid'
][-2][q])/report['grid'][-1][q] for q in ['q3_h','q4_h']}
report['time_relative_change_2_to_1']={q:abs(report['time'][-1][q]-report['time'][-2][q
])/report['time'][-1][q] for q in ['q3_h','q4_h']}
report['time_tolerance']=1e-4
report['spatial_tolerance']=1e-3
report['pass']=all(v<report['spatial_tolerance'] for v in report['
spatial_relative_change_161_to_321'].values()) and all(v<report['time_tolerance'] for v
in report['time_relative_change_2_to_1'].values())
(model.RESULTS_DIR/'numerical_verification.json').write_text(json.dumps(report,indent
=2))

if __name__=='__main__':
    sys.stdout.reconfigure(encoding='utf-8');run()

```

2.4 validate_2d.py

```

# File: code/validate_2d.py
"""Short-duration Q1 validation using the matched-grid axisymmetric solver.

Whole-run extrema and final-time differences are explicitly distinguished.
For the full drying process, run validate_long_2d.py.
"""

import json
import sys
from solve_drying import RESULTS_DIR
from validate_long_2d import run_case

def run():
    report = run_case(1, 41, 21, 10.0)
    report['one_dimensional_midplane_pass'] = (

```

```

        report['whole_run_midplane_max_temperature_difference'] < 0.1
        and report['whole_run_midplane_max_moisture_difference'] < 1e-3)
    output = RESULTS_DIR / 'q1' / 'validation_2d.json'
    output.write_text(json.dumps(report, indent=2), encoding='utf-8')
    print(json.dumps(report, indent=2))

if __name__ == '__main__':
    sys.stdout.reconfigure(encoding='utf-8')
    run()

```

2.5 validate_long_2d.py

```

# File: code/validate_long_2d.py
"""Full-duration axisymmetric validation with matched 1D grid and time step.

z=0 is the symmetry mid-plane; z=L/2 is the exposed end.
For Q4 both solvers use the homogeneous material-shrinkage closure.
"""

import json, math, sys
import numpy as np
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import spsolve
import solve_drying as m

LENGTH=.25

def geometry(nr,nz,radius):
    r=m.radial_grid(nr,radius);z=m.radial_grid(nz,LENGTH/2)
    area,_=m.node_control_volumes(r)
    faces=(z[1:]+z[:-1])/2
    dz=np.diff(np.r_[0.,faces,LENGTH/2])
    return r,z,area,dz,area[:,None]*dz[None,:]

def step2d(old,storage,k,dt,h,ambient,geom):
    r,z,area,dz,volume=geom;nr,nz=old.shape
    ids=np.arange(nr*nz).reshape(nr,nz)
    a=ids[:-1,:].ravel();b=ids[1,:].ravel()
    gr=(2*np.pi*((r[1:]+r[:-1])/2)[:,None]*dz[None,:])*(k[1:,:]+k[:-1,:])/2/np.diff(r)[:,None]).ravel()
    c=ids[:,:-1].ravel();d=ids[:,1:].ravel()
    gz=(area[:,None]*(k[:,1:]+k[:,:-1])/2/np.diff(z)[None,:]).ravel()
    first=np.r_[a,c];second=np.r_[b,d];g=np.r_[gr,gz]
    capacity=(storage*volume/dt).ravel()

```

```

diagonal=capacity+np.bincount(first,weights=g,minlength=old.size)+np.bincount(second,
weights=g,minlength=old.size)
rhs=capacity*old.ravel()
side=ids[-1,:];end=ids[:,-1]
bs=h*2*np.pi*r[-1]*dz;be=h*area
diagonal[side]+=bs;rhs[side]+=bs*ambient
diagonal[end]+=be;rhs[end]+=be*ambient
ix=np.arange(old.size)
matrix=coo_matrix((np.r_[diagonal,-g,-g],(np.r_[ix,first,second],np.r_[ix,second,first
])),shape=(old.size,old.size)).tocsc()
return spsolve(matrix,rhs).reshape(old.shape)

```

```

def run_case(question,nr,nz,dt):
    env=m.load_environment();rh=m.load_radius_history()
    prop={1:m.property_q1,3:m.property_q23,4:m.property_q4}[question]
    temperature=np.full((nr,nz),m.INITIAL_T);moisture=np.full_like(temperature,m.INITIAL_C)
    one_t=temperature[:,0].copy();one_c=moisture[:,0].copy()
    td2=td1=None;maxdiff_t=maxdiff_c=0.;max_end_c=0.;max_location=None
    previous2=previous1=m.INITIAL_C
    maximum=1800 if question==1 else (float(rh.time_s[-1]) if question==4 else 4*86400)
    steps=0
    for t in np.arange(dt,maximum+dt/2,dt):
        radius=rh.values(float(t))[0] if question==4 else m.R_FIXED
        geom=geometry(nr,nz,radius)
        ambient_t,ambient_c=env.values(float(t))
        guess_t=temperature.copy();guess_c=moisture.copy()
        for iteration in range(30):
            rho,cp,k,_=prop(guess_t,guess_c)
            new_t=step2d(temperature,rho*cp,k,dt,m.H_T,ambient_t,geom)
            _,_,_,diffusivity=prop(new_t,guess_c)
            new_c=step2d(moisture,np.ones_like(moisture),diffusivity,dt,m.H_M,ambient_c,
geom)

            et=np.max(np.abs(new_t-guess_t));ec=np.max(np.abs(new_c-guess_c))
            guess_t,guess_c=new_t,new_c
            if et<1e-8 and ec<1e-10: break
        else: raise RuntimeError('2D Picard failed')
        temperature,moisture=guess_t,guess_c
        one_t,one_c,_=m.advance_coupled(one_t,one_c,dt,radius,0.,ambient_t,ambient_c,prop,
question==4)
        maxdiff_t=max(maxdiff_t,float(np.max(np.abs(temperature[:,0]-one_t))))
        maxdiff_c=max(maxdiff_c,float(np.max(np.abs(moisture[:,0]-one_c))))
        max_end_c=max(max_end_c,float(np.max(np.abs(moisture[:,-1]-moisture[:,0]))))
        current2=float(moisture.max());current1=float(one_c.max())
        if td2 is None and current2<=m.THRESHOLD_C:
            td2=float(t-dt+dt*(previous2-m.THRESHOLD_C)/(previous2-current2))
            ij=np.unravel_index(np.argmax(moisture),moisture.shape)

```

```

        max_location=[float(geom[0][ij[0]]),float(geom[1][ij[1]])]
    if td1 is None and current1<=m.THRESHOLD_C:
        td1=float(t-dt+dt*(previous1-m.THRESHOLD_C)/(previous1-current1))
    previous2,previous1=current2,current1;steps+=1
    if steps%720==0:print('2D',question,nr,nz,'time h',t/3600,flush=True)
    if td2 is not None and td1 is not None:break
    return {'question':question,'nr':nr,'nz':nz,'dt_s':dt,'end_s':float(t),
        'terminal_temperature_c':m.TERMINAL_TEMPERATURE_C,'terminal_moisture':m.
    TERMINAL_MOISTURE,
        'threshold_2d_h':None if td2 is None else td2/3600,'threshold_matched_1d_h':None if
    td1 is None else td1/3600,
        'relative_threshold_difference':None if td2 is None else abs(td2-td1)/td2,
        'global_control_r_z_m':max_location,'whole_run_midplane_max_temperature_difference'
    :maxdiff_t,
        'whole_run_midplane_max_moisture_difference':maxdiff_c,
        'whole_run_end_midplane_max_moisture_difference':max_end_c,
        'final_midplane_temperature_difference':float(np.max(np.abs(temperature[:,0]-one_t)
    )),
        'final_midplane_moisture_difference':float(np.max(np.abs(moisture[:,0]-one_c)))})

def run():
    path=m.RESULTS_DIR/'long_2d_verification.json'
    previous=json.loads(path.read_text()) if path.exists() else []
    # Q1 ends before the terminal boundary begins and remains valid. Recompute
    # every long-duration Q3/Q4 case whenever this script is run.
    records=[x for x in previous if x['question']==1]
    for q,nr,nz,dt in [(3,41,21,30.), (4,41,21,30.), (3,81,41,30.), (4,81,41,30.)]:
        record=run_case(q,nr,nz,dt);records.append(record)
        path.write_text(json.dumps(records,indent=2));print(record,flush=True)

if __name__=='__main__':
    sys.stdout.reconfigure(encoding='utf-8');run()

```

2.6 verify_revision.py

```

# File: code/verify_revision.py
"""Check trajectory identity, refinement, closed-system invariance and balances.

Does not write workbooks or regenerate production results.
"""
import hashlib
import json
from pathlib import Path

```

```

import numpy as np
import solve_drying as m

def run():
    output = {}
    with np.load(m.RESULTS_DIR/'q2/fields.npz') as q2, np.load(m.RESULTS_DIR/'q3/fields.npz') as q3:
        output['q2_q3_exact_shared_trajectory'] = all(
            np.array_equal(q2[key][::60], q3[key])
            for key in ['times_s', 'temperature_c', 'moisture'])
        assert output['q2_q3_exact_shared_trajectory']
        assert np.array_equal(q2['threshold_moisture'], q3['threshold_moisture'])
    env=m.load_environment()
    refinement={}
    original_nodes=m.NODES
    try:
        for q,prop,end in [(1,m.property_q1,1800.), (2,m.property_q23,10800.)]:
            results=[]
            for n in [161,321]:
                m.NODES=n
                results.append(m.simulate_fixed(env,prop,end,1.,60.,False))
                refinement[f'q{q}']={
                    'maximum_output_temperature_difference_c':float(np.max(np.abs(results[0]['temperature_c']-results[1]['temperature_c']))),
                    'maximum_output_moisture_difference':float(np.max(np.abs(results[0]['moisture']-results[1]['moisture'])))
                }
                assert refinement[f'q{q}']['maximum_output_temperature_difference_c']<1e-3
                assert refinement[f'q{q}']['maximum_output_moisture_difference']<1e-4
            finally:m.NODES=original_nodes
        output['short_term_refinement_161_to_321_dt1']=refinement
        # Zero exchange and a uniform field must remain unchanged even as R shrinks.
        old_t=np.full(m.NODES,m.INITIAL_T);old_c=np.full(m.NODES,1.2)
        t=old_t.copy();c=old_c.copy();b=m.new_balance()
        for radius in np.linspace(.02,.012,21):
            t,c,_=m.advance_coupled(t,c,10.,radius,0.,m.INITIAL_T,1.2,m.property_q4,True,
            balance=b)
        output['uniform_material_field_temperature_error']=float(np.max(np.abs(t-old_t)))
        output['uniform_material_field_moisture_error']=float(np.max(np.abs(c-old_c)))
        assert output['uniform_material_field_temperature_error']<1e-8
        assert output['uniform_material_field_moisture_error']<1e-10
        output['production_balances']={}
        for q in range(1,5):
            summary=json.loads((m.RESULTS_DIR/f'q{q}/summary.json').read_text(encoding='utf-8'))
            output['production_balances'][f'q{q}']=summary['balance']
            assert summary['balance']['cumulative_mass_equation_residual']<1e-9

```

```

        assert summary['balance'][ 'maximum_step_heat_equation_residual']<1e-10
    output['scope']='Mass balance of implemented dry-reference equation; effective heat
    equation, not total enthalpy.'
    output['source_sha256']={name:hashlib.sha256((Path(__file__).parent/name).read_bytes())
    .hexdigest() for name in ['solve_drying.py', 'validate_long_2d.py', 'run_sensitivity.py'
    ]}
    output['pass']=True
    (m.RESULTS_DIR/'revision_checks.json').write_text(json.dumps(output,indent=2),encoding=
    'utf-8')
    print(json.dumps(output,indent=2))

if __name__=='__main__':run()

```

2.7 run_sensitivity.py

```

# File: code/run_sensitivity.py
"""Boundary-condition sensitivity for the Q3 and Q4 drying times."""

from __future__ import annotations

import json
import sys

from solve_drying import (
    RESULTS_DIR,
    TERMINAL_TEMPERATURE_C,
    load_environment,
    load_radius_history,
    property_q23,
    simulate_fixed,
    simulate_moving,
)

SCENARIOS = {
    "baseline": (1.0, 1.0),
    "temperature_minus_5pct": (0.95, 1.0),
    "temperature_plus_5pct": (1.05, 1.0),
    "ambient_moisture_minus_5pct": (1.0, 0.95),
    "ambient_moisture_plus_5pct": (1.0, 1.05),
}

def run() -> None:

```

```

environment = load_environment()
radius_history = load_radius_history()
output: dict[str, dict[str, float]] = {}
for name, (temperature_scale, moisture_scale) in SCENARIOS.items():
    q3 = simulate_fixed(
        environment,
        property_q23,
        end_time_s=None,
        dt_s=10.0,
        output_interval_s=3600.0,
        stop_at_threshold=True,
        terminal_temp_scale=temperature_scale,
        terminal_moisture_scale=moisture_scale,
    )
    q4 = simulate_moving(
        environment,
        radius_history,
        dt_s=10.0,
        output_interval_s=3600.0,
        stop_at_threshold=True,
        terminal_temp_scale=temperature_scale,
        terminal_moisture_scale=moisture_scale,
    )
    output[name] = {
        "terminal_temperature_scale": temperature_scale,
        "terminal_moisture_scale": moisture_scale,
        "q3_drying_time_h": float(q3["threshold_time_s"]) / 3600.0,
        "q4_drying_time_h": float(q4["threshold_time_s"]) / 3600.0,
    }

baseline = output['baseline']
delta_temperature_c = 0.05 * TERMINAL_TEMPERATURE_C
baseline_temperature_k = TERMINAL_TEMPERATURE_C + 273.15
output['definitions'] = {
    'temperature_perturbation_c': delta_temperature_c,
    'temperature_definition': 'additive Celsius perturbation; dimensionless derivative
normalized by Kelvin temperature',
    'dt_s': 10.0, 'radius_history_fixed': True,
}
for q in ['q3', 'q4']:
    key=f'{q}_drying_time_h'
    derivative=(output['temperature_plus_5pct'][key]-output['temperature_minus_5pct'][
key])/(2*delta_temperature_c)
    moisture_elasticity=(output['ambient_moisture_plus_5pct'][key]-output['
ambient_moisture_minus_5pct'][key])/(0.1*baseline[key])
    output['definitions'][q]={ 'temperature_derivative_h_per_c':derivative,
        'temperature_kelvin_elasticity':derivative*baseline_temperature_k/baseline[key]

```

```

],
    'ambient_moisture_elasticity':moisture_elasticity}
path = RESULTS_DIR / "sensitivity.json"
path.write_text(json.dumps(output, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps(output, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8")
    run()

```

2.8 run_q4_counterfactual.py

```

# File: code/run_q4_counterfactual.py
"""Counterfactual Q4 run using Appendix-4 properties but a fixed radius."""

from __future__ import annotations

import json
import sys

from solve_drying import (
    RESULTS_DIR,
    load_environment,
    property_q4,
    save_npz,
    simulate_fixed,
)

def main() -> None:
    environment = load_environment()
    simulation = simulate_fixed(
        environment,
        property_q4,
        end_time_s=None,
        dt_s=1.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    save_npz(RESULTS_DIR / "q4" / "fixed_radius_counterfactual.npz", simulation)
    summary_path = RESULTS_DIR / "q4" / "summary.json"
    summary = json.loads(summary_path.read_text(encoding="utf-8"))
    fixed_time_h = float(simulation["threshold_time_s"]) / 3600.0
    moving_time_h = float(summary["drying_time_h"])

```

```

summary["fixed_radius_counterfactual"] = {
    "same_appendix4_properties_time_h": fixed_time_h,
    "moving_radius_time_h": moving_time_h,
    "time_reduction_h": fixed_time_h - moving_time_h,
    "relative_reduction": (fixed_time_h - moving_time_h) / fixed_time_h,
}
summary_path.write_text(
    json.dumps(summary, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
print(json.dumps(summary["fixed_radius_counterfactual"], ensure_ascii=False, indent=2))

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8")
    main()

```

2.9 validate_moving_mesh.py

```

# File: code/validate_moving_mesh.py
"""Independent physical-radius moving-control-volume validation of Q4.

No transformed differential operator or  $R^{**}(-2)$  enters the physical solver.
The reference path calls the unchanged fixed-domain advance_coupled routine.
Only outputs under results/q4/validation_moving_mesh are written here.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import math
from pathlib import Path
import sys
import time

sys.dont_write_bytecode = True
import numpy as np
from scipy.linalg import solve_banded
import solve_drying as main

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / 'results/q4/validation_moving_mesh'
TOL_T, TOL_C, MAX_IT = 1e-8, 1e-10, 30

```

```

def physical_geometry(nodes: np.ndarray) -> dict:
    """Physical annuli, per unit axial length; centre and surface half-cells."""
    bounds = np.r_[0.0, (nodes[:-1] + nodes[1:]) / 2.0, nodes[-1]]
    # Factored square difference avoids cancellation in the thin surface cell.
    volumes = math.pi * np.diff(bounds) * (bounds[1:] + bounds[:-1])
    assert np.all(np.diff(nodes) > 0) and np.all(volumes > 0)
    return dict(nodes=nodes, bounds=bounds, volumes=volumes,
                areas=2.0 * math.pi * bounds, distances=np.diff(nodes))

def mesh_motion(old: dict, new: dict, dt: float) -> tuple[np.ndarray, np.ndarray]:
    """Swept-volume flux and discrete GCL residual, computed at every face."""
    speed = (new['bounds'] - old['bounds']) / dt
    average_area = math.pi * (new['bounds'] + old['bounds'])
    swept_flux = average_area * speed
    residual = new['volumes'] - old['volumes'] - dt * np.diff(swept_flux)
    return speed, residual

def solve_physical_inventory(old_value, old_capacity, new_capacity,
                             conductivity, geometry, dt, transfer, ambient,
                             inventory_correction=None):
    """Solve extensive inventory balance with physical area / distance fluxes.

    capacity has units inventory per unit scalar (e.g. dry mass per axial m).
    inventory_correction is an extensive change, not an advection coefficient.
    """
    g = geometry
    face_k = 0.5 * (conductivity[:-1] + conductivity[1:])
    conductance = g['areas'][1:-1] * face_k / g['distances']
    surface = g['areas'][-1] * transfer
    diagonal = new_capacity / dt
    diagonal = diagonal.copy()
    diagonal[:-1] += conductance
    diagonal[1:] += conductance
    diagonal[-1] += surface
    rhs = old_capacity * old_value / dt
    if inventory_correction is not None:
        rhs += inventory_correction / dt
    rhs[-1] += surface * ambient
    band = np.zeros((3, len(old_value)))
    band[0, 1:] = -conductance
    band[1] = diagonal
    band[2, :-1] = -conductance
    result = solve_banded((1, 1), band, rhs, check_finite=False)

```

```

return result, conductance, surface

def advance_moving_control_volumes(old_t, old_c, old_g, new_g, dry_mass_total,
                                    dt, ambient_t, ambient_c,
                                    heat_transfer=main.H_T, mass_transfer=main.H_M):
    """Picard iteration for co-moving material cells (v_s=w).

    Water:  $d(M_d C)/dt + \sum(\rho_d D A \text{ grad } C) + \text{boundary loss} = 0$ .
    Heat: preserve the existing effective a D_t T equation. Its extensive
    representation requires T_old * delta(a V), rather than conserving a V T
    without a source. This correction includes geometry AND variable storage.
    """
    rho_d_old = dry_mass_total / old_g['volumes'].sum()
    rho_d_new = dry_mass_total / new_g['volumes'].sum()
    mass_old = rho_d_old * old_g['volumes']
    mass_new = rho_d_new * new_g['volumes']
    old_rho, old_cp, _, _ = main.property_q4(old_t, old_c)
    old_heat_capacity = old_rho * old_cp * old_g['volumes']
    t, c = old_t.copy(), old_c.copy()
    for iteration in range(1, MAX_IT + 1):
        rho, cp, k, _ = main.property_q4(t, c)
        new_heat_capacity = rho * cp * new_g['volumes']
        correction = old_t * (new_heat_capacity - old_heat_capacity)
        t_next, _, heat_surface = solve_physical_inventory(
            old_t, old_heat_capacity, new_heat_capacity, k, new_g, dt,
            heat_transfer, ambient_t, inventory_correction=correction)
        _, _, _, diffusivity = main.property_q4(t_next, c)
        c_next, _, water_surface = solve_physical_inventory(
            old_c, mass_old, mass_new, rho_d_new * diffusivity, new_g, dt,
            rho_d_new * mass_transfer, ambient_c)
        error_t = np.max(np.abs(t_next - t))
        error_c = np.max(np.abs(c_next - c))
        t, c = t_next, c_next
        if error_t < TOL_T and error_c < TOL_C:
            break
    else:
        raise RuntimeError(f'Physical Picard failed: {error_t=}, {error_c=}')
    if not np.isfinite(t).all() or not np.isfinite(c).all():
        raise FloatingPointError('Nonfinite physical solution')
    if c.min() < -1e-10 or c.max() > main.INITIAL_C + 1e-8:
        raise FloatingPointError('Moisture outside physical bounds')
    water_change = np.sum(mass_new * c - mass_old * old_c)
    water_loss = dt * water_surface * (c[-1] - ambient_c)
    # Report BOTH extensive-inventory closure and effective-heat closure.
    heat_inventory_change = np.sum(new_heat_capacity * t - old_heat_capacity * old_t)
    heat_correction = np.sum(correction)

```

```

heat_loss = dt * heat_surface * (t[-1] - ambient_t)
return t, c, iteration, dict(
    water_change=float(water_change), water_loss=float(water_loss),
    water_residual=float(water_change + water_loss),
    dry_cell_change=float(np.max(np.abs(mass_new - mass_old))),
    heat_residual=float(heat_inventory_change - heat_correction + heat_loss),
    heat_reference=float(main.INITIAL_T * new_heat_capacity.sum()))

def preservation_manifest():
    files = [ROOT/'code/solve_drying.py', ROOT/'code/write_results.py']
    files += list((ROOT/'results').rglob('*.npz')) + list((ROOT/'results').rglob('*.json'))
    files += list(ROOT.rglob('result4.xlsx'))
    for directory in ['figures', 'drawio', 'paper/cumcm-1.1.0/figures']:
        files += list((ROOT/directory).glob('*'))
    return {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
            for p in files if p.is_file() and OUT not in p.parents}

def check_preservation(before):
    changed = [k for k,v in before.items()
               if hashlib.sha256((ROOT/k).read_bytes()).hexdigest() != v]
    if changed:
        raise RuntimeError(f'Protected files changed: {changed}')
    return len(before)

def run_checks():
    """Isolated physics tests: GCL, insulated material, stationary-grid limit."""
    xi=main.radial_grid(main.NODES)
    g0=physical_geometry(.02*xi); g1=physical_geometry(.013*xi)
    _,res=mesh_motion(g0,g1,1.0)
    mass=g0['volumes'].sum() # rho_d(0)=1; arbitrary normalization cancels.
    tt=np.full(main.NODES,28.0); cc=np.full(main.NODES,2.55)
    t,c,_,diag=advance_moving_control_volumes(tt,cc,g0,g1,mass,1,28,0,
                                              heat_transfer=0,mass_transfer=0)

    assert np.max(np.abs(c-cc)) < 1e-9
    assert np.max(np.abs(t-tt)) < 1e-7
    assert np.max(np.abs(res)) / mass < 1e-12
    # Smooth nonuniform state, zero mesh motion: same physical PDE as baseline.
    tt=28+4*xi**2; cc=2.55-.5*xi**2
    tp,cp,_,_=advance_moving_control_volumes(tt,cc,g0,g0,mass,1,40,.03)
    tr,cr,_=main.advance_coupled(tt,cc,1,.02,0,40,.03,main.property_q4,False)
    assert np.max(np.abs(tp-tr))<1e-7
    assert np.max(np.abs(cp-cr))<1e-9
    return dict(gcl_normalized_max=float(np.max(np.abs(res))/mass),
               insulated_temperature_max_error=float(np.max(np.abs(t-28))),

```

```

        insulated_moisture_max_error=float(np.max(np.abs(c-2.55))),
        stationary_temperature_max_error=float(np.max(np.abs(tp-tr))),
        stationary_moisture_max_error=float(np.max(np.abs(cp-cr)))

def simulate_moving_mesh(dt_s=1.0, max_time_s=259200.0):
    """Integrate both implementations independently on the same time levels.

    Save full 161-node fields every minute and centre/surface/errors every step.
    No reference state is injected into the physical iteration.
    """
    env=main.load_environment(); radius=main.load_radius_history()
    xi=main.radial_grid(main.NODES)
    geometry=physical_geometry(radius.values(0)[0]*xi)
    dry_mass_total=geometry['volumes'].sum()
    t_map=np.full(main.NODES,main.INITIAL_T); c_map=np.full(main.NODES,main.INITIAL_C)
    t_mov=t_map.copy(); c_mov=c_map.copy()
    events={}
    times=[0.0]; radii=[.02]
    fields_t_map=[t_map.copy()]; fields_t_mov=[t_mov.copy()]
    fields_c_map=[c_map.copy()]; fields_c_mov=[c_mov.copy()]
    trace=[[0.0,c_map[0],c_mov[0],c_map[-1],c_mov[-1],0.0,0.0,0.0]]
    balances=dict(water_change=0.,water_loss=0.,maximum_step_water_residual=0.,
                  maximum_heat_residual=0.,maximum_dry_cell_relative_drift=0.,
                  maximum_gcl_residual=0.,max_relative_mesh_speed=0.)
    iters=[0,0]; iter_sum=np.zeros(2); monotone=True
    tnow=0.0; next_output=60.0; start=time.perf_counter()
    while tnow < min(max_time_s,float(radius.time_s[-1]))-1e-8:
        step=min(dt_s,next_output-tnow,max_time_s-tnow)
        newtime=tnow+step
        r,r_dot=radius.values(newtime); te,ce=env.values(newtime)
        new_geometry=physical_geometry(r*xi)
        speed,gcl=mesh_motion(geometry,new_geometry,step)
        # Solid faces follow the identical trajectories. The relative flux
        # rho_d C (v_s-w) A is identically zero, not silently omitted.
        solid_face_speed=speed.copy()
        balances['max_relative_mesh_speed']=max(balances['max_relative_mesh_speed'],
                                                float(np.max(abs(solid_face_speed-speed))))

        old_t_map,old_c_map=t_map,c_map
        old_t_mov,old_c_mov=t_mov,c_mov
        t_map,c_map,im=main.advance_coupled(t_map,c_map,step,r,0.0,te,ce,main.property_q4,
        True)
        t_mov,c_mov,ip,diag=advance_moving_control_volumes(
            t_mov,c_mov,geometry,new_geometry,dry_mass_total,step,te,ce)
        for name,oldt,oldc,tt,cc in [('mapping',old_t_map,old_c_map,t_map,c_map),
                                     ('moving_mesh',old_t_mov,old_c_mov,t_mov,c_mov)]:
            if name not in events and oldc.max()>main.THRESHOLD_C>=cc.max():

```

```

        fraction=(oldc.max()-main.THRESHOLD_C)/(oldc.max()-cc.max())
        et=tnow+fraction*step
        events[name]=dict(time_s=et,time_h=et/3600,
                           temperature=oldt+fraction*(tt-oldt),
                           moisture=oldc+fraction*(cc-oldc),
                           radius_m=radius.values(et)[0])

    reference=dry_mass_total*main.INITIAL_C
    balances['water_change']+=diag['water_change']/reference
    balances['water_loss']+=diag['water_loss']/reference
    balances['maximum_step_water_residual']=max(balances['maximum_step_water_residual'],abs(diag['water_residual']/reference))
    balances['maximum_heat_residual']=max(balances['maximum_heat_residual'],abs(diag['heat_residual']/diag['heat_reference']))
    balances['maximum_dry_cell_relative_drift']=max(balances['maximum_dry_cell_relative_drift'],float(np.max(abs((dry_mass_total/new_geometry['volumes'].sum())*new_geometry['volumes']/(geometry['volumes']*(dry_mass_total/geometry['volumes'].sum()))-1))))
    balances['maximum_gcl_residual']=max(balances['maximum_gcl_residual'],float(np.max(abs(gcl))/geometry['volumes'].sum()))
    monotone=monotone and bool(np.max(np.diff(c_mov))<=2e-9)
    iters=[max(iters[0],im),max(iters[1],ip)]; iter_sum += [im,ip]
    dc=float(np.max(abs(c_map-c_mov))); dtemp=float(np.max(abs(t_map-t_mov)))
    trace.append([newtime,c_map[0],c_mov[0],c_map[-1],c_mov[-1],dc,dtemp,abs(c_map[0]-c_mov[0])])
    tnow=newtime;geometry=new_geometry
    if tnow>=next_output-1e-8:
        times.append(tnow);radii.append(r)
        fields_t_map.append(t_map.copy());fields_t_mov.append(t_mov.copy())
        fields_c_map.append(c_map.copy());fields_c_mov.append(c_mov.copy())
        next_output += 60.0
        if int(round(tnow))%21600==0:
            print(f'{tnow/3600:.0f} h: centre C={c_mov[0]:.9f}, max |dC|={dc:.3e}, elapsed={time.perf_counter()-start:.1f}s',flush=True)
            if len(events)==2:
                break
    arrays=dict(times_s=np.array(times),radii_m=np.array(radii),xi=xi,
                mapping_temperature=np.array(fields_t_map),moving_temperature=np.array(fields_t_mov),
                mapping_moisture=np.array(fields_c_map),moving_moisture=np.array(fields_c_mov),
                step_trace=np.array(trace))
    for name,e in events.items():
        for key in ['temperature','moisture']:
            arrays[f'{name}_event_{key}']=e[key]
    np.savez_compressed(OUT/'fields.npz',**arrays)
    balances['cumulative_water_residual']=abs(balances['water_change']+balances['water_loss'])

```

```

report=dict(dt_s=dt_s,nodes=main.NODES,tolerances=dict(temperature=TOL_T,moisture=TOL_C
,max_iterations=MAX_IT),
            elapsed_seconds=time.perf_counter()-start,time_steps=len(trace)-1,
            picard_max=dict(mapping=iters[0],moving_mesh=iters[1]),
            picard_mean=dict(mapping=float(iter_sum[0]/(len(trace)-1)),moving_mesh=
float(iter_sum[1]/(len(trace)-1))),
            monotone_profiles=monotone,balances=balances,
            events={name:{k:float(v) for k,v in e.items() if k not in ['temperature','
moisture']}} for name,e in events.items()),
            max_abs_moisture_field_difference=float(arrays['step_trace'][:,5].max()),
            max_abs_temperature_field_difference=float(arrays['step_trace'][:,6].max())
,
            max_abs_center_moisture_difference=float(arrays['step_trace'][:,7].max()))
if len(events)==2:
    delta=abs(events['mapping']['time_s']-events['moving_mesh']['time_s'])
    report['drying_time_abs_difference_s']=delta
    report['drying_time_relative_difference']=delta/events['mapping']['time_s']
    report['event_comparison_note']='Own-event profiles and common mapping-event centre
/surface values are distinguished; trajectories compare identical time levels.'
(OUT/'summary.json').write_text(json.dumps(report,indent=2),encoding='utf8')
return arrays,report

def write_comparisons(arrays,report):
    rows=[]
    def add(metric,a,b,unit):
        rows.append(dict(metric=metric,unit=unit,mapping=float(a),moving_mesh=float(b),
            absolute_difference=abs(float(a)-float(b)),
            relative_difference=abs(float(a)-float(b))/abs(float(a)) if a else
None))
    if len(report['events'])!=2:
        return
    add('drying_time',report['events']['mapping']['time_h'],report['events']['moving_mesh'
]['time_h'],'h')
    profiles=[]
    for label,seconds in [('6h',21600),('24h',86400),('48h',172800),('own_event',None)]:
        if seconds is None:
            ca=arrays['mapping_event_moisture'];cb=arrays['moving_mesh_event_moisture']
            ta=arrays['mapping_event_temperature'];tb=arrays['moving_mesh_event_temperature
']
            ra=report['events']['mapping']['radius_m']; rb=report['events']['moving_mesh']['
radius_m']
        else:
            i=int(np.flatnonzero(arrays['times_s']==seconds)[0])
            ca=arrays['mapping_moisture'][i];cb=arrays['moving_moisture'][i]
            ta=arrays['mapping_temperature'][i];tb=arrays['moving_temperature'][i]
            ra=rb=arrays['radii_m'][i]

```

```

    for point,k in [ ('center',0), ('surface',-1)]:
        add(f'{label}_{point}_moisture',ca[k],cb[k], 'kg/kg')
    for j,xi in enumerate(arrays['xi']):
        profiles.append(dict(time_label=label,node=j,r_mapping_cm=100*ra*xi,r_moving_cm
=100*rb*xi,
                                C_mapping=ca[j],C_moving=cb[j],T_mapping=ta[j],T_moving=tb
[j]))
shared_time=report['events']['mapping']['time_s']
trace=arrays['step_trace']
for point,ia,ib in [ ('center',1,2), ('surface',3,4)]:
    add(f'common_event_{point}_moisture',np.interp(shared_time,trace[:,0],trace[:,ia]),
        np.interp(shared_time,trace[:,0],trace[:,ib]), 'kg/kg')
# Compute the time relative difference in seconds consistently with summary.
rows[0]['relative_difference']=report['drying_time_relative_difference']
for name,data in [ ('comparison.csv',rows), ('profiles.csv',profiles)]:
    with (OUT/name).open('w',newline='',encoding='utf-8-sig') as stream:
        writer=csv.DictWriter(stream,fieldnames=data[0].keys());writer.writeheader();
writer.writerow(data)
text='| Metric | Unit | Fixed mapping | Physical moving mesh | Relative difference |\n
|---|---|---:|---:|---:|\n'
for row in rows:
    text+=f"| {row['metric']} | {row['unit']} | {row['mapping']:.12g} | {row['
moving_mesh']:.12g} | {row['relative_difference']:.4e} |\n"
(OUT/'comparison.md').write_text(text,encoding='utf8')
# Match regenerated reference fields to the pre-existing output, without
# changing original files or using them as integration states.
old=np.load(ROOT/'results/q4/fields.npz')
source=json.loads((ROOT/'results/q4/summary.json').read_text(encoding='utf8'))
assert np.array_equal(old['times_s'],arrays['times_s'])
mapped_c=np.array([np.interp(old['xi'],arrays['xi'],row) for row in arrays['
mapping_moisture']])
mapped_t=np.array([np.interp(old['xi'],arrays['xi'],row) for row in arrays['
mapping_temperature']])
reproduction=dict(max_C_difference=float(np.max(abs(mapped_c-old['moisture']))),
                    max_T_difference=float(np.max(abs(mapped_t-old['temperature_c']))),
                    drying_time_difference_s=abs(report['events']['mapping']['time_s']-
source['drying_time_s']))
(OUT/'reference_reproduction.json').write_text(json.dumps(reproduction,indent=2),
encoding='utf8')
assert reproduction['max_C_difference']<1e-12 and reproduction['max_T_difference']<1e
-10
assert reproduction['drying_time_difference_s']<1e-6

def run():
    sys.stdout.reconfigure(encoding='utf8')
    parser=argparse.ArgumentParser();parser.add_argument('--check-only',action='store_true'

```

```

)
args=parser.parse_args()
OUT.mkdir(parents=True,exist_ok=True)
before=preservation_manifest()
(OUT/'protected_files_sha256.json').write_text(json.dumps(before,ensure_ascii=False,
indent=2),encoding='utf8')
checks=run_checks();(OUT/'unit_checks.json').write_text(json.dumps(checks,indent=2),
encoding='utf8')
print('Physics checks:',checks,flush=True)
if not args.check_only:
    arrays,report=simulate_moving_mesh()
    write_comparisons(arrays,report)
    assert len(report['events'])==2 and report['monotone_profiles']
    print(json.dumps(report,indent=2),flush=True)
print('Protected files verified:',check_preservation(before),flush=True)

if __name__=='__main__':
    run()

```

2.10 check_moving_mesh_delivery.py

```

# File: code/check_moving_mesh_delivery.py
"""Verify saved Q4 results and the paper without integrating or packaging by default.

Example: python check_moving_mesh_delivery.py --log ../paper/cumcm-1.1.0/build/main.log
Use --report to choose the JSON destination, and --package only to copy main.pdf
to the submission filename. Historical protection hashes are provenance, not a
baseline for a later revision; this run separately checks before/after hashes.
"""

from __future__ import annotations

import argparse
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import re
import shutil
import subprocess
import sys

sys.dont_write_bytecode = True
import numpy as np
import pymupdf

```

```

from solve_drying import INITIAL_C, THRESHOLD_C

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "results/q4/validation_moving_mesh"
PAPER = ROOT / "paper/cumcm-1.1.0"

def require(condition, message):
    if not condition:
        raise ValueError(message)

def sha256(path):
    with path.open("rb") as stream:
        return hashlib.file_digest(stream, "sha256").hexdigest()

def load_json(path):
    return json.loads(path.read_text(encoding="utf8"))

def git_output(*args):
    try:
        return subprocess.check_output(
            ["git", "-C", str(ROOT), *args], stderr=subprocess.DEVNULL,
            text=True, encoding="utf8").strip() or None
    except (OSError, subprocess.CalledProcessError):
        return None

def snapshot(report_path):
    """Protect calculation code, supplied inputs and all saved numerical results."""
    files = set((ROOT / "code").glob("*.py"))
    files.update((ROOT / "paper").rglob("*.py"))
    files.update(path for path in (ROOT / "附件").rglob("*") if path.is_file())
    files.update(ROOT.rglob("*.xlsx"))
    for pattern in ("*.npz", "*.json", "*.csv"):
        files.update((ROOT / "results").rglob(pattern))
    return {path.relative_to(ROOT).as_posix(): sha256(path)
            for path in sorted(files) if path.resolve() != report_path}

def check_arrays(arrays, summary, reference, official):
    """Check dimensions against coordinates and official Q4 data, not old lengths."""
    for name, values in arrays.items():
        require(np.isfinite(values).all(), f"Nonfinite validation array: {name}")
    for name, values in reference.items():

```

```

    require(np.isfinite(values).all(), f"Nonfinite official Q4 array: {name}")
times, xi, trace = arrays["times_s"], arrays["xi"], arrays["step_trace"]
require(times.ndim == xi.ndim == 1 and len(times) > 1 and len(xi) > 1,
        "Time and node coordinates must be nonempty vectors")
require(np.array_equal(times, reference["times_s"]), "Q4 output times differ")
require(np.all(np.diff(times) > 0) and times[0] == 0, "Invalid output time axis")
sample_dt = float(times[1] - times[0])
require(np.allclose(np.diff(times), sample_dt, rtol=0, atol=1e-8),
        "Irregular field output sampling")
require(np.all(np.diff(xi) > 0) and xi[0] == 0 and xi[-1] == 1,
        "Invalid normalized radial grid")
require(len(xi) == summary["nodes"] == official["numerics"]["nodes"],
        "Node coordinates disagree with solver metadata")
require(len(reference["xi"]) == official["numerics"]["output_nodes"],
        "Official Q4 output-node metadata differs")
require(times[-1] == official["output_end_s"], "Q4 output end time differs")
require(arrays["radii_m"].shape == times.shape and
        np.array_equal(arrays["radii_m"], reference["radii_m"]) and
        np.all(arrays["radii_m"] > 0), "Radius history differs")
shape = (len(times), len(xi))
for name in ("mapping_moisture", "moving_moisture",
            "mapping_temperature", "moving_temperature"):
    require(arrays[name].shape == shape, f"Wrong field shape: {name}")
for name in ("mapping_moisture", "moving_moisture"):
    values = arrays[name]
    require(values.min() >= -1e-10 and values.max() <= INITIAL_C + 1e-8,
            f"Moisture outside physical bounds: {name}")
    require(np.max(np.diff(values, axis=1)) <= 2e-9,
            f"Nonmonotone radial moisture profile: {name}")
require(summary["monotone_profiles"], "Full-step monotonicity check failed")
require(trace.shape == (summary["time_steps"] + 1, 8), "Trace metadata differs")
dt = float(summary["dt_s"])
require(dt > 0 and dt == official["numerics"]["dt_s"], "Q4 time steps differ")
require(trace[0, 0] == 0 and trace[-1, 0] == times[-1], "Trace endpoints differ")
increments = np.diff(trace[:, 0])
require(np.all(increments > 0) and np.all(increments <= dt + 1e-8),
        "Missing, duplicate, or oversized integration steps")
short = np.flatnonzero(increments < dt - 1e-8) + 1
require(np.all(np.isclose(np remainder(trace[short, 0], sample_dt), 0, atol=1e-8)),
        "Short steps must end at field-output times")
indices = np.searchsorted(trace[:, 0], times)
require(np.array_equal(trace[indices, 0], times), "Field times missing from trace")
for name, node, column in (("mapping_moisture", 0, 1), ("moving_moisture", 0, 2),
                          ("mapping_moisture", -1, 3), ("moving_moisture", -1, 4)):
    require(np.allclose(arrays[name][:, node], trace[indices, column], rtol=0, atol=1e-12),
            f"Field and step trace disagree: {name}, node {node}")

```

```

require(np.all(trace[:, 5:] >= 0), "Negative absolute errors in step trace")
require(np.allclose(abs(trace[:, 1] - trace[:, 2]), trace[:, 7], rtol=0, atol=1e-14),
        "Center-error column is inconsistent")
for field, column, tolerance in (("moisture", 5, 1e-14), ("temperature", 6, 1e-12)):
    errors = np.max(abs(arrays[f"mapping_{field}"] - arrays[f"moving_{field}"]), axis
=1)
    require(np.allclose(errors, trace[indices, column], rtol=0, atol=tolerance),
            f"Full-field {field} errors disagree with trace")
error_limits = {"max_abs_moisture_field_difference": (5, 1e-9),
                "max_abs_temperature_field_difference": (6, 1e-7),
                "max_abs_center_moisture_difference": (7, 1e-9)}
for name, (column, limit) in error_limits.items():
    value = float(trace[:, column].max())
    require(summary[name] == value and value < limit, f"Error check failed: {name}")
for name, column in (("mapping", 1), ("moving_mesh", 2)):
    event = summary["events"][name]
    moisture = arrays[f"{name}_event_moisture"]
    temperature = arrays[f"{name}_event_temperature"]
    require(moisture.shape == temperature.shape == xi.shape, "Wrong event field shape")
    require(abs(moisture.max() - THRESHOLD_C) < 1e-10, "Event misses drying threshold")
    crossings = np.flatnonzero(trace[:, column] <= THRESHOLD_C)
    require(len(crossings) > 0 and crossings[0] > 0, "Threshold crossing missing")
    i = crossings[0]
    fraction = ((trace[i-1, column] - THRESHOLD_C) /
                (trace[i-1, column] - trace[i, column]))
    crossing_time = trace[i-1, 0] + fraction * (trace[i, 0] - trace[i-1, 0])
    require(abs(crossing_time - event["time_s"]) < 1e-6, "Event time differs from trace
")
    require(abs(event["time_s"] / 3600 - event["time_h"]) < 1e-12, "Event time units
differ")
delta = abs(summary["events"]["mapping"]["time_s"] -
            summary["events"]["moving_mesh"]["time_s"])
require(delta == summary["drying_time_abs_difference_s"] and delta < 1e-5,
        "Independent drying-time error exceeds tolerance")
require(np.isclose(summary["drying_time_relative_difference"],
                    delta / summary["events"]["mapping"]["time_s"], rtol=1e-12, atol=1e
-18),
        "Relative drying-time error is inconsistent")
reproduction = {}
for field, key, limit in (("moisture", "moisture", 1e-12),
                          ("temperature", "temperature_c", 1e-10)):
    expected = np.array([np.interp(reference["xi"], xi, row)
                        for row in arrays[f"mapping_{field}"]])
    require(reference[key].shape == expected.shape, f"Official {field} field shape
differs")
    difference = float(np.max(abs(expected - reference[key])))
    require(difference < limit, f"Saved validation no longer reproduces official {field}

```

```

    })
    reproduction[f"max_{field}_difference"] = difference
    time_difference = abs(summary["events"]["mapping"]["time_s"] - official["drying_time_s"
    ])
    require(time_difference < 1e-6, "Official drying time differs")
    reproduction["drying_time_difference_s"] = time_difference
    return dict(field_shape=list(shape), step_trace_shape=list(trace.shape),
                output_end_s=float(times[-1]), field_sampling_s=sample_dt, dt_s=dt,
                error_limits={name: limit for name, (_, limit) in error_limits.items()},
                reference_reproduction=reproduction)

def check_paper(log_path, skip_build_check):
    pdf = PAPER / "main.pdf"
    with pymupdf.open(pdf) as doc:
        result = dict(pdf_pages=len(doc), paper_sha256=sha256(pdf), validation_pages=[
            i + 1 for i, page in enumerate(doc) if any(term in page.get_text() for term in
                ("移动网格独立验证", "移动网格与固定域映射的数值比较", "两种坐标实现的中心"))])
    if skip_build_check:
        result["build_check"] = {"status": "not_verified", "reason": "Explicit --skip-build
        -check"}
        return result
    if log_path is None:
        candidates = [PAPER / "build/main.log", PAPER / "main.log"]
        available = [path for path in candidates if path.is_file()]
        require(available, "No build log: supply --log, or explicitly use --skip-build-
        check")
        log_path = max(available, key=lambda path: path.stat().st_mtime)
        require(log_path.is_file(), f"Build log missing: {log_path}")
        log = log_path.read_text(encoding="utf8", errors="replace")
        bad = [item for item in ("Overfull", "undefined references", "Missing character:",
                                "Float too large", "! LaTeX Error", "Emergency stop") if item
        in log]
        require(not bad, f"LaTeX warnings/errors require review: {bad}")
        completed = re.findall(r"Output written on .*\((\d+) pages?", log, flags=re.DOTALL)
        require(completed and int(completed[-1]) == result["pdf_pages"],
                "Log has no successful TeX output with the current PDF page count")
        require(log_path.stat().st_mtime >= pdf.stat().st_mtime - 120,
                "Build log predates current PDF; supply the current compilation log")
        result["build_check"] = dict(status="passed", log_path=str(log_path.resolve()),
                                    log_sha256=sha256(log_path), warnings_or_errors=bad)
    return result

def historical_provenance():
    path = OUT / "protected_files_sha256.json"
    if not path.is_file():

```

```

        return {"role": "historical_only", "available": False}
manifest = load_json(path)
changed = []
for name, old_hash in manifest.items():
    current = ROOT / name.replace("\\", "/")
    if not current.is_file() or sha256(current) != old_hash:
        changed.append(name)
return dict(role="historical_only_not_current_acceptance_baseline",
            path=path.relative_to(ROOT).as_posix(), sha256=sha256(path),
            last_recorded_commit=git_output("log", "-1", "--format=%H", "--", str(path)
),
            different_from_historical=changed)

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--log", type=Path, help="Current TeX log (default: newest paper/
build/main.log or paper/main.log)")
    parser.add_argument("--report", type=Path, default=OUT / "delivery_checks.json")
    parser.add_argument("--skip-build-check", action="store_true", help="Check data/PDF
only; report will explicitly be partial")
    parser.add_argument("--package", action="store_true", help="Also copy verified main.pdf
to A题_完整论文.pdf")
    args = parser.parse_args()
    require(not (args.package and args.skip_build_check), "Packaging requires build-log
verification")
    report_path = args.report.resolve()
    require(report_path.suffix.lower() == ".json", "Report must be a JSON file")
    before = snapshot(report_path)
    # A report may replace an earlier delivery report, but never a numerical result or
    historical manifest.
    if report_path.exists():
        previous = load_json(report_path)
        require("protected_files_unchanged" in previous or previous.get("check_kind") == "
q4_delivery",
                "Refusing to overwrite a file that is not a delivery-check report")
    with np.load(OUT / "fields.npz", allow_pickle=False) as saved, \
        np.load(ROOT / "results/q4/fields.npz", allow_pickle=False) as formal:
        arrays = {name: saved[name] for name in saved.files}
        reference = {name: formal[name] for name in formal.files}
    result = dict(check_kind="q4_delivery", schema_version=2,
                  checked_at_utc=datetime.now(timezone.utc).isoformat(), git_head=
git_output("rev-parse", "HEAD"),
                  numerical_checks=check_arrays(arrays, load_json(OUT / "summary.json"),
reference,
                  load_json(ROOT / "results/q4/summary.json")
),

```

```

        historical_manifest=historical_provenance(),
        **check_paper(args.log, args.skip_build_check))
if args.package:
    destination = ROOT / "A题_完整论文.pdf"
    shutil.copy2(PAPER / "main.pdf", destination)
    result["packaged_pdf"] = dict(path=str(destination), sha256=sha256(destination))
after = snapshot(report_path)
require(before == after, "Protected calculation/input/result files changed during this
check")
result["protection"] = dict(unchanged=True, file_count=len(before), before_sha256=
before, after_sha256=after)
result["status"] = "partial_build_not_verified" if args.skip_build_check else "passed"
report_path.parent.mkdir(parents=True, exist_ok=True)
report_path.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n",
encoding="utf8")
print(json.dumps({key: result[key] for key in ("status", "numerical_checks", "pdf_pages",
"build_check")},
ensure_ascii=False, indent=2))
print(f"Report: {report_path}")

if __name__ == "__main__":
    try:
        main()
    except (ValueError, OSError, KeyError) as error:
        print(f"Delivery check failed: {error}", file=sys.stderr)
        sys.exit(1)

```

2.11 figure_style.py

```

# File: code/figure_style.py
"""Shared, reproducible typography for all manuscript data figures."""
from pathlib import Path
import logging
import warnings

import matplotlib as mpl
from matplotlib import font_manager

def configure_fonts() -> None:
    """Require the actual regular fonts, rather than a platform-dependent fallback."""
    for family in ("Times New Roman", "SimSun"):
        path = font_manager.findfont(
            font_manager.FontProperties(family=family, weight="normal", style="normal"),

```

```

        fallback_to_default=False,
    )
    resolved = font_manager.get_font(path)
    if resolved.family_name != family or resolved.style_name != "Regular":
        raise RuntimeError(
            f"Expected {family} Regular; resolved {resolved.family_name} "
            f"{resolved.style_name} at {path}"
        )
mpl.rcParams.update({
    "font.family": ["Times New Roman", "SimSun"],
    "font.weight": "normal",
    "font.size": 11,
    "axes.labelsize": 11,
    "axes.labelweight": "normal",
    "axes.titlesize": 11,
    "axes.titleweight": "normal",
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
    "legend.title_fontsize": 10,
    "figure.titlesize": 12,
    "figure.titleweight": "normal",
    "mathtext.fontset": "custom",
    "mathtext.rm": "Times New Roman",
    "mathtext.it": "Times New Roman:italic",
    "mathtext.bf": "Times New Roman:bold",
    "mathtext.sf": "Times New Roman",
    "mathtext.fallback": "stix",
    "axes.unicode_minus": True,
    "pdf.fonttype": 42,
    "svg.fonttype": "none",
    "savefig.dpi": 300,
    "savefig.bbox": "tight",
})

def save_figure(fig, filename) -> None:
    """Export embedded-font PDF and PNG; never silently accept missing glyphs."""
    filename = Path(filename)
    filename.parent.mkdir(parents=True, exist_ok=True)
    from matplotlib.text import Text

    # Non-math Chinese runs need an explicit face; $...$ keeps Times math.
    for text in fig.findobj(Text):
        value = text.get_text()
        if "$" in value and any("\u3400" <= ch <= "\u9fff" for ch in value):
            text.set_fontfamily("SimSun")

```

```

class MissingGlyphError(logging.Handler):
    def emit(self, record):
        message = record.getMessage()
        if "does not have a glyph" in message or "dummy symbol" in message:
            raise RuntimeError(message)

logger = logging.getLogger("matplotlib")
guard = MissingGlyphError()
logger.addHandler(guard)
try:
    with warnings.catch_warnings():
        warnings.filterwarnings("error", message=r"Glyph .* missing from font")
        fig.canvas.draw()
        for suffix in ("pdf", "png"):
            fig.savefig(f"{filename}.{suffix}", bbox_inches="tight", dpi=300)
finally:
    logger.removeHandler(guard)

```

2.12 make_figures.py

```

# File: code/make_figures.py
# Academic Figure Skill Asset Confirmation (verified against assets/figures/)
# (a) LineTrend → assets/figures/LineTrend/plot_sweep.py → param inherit
# (b) LineTrend → assets/figures/LineTrend/plot_sweep.py → param inherit
# RULE: "native run" = load pre-rendered PNG via Image.open().ax.imshow().
#       "param inherit" = drawing function below that copies Class A/B/C values.
#       The production asset's data semantics differ, so only its clean line/marker
#       structure is inherited.

from figure_style import configure_fonts, save_figure

configure_fonts()

# Academic Figure Skill Nature/Cell/Science Color Palette -- COPY VERBATIM
CATEGORICAL = ["#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666"]
CATEGORICAL_EXTENDED = [
    "#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666",
    "#4393C3", "#D6604D", "#5AAE61", "#B35806", "#9970AB", "#999999",
]
DIVERGING = ["#2166AC", "#F7F7F7", "#B2182B"]
SEQUENTIAL = ["#F7FBFF", "#6BAED6", "#08306B"]
ACCENT_RED = "#B2182B"
GREY = "#999999"
BLACK = "#222222"

```

```

import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.ticker import MaxNLocator, StrMethodFormatter

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / "results"
FIGURES = ROOT / "figures"
MM_TO_INCH = 1.0 / 25.4
TIME_COLORS = ["#BFD7EA", "#6BAED6", "#2166AC", "#762A83", "#B2182B"]

def load_summary(question: str) -> dict:
    return json.loads((RESULTS / question / "summary.json").read_text(encoding="utf-8"))

def nearest_indices(times: np.ndarray, requested: list[float]) -> list[int]:
    return [int(np.argmin(np.abs(times - value))) for value in requested]

def panel_labels(fig, axes: list[plt.Axes], labels: str = "ab") -> None:
    for label, axis in zip(labels, axes):
        box = axis.get_position()
        fig.text(
            0.5 * (box.x0 + box.x1),
            0.025,
            f"({label})",
            ha="center",
            va="bottom",
            fontsize=11,
            fontweight="bold",
        )

def common_axis_style(axis: plt.Axes) -> None:
    axis.tick_params(length=3.0, width=0.6, labelsz=10)
    axis.xaxis.label.set_size(11)
    axis.yaxis.label.set_size(11)
    axis.xaxis.set_major_locator(MaxNLocator(nbins=5))
    axis.yaxis.set_major_locator(MaxNLocator(nbins=6, steps=[1, 2, 2.5, 5, 10]))
    axis.spines["top"].set_visible(False)
    axis.spines["right"].set_visible(False)

```

```

axis.spines["left"].set_linewidth(0.6)
axis.spines["bottom"].set_linewidth(0.6)
axis.grid(False)

def plot_profiles(
    axis: plt.Axes,
    radius_cm: np.ndarray,
    values: np.ndarray,
    labels: list[str],
    ylabel: str,
    show_legend: bool = True,
) -> None:
    for index, (profile, label) in enumerate(zip(values, labels)):
        color = TIME_COLORS[index if len(values) == len(TIME_COLORS) else int(
            round(index * (len(TIME_COLORS) - 1) / max(len(values) - 1, 1))
        )]
        axis.plot(
            radius_cm,
            profile,
            color=color,
            linewidth=1.3,
            marker="o",
            markersize=2.3,
            markevery=4,
            label=label,
        )
    axis.set_xlabel("到药材中心的距离 / cm")
    axis.set_ylabel(ylabel, fontfamily="SimSun" if "$" in ylabel else None)
    axis.set_xlim(float(radius_cm[0]), float(radius_cm[-1]))
    if show_legend:
        axis.legend(
            loc="lower center",
            bbox_to_anchor=(0.5, 1.01),
            ncol=min(len(labels), 3),
            columnspacing=0.9,
            handlelength=1.8,
            fontsize=10,
        )
    common_axis_style(axis)
    axis.set_xticks(np.linspace(float(radius_cm[0]), float(radius_cm[-1]), 5))
    axis.xaxis.set_major_formatter(StrMethodFormatter("{x:g}"))

def figure_q1() -> None:
    data = np.load(RESULTS / "q1" / "fields.npz")
    times = data["times_s"]

```

```

radius_cm = data["radius_m"] * 100.0
requested = [100, 600, 1200, 1800]
indexes = nearest_indices(times, requested)
labels = [f"{time:d} s" for time in requested]
fig, axes = plt.subplots(
    1,
    2,
    figsize=(183 * MM_TO_INCH, 82 * MM_TO_INCH),
    gridspec_kw={"wspace": 0.22},
)
plot_profiles(
    axes[0],
    radius_cm,
    data["temperature_c"][indexes],
    labels,
    "温度 / °C",
    show_legend=False,
)
plot_profiles(
    axes[1],
    radius_cm,
    data["moisture"][indexes],
    labels,
    r"水分浓度  $\text{kg/kg}$ ",
    show_legend=False,
)
axes[1].set_ylim(1.4, 2.60)
handles, legend_labels = axes[0].get_legend_handles_labels()
fig.legend(
    handles,
    legend_labels,
    loc="upper center",
    bbox_to_anchor=(0.5, 0.975),
    ncol=len(legend_labels),
    columnspacing=1.2,
    handlelength=1.8,
    handletextpad=0.45,
    borderaxespad=0.0,
    fontsize=10,
    frameon=False,
)
fig.subplots_adjust(left=0.10, right=0.98, top=0.84, bottom=0.24, wspace=0.34)
panel_labels(fig, list(axes))
save_figure(fig, str(FIGURES / "q1_preheat_profiles"))
plt.close(fig)

```

```

def figure_q2() -> None:
    data = np.load(RESULTS / "q2" / "fields.npz")
    times = data["times_s"]
    radius_cm = data["radius_m"] * 100.0
    requested = [1800, 5400, 10800]
    indexes = nearest_indices(times, requested)
    labels = ["0.5 h", "1.5 h", "3.0 h"]
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(183 * MM_TO_INCH, 82 * MM_TO_INCH),
        gridspec_kw={"wspace": 0.22},
    )
    plot_profiles(
        axes[0],
        radius_cm,
        data["temperature_c"][indexes],
        labels,
        "温度 / °C",
        show_legend=False,
    )
    plot_profiles(
        axes[1],
        radius_cm,
        data["moisture"][indexes],
        labels,
        r"水分浓度  $\text{kg/kg}$ ",
        show_legend=False,
    )
    handles, legend_labels = axes[0].get_legend_handles_labels()
    fig.legend(
        handles,
        legend_labels,
        loc="upper center",
        bbox_to_anchor=(0.5, 0.975),
        ncol=len(legend_labels),
        columnspacing=1.2,
        handlelength=1.8,
        handletextpad=0.45,
        borderaxespad=0.0,
        fontsize=10,
        frameon=False,
    )
    fig.subplots_adjust(left=0.10, right=0.98, top=0.84, bottom=0.24, wspace=0.34)
    panel_labels(fig, list(axes))
    save_figure(fig, str(FIGURES / "q2_coupled_profiles"))
    plt.close(fig)

```

```

def figure_q3() -> None:
    data = np.load(RESULTS / "q3" / "fields.npz")
    summary = load_summary("q3")
    time_h = data["times_s"] / 3600.0
    radius_cm = data["radius_m"] * 100.0
    end_h = float(summary["drying_time_h"])
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(183 * MM_TO_INCH, 86 * MM_TO_INCH),
        gridspec_kw={"wspace": 0.28},
    )
    axes[0].plot(time_h, data["moisture"][:, 0], color=CATEGORICAL[0], linewidth=1.4, label="中心")
    axes[0].plot(
        time_h,
        data["moisture"][:, -1],
        color=CATEGORICAL[3],
        linewidth=1.2,
        linestyle="--",
        label="表面",
    )
    axes[0].axhline(0.15, color=ACCENT_RED, linewidth=0.8, linestyle=":", label="阈值 0.15")
    axes[0].axvline(end_h, color=GREY, linewidth=0.7, linestyle="--")
    axes[0].annotate(
        f"{end_h:.3f} h",
        xy=(end_h, 0.15),
        xytext=(end_h - 16, 0.42),
        arrowprops={"arrowstyle": "->", "lw": 0.7, "color": BLACK},
        fontsize=10,
    )
    axes[0].set_xlabel("时间 / h")
    axes[0].set_ylabel(r"水分浓度  $\text{kg/kg}$ ", fontfamily="SimSun")
    axes[0].set_xlim(0, max(time_h))
    axes[0].set_ylim(0, 2.65)
    axes[0].legend(loc="upper right", frameon=False)
    common_axis_style(axes[0])

    requested_h = [6, 18, 36, 48, end_h]
    indexes = nearest_indices(time_h, requested_h)
    labels = ["6 h", "18 h", "36 h", "48 h", f"{end_h:.3f} h"]
    profiles = data["moisture"][indexes].copy()
    profiles[-1] = data["threshold_moisture"]
    plot_profiles(
        axes[1],

```

```

        radius_cm,
        profiles,
        labels,
        r"水分浓度  $\text{C}\backslash,\backslash,\mathrm{(kg/kg)}$ ",
        show_legend=False,
    )
    axes[1].axhline(0.15, color=ACCENT_RED, linewidth=0.8, linestyle=":")
    axes[1].set_ylim(0, 1.10)
    handles, legend_labels = axes[1].get_legend_handles_labels()
    fig.legend(
        handles,
        legend_labels,
        loc="upper center",
        bbox_to_anchor=(0.5, 0.975),
        ncol=len(legend_labels),
        columnspacing=1.0,
        handlelength=1.8,
        handletextpad=0.45,
        borderaxespad=0.0,
        fontsize=10,
        frameon=False,
    )
    fig.subplots_adjust(left=0.10, right=0.98, top=0.84, bottom=0.24, wspace=0.34)
    panel_labels(fig, list(axes))
    save_figure(fig, str(FIGURES / "q3_threshold_diagnostics"))
    plt.close(fig)

```

def figure_q4() -> None:

```

    fixed = np.load(RESULTS / "q4" / "fixed_radius_counterfactual.npz")
    q4 = np.load(RESULTS / "q4" / "fields.npz")
    summary4 = load_summary("q4")
    time_fixed_h = fixed["times_s"] / 3600.0
    time4_h = q4["times_s"] / 3600.0
    end_fixed_h = float(
        summary4["fixed_radius_counterfactual"]["same_appendix4_properties_time_h"]
    )
    end4_h = float(summary4["drying_time_h"])
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(183 * MM_TO_INCH, 86 * MM_TO_INCH),
        gridspec_kw={"width_ratios": [1.35, 1.0], "wspace": 0.34},
    )
    axes[0].plot(
        time_fixed_h,
        fixed["moisture"][ :, 0],

```

```

        color=CATEGORICAL[0],
        linewidth=1.3,
        linestyle="--",
        label="固定半径对照（附录 4 物性）",
    )
axes[0].plot(
    time4_h,
    q4["moisture"][:, 0],
    color=ACCENT_RED,
    linewidth=1.5,
    label="收缩模型（问题 4）",
)
axes[0].axhline(0.15, color=GREY, linewidth=0.8, linestyle=":")
axes[0].scatter(
    [end_fixed_h, end4_h],
    [0.15, 0.15],
    color=[CATEGORICAL[0], ACCENT_RED],
    s=18,
    zorder=4,
)
axes[0].annotate(
    f"{end4_h:.3f} h",
    xy=(end4_h, 0.15),
    xytext=(end4_h - 15, 0.43),
    arrowprops={"arrowstyle": "->", "lw": 0.7, "color": ACCENT_RED},
    color=ACCENT_RED,
    fontsize=10,
)
axes[0].set_xlabel("时间 / h")
axes[0].set_ylabel(r"中心水分浓度  $C\backslash, /\backslash, \mathrm{(kg/kg)}$ ", fontfamily="SimSun")
axes[0].set_xlim(0, max(time_fixed_h))
axes[0].set_ylim(0, 2.65)
handles, legend_labels = axes[0].get_legend_handles_labels()
fig.legend(
    handles, legend_labels, loc="upper center", bbox_to_anchor=(0.5, 0.975),
    ncol=2, columnspacing=1.2, handlelength=1.8, fontsize=10,
    borderaxespad=0.0,
    frameon=False,
)
common_axis_style(axes[0])

axes[1].plot(
    time4_h,
    q4["radii_m"] * 100.0,
    color=CATEGORICAL[4],
    linewidth=1.4,
    marker="o",

```

```

        markersize=2.2,
        markevery=max(1, len(time4_h) // 12),
    )
    axes[1].scatter([end4_h], [summary4["radius_at_end_cm"]], color=ACCENT_RED, s=18,
                    zorder=4)
    axes[1].annotate(
        f"{summary4['radius_at_end_cm']:.3f} cm",
        xy=(end4_h, summary4["radius_at_end_cm"]),
        xytext=(end4_h - 17, 1.33),
        arrowprops={"arrowstyle": "->", "lw": 0.7, "color": BLACK},
        fontsize=10,
    )
    axes[1].set_xlabel("时间 / h")
    axes[1].set_ylabel(r"药材半径 $R$, / \, \mathrm{cm}$", fontfamily="SimSun")
    axes[1].set_xlim(0, max(time4_h))
    axes[1].set_ylim(1.12, 2.05)
    common_axis_style(axes[1])
    fig.subplots_adjust(left=0.10, right=0.98, top=0.83, bottom=0.24, wspace=0.38)
    panel_labels(fig, list(axes))
    save_figure(fig, str(FIGURES / "q4_shrinkage_comparison"))
    plt.close(fig)

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    FIGURES.mkdir(parents=True, exist_ok=True)
    figure_q1()
    figure_q2()
    figure_q3()
    figure_q4()
    print("Generated q1--q4 PDF and PNG figures.")

if __name__ == "__main__":
    main()

```

2.13 make_additional_figures.py

```

# File: code/make_additional_figures.py
# Academic Figure Skill Asset Confirmation (verified against assets/figures/)
# q1/q2 surfaces -> no continuous PDE surface asset -> cross-type inherit
# q2 map -> heatmap/plot_composition.py -> param inherit (continuous field, not counts)
# q3 threshold region -> no semantic match -> cross-type inherit
# q4 field / disks -> heatmap shared quantitative scale -> param inherit
# Shared CUMCM typography is applied after the common plot settings below.

```

```

import matplotlib as mpl
mpl.rcParams.update({
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.linewidth": 0.6,
    "xtick.direction": "out",
    "ytick.direction": "out",
    "xtick.major.width": 0.6,
    "ytick.major.width": 0.6,
    "legend.frameon": False,
})

# Academic Figure Skill Nature/Cell/Science Color Palette -- COPY VERBATIM
CATEGORICAL = ["#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666"]
CATEGORICAL_EXTENDED = [
    "#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666",
    "#4393C3", "#D6604D", "#5AAE61", "#B35806", "#9970AB", "#999999",
]
DIVERGING = ["#2166AC", "#F7F7F7", "#B2182B"]
SEQUENTIAL = ["#F7FBFF", "#6BAED6", "#08306B"]
ACCENT_RED = "#B2182B"
GREY = "#999999"
BLACK = "#222222"

import json
import shutil
import sys
from pathlib import Path
import matplotlib.patheffects as pe
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, Normalize
from matplotlib.patches import Circle
sys.dont_write_bytecode = True
from solve_drying import property_q23
from figure_style import configure_fonts, save_figure

configure_fonts()

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / 'figures'
PAPER = ROOT / 'paper/cumcm-1.1.0/figures'
REPORT = ROOT / 'reports/figure_additions_20260911'
BLUE = LinearSegmentedColormap.from_list('moisture', SEQUENTIAL)
NORM_C = Normalize(0, 2.55)
METRICS = {}

```

```

def read(q):
    with np.load(ROOT / 'results' / q / 'fields.npz') as z:
        d = {k:z[k] for k in z.files}
    for key in ['temperature_c','moisture']:
        assert np.isfinite(d[key]).all()
        assert d[key].shape == (len(d['times_s']), len(d['xi'] if q == 'q4' else d['radius_m']))
    assert np.all(np.diff(d['times_s']) > 0)
    assert d['moisture'].min() >= 0
    assert np.max(np.diff(d['moisture'],axis=1)) < 2e-8
    return d

def summary(q):
    return json.loads((ROOT/'results'/q/'summary.json').read_text(encoding='utf8'))

def event_data(q):
    d=read(q); s=summary(q); end=s['drying_time_s']
    keep=d['times_s'] < end
    t=np.r_[d['times_s'][keep],end]/3600
    c=np.vstack([d['moisture'][keep],d['threshold_moisture']])
    assert abs(c[-1].max()-15)<1e-9
    return d,s,t,c,keep

def save(fig,name):
    save_figure(fig,str(OUT/name))
    shutil.copy2(OUT/f'{name}.pdf',PAPER/f'{name}.pdf')
    plt.close(fig)

def surface(q,key,name,zlabel,cmap,limit,ticks):
    d=read(q)
    mask=d['times_s']<=limit
    t=d['times_s'][mask]/(60 if q=='q1' else 3600)
    r=d['radius_m']*100
    v=d[key][mask]
    x,y=np.meshgrid(t,r,indexing='ij')
    fig=plt.figure(figsize=(183/25.4,108/25.4))
    ax=fig.add_axes([.04,.10,.76,.88],projection='3d')
    norm=Normalize(float(v.min()),float(v.max()))
    ax.plot_surface(x,y,v,cmap=cmap,norm=norm,rstride=1,cstride=1,
                    linewidth=0,antialiased=False,shade=False,rasterized=True)
    # Outline the two physical boundaries; all surface cells remain represented.
    ax.plot(t,np.zeros_like(t),v[:,0],color='#222222',lw=.9)
    ax.plot(t,np.full_like(t,2),v[:,-1],color='#222222',lw=.9,ls='--')
    ax.set(xlim=(t[0],t[-1]),ylim=(0,2),zlim=(ticks[0],ticks[-1]))
    ax.set_xticks([0,10,20,30] if q=='q1' else [0,1,2,3])
    ax.set_yticks([0,1,2]); ax.set_zticks(ticks)
    ax.set_xlabel('时间 / '+'(min' if q=='q1' else 'h'),labelpad=7)

```

```

ax.set_ylabel(r'半径 $r\backslash,\backslash\mathrm{cm}$',labelpad=6)
ax.set_zlabel('')
fig.text(.10,.62,zlabel,rotation=90,va='center',ha='center',fontsize=11)
ax.view_init(elev=30,azim=45 if q=='q2' else -135)
ax.set_box_aspect((1.7,1,1.05))
for axis in [ax.xaxis,ax.yaxis,ax.zaxis]:
    axis.pane.fill=False
    axis.pane.set_edgecolor('#DDDDDD')
    axis._axinfo['grid'].update(color='#E3E3E3',linewidth=.35)
cb=fig.colorbar(mpl.cm.ScalarMappable(norm=norm,cmap=cmap),cax=fig.add_axes
([.82,.28,.022,.48]))
cb.set_label(zlabel,labelpad=8)
fig.text(.44,.03,'实线: 中心边界    虚线: 表面边界',ha='center',fontsize=10)
METRICS[name]={'shape':list(v.shape),'min':float(v.min()),'max':float(v.max()),'
time_range_s':[0,limit]}
save(fig,name)

def label_contours_inside(ax, contours):
    """Keep numeric labels inside the plotted domain after enlarging the type."""
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    points = []
    for segments in contours.allsegs:
        paths = [segment for segment in segments if len(segment) > 1]
        if not paths:
            continue
        vertices = np.concatenate(paths)
        normalized = (vertices - [xmin, ymin]) / [xmax - xmin, ymax - ymin]
        inside = ((normalized[:, 0] > .07) & (normalized[:, 0] < .96)
                  & (normalized[:, 1] > .10) & (normalized[:, 1] < .88))
        if not np.any(inside):
            continue
        candidates = vertices[inside]
        score = np.sum((normalized[inside] - [.55, .60]) ** 2, axis=1)
        points.append(tuple(candidates[np.argmin(score)]))
    return ax.clabel(contours, fmt='%g', fontsize=10, inline=True, manual=points)

def surface_heatmap(q,key,name,zlabel,cmap,limit,ticks):
    d=read(q)
    mask=d['times_s']<=limit
    t=d['times_s'][mask]/(60 if q=='q1' else 3600)
    r=d['radius_m']*100
    v=d[key][mask]
    fig,ax=plt.subplots(figsize=(183/25.4,82/25.4))
    fig.subplots_adjust(left=.10,right=.86,bottom=.20,top=.93)
    norm=Normalize(float(v.min()),float(v.max()))

```

```

im=ax.pcolormesh(t,r,v.T,cmap=cmap,norm=norm,shading='auto',rasterized=True)
cs=ax.contour(t,r,v.T,levels=ticks,colors='white',linewidths=.75)
labels=label_contours_inside(ax,cs)
for label in labels:
    label.set_color(BLACK)
    label.set_path_effects([pe.withStroke(linewidth=.8,foreground='white')])
ax.plot(t,np.zeros_like(t),color='#222222',lw=.85)
ax.plot(t,np.full_like(t,2),color='#222222',lw=.85,ls='--')
ax.set(xlim=(t[0],t[-1]),ylim=(0,2),xlabel='时间 / '+'(min' if q=='q1' else 'h'),ylabel=
r'到药材中心的距离 $r\,,\backslash,\mathrm{cm}$')
ax.set_xticks([0,10,20,30] if q=='q1' else [0,1,2,3])
ax.set_yticks([0,.5,1,1.5,2])
cb=fig.colorbar(im,cax=fig.add_axes([.89,.20,.023,.73]))
cb.set_label(zlabel,labelpad=8)
fig.text(.47,.035,'下边界: 中心    上边界: 表面',ha='center',fontsize=10)
METRICS[name]={ 'shape':list(v.shape), 'min':float(v.min()), 'max':float(v.max()), '
time_range_s':[0,limit], 'plot':'2D heatmap with contour lines'}
save(fig,name)

def diffusivity():
    d=read('q2'); m=d['times_s']<=10800
    t=d['times_s'][m]/3600; r=d['radius_m']*100
    z=property_q23(d['temperature_c'][m],d['moisture'][m])[-1]*1e9
    fig,ax=plt.subplots(figsize=(183/25.4,78/25.4))
    fig.subplots_adjust(left=.10,right=.86,bottom=.20,top=.94)
    im=ax.pcolormesh(t,r,z.T,cmap='viridis',shading='auto',rasterized=True)
    cs=ax.contour(t,r,z.T,levels=[6,8,10,12],colors='white',linewidths=.8)
    labels=label_contours_inside(ax,cs)
    import matplotlib.path_effects as pe
    for label in labels:
        label.set_color(BLACK)
        label.set_path_effects([pe.withStroke(linewidth=.8,foreground='white')])
    ax.set(xlim=(0,3),ylim=(0,2),xlabel='时间 / h',ylabel=r'到药材中心的距离 $r\,,\backslash,\mathrm{cm}$')
    ax.plot(t,np.zeros_like(t),color='#222222',lw=.8)
    ax.plot(t,np.full_like(t,2),color='#222222',lw=.8,ls='--')
    ax.set_xticks([0,1,2,3])
    ax.set_yticks([0,.5,1,1.5,2])
    cb=fig.colorbar(im,cax=fig.add_axes([.89,.20,.023,.74]))
    cb.set_label(r'扩散系数 $D\,,\backslash,(10^{-9}\,,\backslash,\mathrm{m}^2/\mathrm{s})$',labelpad=8)
    METRICS['q2_diffusivity_map']={ 'shape':list(z.shape), 'min_1e9':float(z.min()), 'max_1e9'
:float(z.max()), 'center_end_1e9':float(z[-1,0]), 'surface_end_1e9':float(z[-1,-1])}
    save(fig,'q2_diffusivity_map')

def threshold_front():
    d,s,t,c,keep=event_data('q3'); r=d['radius_m']*100
    front=np.empty(len(t))

```

```

for i,row in enumerate(c):
    if row[-1]>.15: front[i]=2
    elif row[0]<=.15+1e-12: front[i]=0
    else: front[i]=np.interp(.15,row[:::-1],r[:::-1])
fig,ax=plt.subplots(figsize=(183/25.4,85/25.4))
fig.subplots_adjust(left=.10,right=.97,bottom=.19,top=.90)
ax.fill_between(t,0,front,facecolor='#BFD7EA',edgecolor='none')
ax.fill_between(t,front,2,facecolor='#F5F5F5',edgecolor='#D2D2D2',hatch='///',linewidth=0)
start=np.argmax(c[:, -1]<=.15)
ts=np.interp(.15,c[start-1:start+1,-1][::-1],t[start-1:start+1][::-1])
ax.plot(t[start:],front[start:],color=ACCENT_RED,lw=1.8)
ax.scatter([ts,t[-1]],[2,0],s=24,color=ACCENT_RED,zorder=6,clip_on=False)
ax.text(14,.42,'未达标区域\n'+r'$C > 0.15\ \mathrm{kg/kg}$',ha='center',fontsize=10,
color='#17466B')
ax.text(43,1.55,'已达标区域\n'+r'$C < 0.15\ \mathrm{kg/kg}$',ha='center',fontsize=10,
bbox=dict(facecolor='white',edgecolor='none',alpha=.92,pad=3))
ax.annotate(f'表面首达 {ts:.3f} h',xy=(ts,2),xytext=(ts+2,2.13),fontsize=10,
arrowprops=dict(arrowstyle='-',color=GREY,lw=.7),annotation_clip=False)
ax.annotate(f'中心首达 {t[-1]:.3f} h',xy=(t[-1],0),xytext=(40,.32),fontsize=10,
arrowprops=dict(arrowstyle='->',color=ACCENT_RED,lw=.8))
ax.set(xlim=(0,t[-1]),ylim=(0,2),xlabel='时间 / h',ylabel=r'半径 $r\backslash,\backslash,\mathrm{cm}$')
event_h = float("drying_time_h")
ticks = [0, 12, 24, 36, 48, event_h]
ax.set_xticks(ticks); ax.set_xticklabels(['0', '12', '24', '36', '48', f'{event_h:.3f}'])
ax.set_yticks([0,.5,1,1.5,2])
METRICS['q3_drying_front']={'surface_crossing_h':float(ts),'center_crossing_h':float(t[-1]),'lag_h':float(t[-1]-ts),'radial_interpolation':'linear between 21 saved output nodes','time_samples':len(t)}
np.savez_compressed(REPORT/'q3_front_data.npz',time_h=t,threshold_radius_cm=front)
save(fig,'q3_drying_front')

def shrinking():
    d,s,t,c,keep=event_data('q4')
    radii=np.r_[d['radii_m']][keep]*100,s['radius_at_end_cm']]
    xi=d['xi']; xx=np.broadcast_to(t[:,None],c.shape); yy=radii[:,None]*xi
    fig=plt.figure(figsize=(183/25.4,139/25.4))
    ax=fig.add_axes([.095,.48,.74,.46])
    im=ax.pcolormesh(xx,yy,c,cmap=BLUE,norm=NORM_C,shading='gouraud',rasterized=True)
    ax.plot(t,radii,color=BLACK,lw=1.2)
    con=ax.contour(xx,yy,c,levels=[.15],colors=ACCENT_RED,linewidths=1.2)
    # Place the label on the middle of the threshold curve, away from the axes.
    label_index=int(np.argmin(abs(t-.60*t[-1])))
    label_radius=float(np.interp(.15,c[label_index,::-1],yy[label_index,::-1]))
    contour_labels = ax.clabel(con,fmt={.15:r'$C = 0.15$'},fontsize=10,inline=True,
manual=[(float(t[label_index]),label_radius)])

```



```

(REPORT/'metrics.json').write_text(json.dumps(METRICS,ensure_ascii=False,indent=2),
encoding='utf8')
print(json.dumps(METRICS,ensure_ascii=False,indent=2))

if __name__=='__main__':
    main()

```

2.14 plot_moving_mesh_validation.py

```

# File: code/plot_moving_mesh_validation.py
"""Render real Q4 validation data and generate the manuscript result table.

Asset confirmation: academic-figure-skill assets/figures/LineTrend/plot_sweep.py
was inspected; inherit open axes and overlaid line encoding, not example data.
Reuse the project's verified typography/palette/export baseline by import.
"""

import sys
sys.dont_write_bytecode = True
import csv
import json
import shutil
from pathlib import Path
import numpy as np
import matplotlib
matplotlib.use('Agg')
from make_additional_figures import CATEGORICAL
from figure_style import configure_fonts, save_figure
configure_fonts()
import matplotlib.pyplot as plt
from validate_moving_mesh import OUT, ROOT, write_comparisons, check_preservation

def render_figures(arrays):
    tr=arrays['step_trace']; hours=tr[:,0]/3600
    fig,axes=plt.subplots(1,2,figsize=(7.2,3.6))
    fig.subplots_adjust(left=.105,right=.98,bottom=.26,top=.96,wspace=.40)
    ax=axes[0]
    ax.plot(hours,tr[:,1],color=CATEGORICAL[0],lw=1.65,label='固定域映射法')
    ax.plot(hours,tr[:,2],color=CATEGORICAL[1],lw=1.0,ls=(0,(5,3)),label='直接移动网格法')
    ax.axhline(.15,color='#666666',ls=':',lw=.8)
    ax.text(5,.20,r'$C=0.15$',fontsize=10,color='#555555')
    ax.set(xlim=(0,52),ylim=(0,2.65),xlabel='时间 / h',ylabel='中心水分浓度 / (kg/kg)')
    ax.set_xticks([0,12,24,36,48]);ax.legend(loc='upper right',fontsize=10)
    ax.text(.5,-.30,'(a) 中心水分轨迹',transform=ax.transAxes,ha='center',fontsize=10)
    ax=axes[1]

```

```

for h,color in [(24,CATEGORICAL[0]),(48,CATEGORICAL[4])]:
    i=int(np.flatnonzero(arrays['times_s']==h*3600)[0])
    r=100*arrays['radii_m'][i]*arrays['xi']
    ax.plot(r,arrays['mapping_moisture'][i],color=color,lw=1.65,label=f'{h} h, 固定域')
    ax.plot(r,arrays['moving_moisture'][i],color=color,lw=.85,ls='--',
            marker='o',mfc='white',mew=.65,ms=2.5,markevery=20,label=f'{h} h, 移动网格')
ax.set(xlim=(0,1.24),ylim=(.04,.30),xlabel=r'物理半径 $r\,,/\,,\mathrm{cm}$',ylabel='水分
浓度 / (kg/kg)')
ax.xaxis.label.set_fontfamily('SimSun')
ax.set_xticks([0,.3,.6,.9,1.2]);ax.legend(loc='lower left',fontsize=10)
ax.text(.5,-.30,'(b) 完整径向剖面',transform=ax.transAxes,ha='center',fontsize=10)
save_figure(fig,OUT/'q4_moving_mesh_validation');plt.close(fig)
shutil.copy2(OUT/'q4_moving_mesh_validation.pdf',ROOT/'paper/cumcm-1.1.0/figures/
q4_moving_mesh_validation.pdf')

fig,axes=plt.subplots(1,2,figsize=(7.2,3.2))
fig.subplots_adjust(left=.12,right=.98,bottom=.23,top=.90,wspace=.52)
for ax,column,label,color in [(axes[0],5,'水分场最大绝对差 / (kg/kg)',CATEGORICAL[0]),
                               (axes[1],6,'温度场最大绝对差 / °C',CATEGORICAL[1])]:
    ax.plot(hours,tr[:,column],lw=.6,color=color,rasterized=True)
    ax.set(xlim=(0,52),ylim=(0,None),xlabel='时间 / h',ylabel=label)
    if '$' in label:
        ax.yaxis.label.set_fontfamily('SimSun')
    ax.set_xticks([0,12,24,36,48]);ax.ticklabel_format(axis='y',style='sci',scilimits
=(0,0))
save_figure(fig,OUT/'q4_moving_mesh_errors');plt.close(fig)

def main():
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--figures-only', action='store_true',
                        help='Render saved fields without rewriting numerical reports or
tables')
    args = parser.parse_args()
    arrays=np.load(OUT/'fields.npz')
    render_figures(arrays)
    if args.figures_only:
        return
    report=json.loads((OUT/'summary.json').read_text(encoding='utf8'))
    report['event_comparison_note']='Own-event profiles and common mapping-event centre/
surface values are distinguished; trajectories compare identical time levels.'
    (OUT/'summary.json').write_text(json.dumps(report,indent=2),encoding='utf8')
    write_comparisons(arrays,report)
    rows=list(csv.DictReader((OUT/'comparison.csv').open(encoding='utf-8-sig'))))
    labels={'drying_time':r'$t_d$/h'}
    for time,caption in [(('6h','6 h'),('24h','24 h'),('48h','48 h'),('common_event',r'$t_d

```

```

^{\rm map}$')]:
    for point,cn in [('center','中心'),('surface','动态表面')]:
        labels[f'{time}_{point}_moisture']=f'{caption}, {cn}'
tex=[r'\begin{table}[H]',r'\centering',r'\caption{移动网格与固定域映射的数值比较}\label{
tab:q4movingvalidation}',
    r'\begin{tabular}{lrrr}',r'\toprule',r'指标 & 固定域映射法 & 移动网格法 & 相对差\\',r
'\midrule']
md='| 指标 | 固定域映射法 | 移动网格法 | 相对差 | \n|---|---:|---:|---:|\n'
for row in rows:
    if row['metric'] not in labels: continue
    a,b,d=map(float,[row['mapping'],row['moving_mesh'],row['relative_difference']])
    mant,exponent=f'{d:.2e}'.split('e')
    diff=rf'${mant}\times 10^{{int(exponent)}}$' if d else '$0$'
    label=labels[row['metric']]
    tex.append(f'{label} & {a:.9f} & {b:.9f} & {diff}'+r'\\\')
    md+=f'| {label} | {a:.12g} | {b:.12g} | {d:.3e} | \n'
tex += [r'\bottomrule',r'\end{tabular}',r'\par\smallskip',
    r'\footnotesize 水分单位为 kg/kg; 相对差以固定域结果为分母。末两行为同一固定域达标时
刻的比较。}',r'\end{table}']
(ROOT/'paper/cumcm-1.1.0/contents/sections/q4_moving_mesh_table.tex').write_text('\n'.
join(tex)+'\n',encoding='utf8')
(OUT/'comparison_table_zh.md').write_text(md,encoding='utf8')
count=check_preservation(json.loads((OUT/'protected_files_sha256.json').read_text(
encoding='utf8'))))
print(f'Figures and table generated; {count} original files unchanged.')

if __name__=='__main__':
    main()

```

2.15 make_geometry_diagram.py

```

# File: code/make_geometry_diagram.py
"""Build an editable DrawIO cylinder / cylindrical-coordinate schematic."""
from pathlib import Path
import xml.etree.ElementTree as ET

ROOT=Path(__file__).resolve().parents[1]
doc=ET.Element('mxfile',host='app.diagrams.net')
diagram=ET.SubElement(doc,'diagram',name='药材几何与径向计算域',id='geometry')
model=ET.SubElement(diagram,'mxGraphModel',dx='1000',dy='720',grid='0',page='0',math='0',
    shadow='0')
tree=ET.SubElement(model,'root')
ET.SubElement(tree,'mxCell',id='0')
ET.SubElement(tree,'mxCell',id='1',parent='0')

```

```

count=1

def ident():
    global count
    count+=1
    return str(count)

def cell(x,y,w,h,style,value=''):
    el=ET.SubElement(tree,'mxCell',id=ident(),value=value,style=style,vertex='1',parent='1'
    )
    ET.SubElement(el,'mxGeometry',x=str(x),y=str(y),width=str(w),height=str(h),attrib={'as':
    'geometry'})
    return el

def label(x,y,w,h,text,size=24,align='center'):
    html=f'<div style="font-family:Times New Roman;font-size:{size}px;color:#111111;">{text
    }</div>'
    return cell(x,y,w,h,f'text;html=1;strokeColor=None;fillColor=None;align={align};
    verticalAlign=middle;whiteSpace=wrap;fontSize={size};fontFamily=Times New Roman;',html)

def zh(s):return f'<span style="font-family:SimSun;">{s}</span>'

def ellipse(x,y,w,h,fill='none',dash=False,width=2.5,color='#111111'):
    return cell(x,y,w,h,f'ellipse;html=1;fillColor={fill};strokeColor={color};strokeWidth={
    width};dashed={int(dash)};')

def line(x1,y1,x2,y2,arrow='none',start='none',dash=False,color='#111111',width=2.5,points=
    None):
    el=ET.SubElement(tree,'mxCell',id=ident(),edge='1',parent='1',style=f'edgeStyle=None;
    html=1;rounded=0;strokeColor={color};strokeWidth={width};endArrow={arrow};startArrow={
    start};endFill=1;startFill=1;dashed={int(dash)};')
    g=ET.SubElement(el,'mxGeometry',relative='1',attrib={'as':'geometry'})
    ET.SubElement(g,'mxPoint',x=str(x1),y=str(y1),attrib={'as':'sourcePoint'})
    ET.SubElement(g,'mxPoint',x=str(x2),y=str(y2),attrib={'as':'targetPoint'})
    if points:
        arr=ET.SubElement(g,'Array',attrib={'as':'points'})
        for x,y in points: ET.SubElement(arr,'mxPoint',x=str(x),y=str(y))

# Projection: the axial length is 625 units, transverse diameter 100 units,
# preserving L/(2 R0)=6.25 before oblique projection of the circular ends.
ellipse(753,150,44,100,fill='#F2F2F2')
cell(150,150,625,100,'fillColor=#F2F2F2;strokeColor=None;')
line(150,150,775,150);line(150,250,775,250)
ellipse(440.5,150,44,100,fill='#F2F2F2',dash=True,width=1.8,color='#666666')
ellipse(128,150,44,100,fill='FFFFFF')

# Cartesian reference frame at the left-end centre, z along the cylinder axis.
line(150,200,860,200,arrow='block',dash=True,color='#666666',width=1.8)

```

```

line(150,200,91,254,arrow='block',width=2)
line(150,200,150,95,arrow='block',width=2)
label(94,185,25,30,'<i>O</i>')
label(74,247,25,30,'<i>x</i>')
label(133,64,34,30,'<i>y</i>')
label(860,183,30,30,'<i>z</i>')

# Initial radius and cylinder length.
line(150,200,150,150,arrow='classic',start='classic',width=2)
label(8,137,115,42,'<i>R</i><sub>0</sub> = 2 cm')
line(120,168,145,179,color='#666666',width=1.5)
line(150,265,150,312,color='#666666',width=1.5)
line(775,265,775,312,color='#666666',width=1.5)
line(150,301,775,301,arrow='classic',start='classic',width=2)
label(362.5,307,200,34,'<i>L</i> = 25 cm')
label(372.5,251,180,35,'<i>z</i> = <i>L</i>/2')

# Early-stage exchange directions (not universal sign assumptions).
line(344,105,344,143,arrow='block')
label(236.5,61,215,35,zh('预热热量传入'))
line(645,143,645,105,arrow='block')
label(550,61,190,35,zh('水分向外迁移'))
label(24,352,870,34,'(a) '+zh('初始圆柱几何与坐标系'))

# Enlarged midsection: the local origin is 0m, not the end-centre 0.
ellipse(124,454,192,192,fill='#F2F2F2')
line(220,550,355,550,arrow='block',width=2)
line(220,550,220,432,arrow='block',width=2)
label(355,535,30,30,'<i>x</i>');label(204,404,32,28,'<i>y</i>')
label(179,550,40,35,'<i>O</i><sub>m</sub>')
# A radial ray at 35 degrees, terminating on the circular boundary.
line(220,550,298.64,494.94,arrow='classic',width=2.5)
label(244,495,30,30,'<i>r</i>')
# Theta arc approximated by a short polyline, preserving editability.
import math
arc=[(220+40*math.cos(a),550-40*math.sin(a)) for a in [i*math.pi/180 for i in range
(0,36,5)]]
line(*arc[0],*arc[-1],arrow='classic',width=1.8,points=arc[1:-1])
label(267,520,30,30,'<i>θ</i>')
label(78,658,284,35,zh('中截面放大: ')+<i>z</i> = <i>L</i>/2')

# The one-dimensional radial domain retains spatial gradients.
line(410,550,491,550,arrow='block',width=2.5)
label(379,492,144,38,zh('轴对称降维'))
line(565,550,820,550,width=3)
ellipse(559,544,12,12,fill='#111111',width=2)
ellipse(814,544,12,12,fill='#FFFFFF',width=2)

```

```

label(532,574,66,32,'<i>r</i> = 0')
label(759,574,122,32,'<i>r</i> = <i>R</i>(<i>t</i>)'')
label(509,614,112,34,zh('中心对称'))
label(732.5,614,175,34,zh('表面对流交换'))
label(574,492,237,38,'<i>T</i>(<i>r</i>,<i>t</i>), <i>C</i>(<i>r</i>,<i>t</i>)'')
label(539.5,658,306,35,'0 ≤ <i>r</i> ≤ <i>R</i>(<i>t</i>)'')
label(24,710,870,34,'(b) '+zh('中截面圆柱坐标与一维径向计算域'))

out=ROOT/'drawio/geometry_coordinates.drawio'
ET.ElementTree(doc).write(out,encoding='utf-8',xml_declaration=True)
print(out)

```

2.16 run_figures.sh

```

# File: code/run_figures.sh
#!/bin/zsh
# Rebuild all data-driven figures with the repository's fixed font environment.
set -euo pipefail
SCRIPT_DIR="${0:A:h}"
export DYLD_LIBRARY_PATH="/opt/homebrew/opt/expat/lib${DYLD_LIBRARY_PATH:+:
    $DYLD_LIBRARY_PATH}"
export MPLCONFIGDIR="${MPLCONFIGDIR:-$HOME/.matplotlib-cumcm2026}"
mkdir -p "$MPLCONFIGDIR"
cd "$SCRIPT_DIR/.."

python3 - <<'PY'
from matplotlib import font_manager
font_manager._load_fontmanager(try_read_cache=False)
for name in ("Times New Roman", "Songti SC"):
    print(f"{name}: {font_manager.findfont(name, fallback_to_default=False)}")
PY

python3 "$SCRIPT_DIR/make_figures.py"
if python3 -c 'import openpyxl' >/dev/null 2>&1; then
    python3 "$SCRIPT_DIR/make_additional_figures.py"
else
    echo "提示: 当前 Python 缺少 openpyxl, 已完成基础图; 运行附加图前请安装 openpyxl。" >&2
fi

```

2.17 inspect_inputs.py

```

# File: inspect_inputs.py
from pathlib import Path

```

```

from openpyxl import load_workbook

BASE = Path(__file__).resolve().parent / "附件"

def preview_workbook(path: Path) -> None:
    print(f"\n=== {path.name} ===")
    book = load_workbook(path, read_only=True, data_only=False)
    print("sheets:", book.sheetnames)
    for sheet in book.worksheets:
        print(f"[{sheet.title}] size={sheet.max_row}x{sheet.max_column}")
        rows = list(sheet.iter_rows(values_only=True))
        for row in rows[:8]:
            print(row)
        if len(rows) > 8:
            print("... tail ...")
            for row in rows[-3:]:
                print(row)

if __name__ == "__main__":
    paths = [BASE / "附件1.xlsx", BASE / "附件2.xlsx"]
    paths.extend(sorted((BASE / "附件3").glob("*.xlsx"))))
    for workbook_path in paths:
        preview_workbook(workbook_path)

```

2.18 prepare_inputs.py

```

# File: comsol_q1/prepare_inputs.py
"""Prepare COMSOL boundary data and compare its output with the Q1 baseline."""

from __future__ import annotations

import csv
import json
import sys
from pathlib import Path

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator

HERE = Path(__file__).resolve().parent

```

```

PROJECT = HERE.parent

def read_environment() -> np.ndarray:
    book = load_workbook(PROJECT / "附件" / "附件1.xlsx", read_only=True, data_only=True)
    rows = []
    for row in book.active.iter_rows(min_row=2, values_only=True):
        if row[0] is not None:
            rows.append(tuple(float(x) for x in row[:3]))
    return np.asarray(rows)

def prepare() -> None:
    raw = read_environment()
    seconds = np.arange(0.0, 1800.0 + 1.0, 1.0)
    temperature = PchipInterpolator(raw[:, 0], raw[:, 1])(seconds)
    moisture = PchipInterpolator(raw[:, 0], raw[:, 2])(seconds)
    for name, values in (("ambient_temperature.txt", temperature), ("ambient_moisture.txt",
        moisture)):
        with (HERE / name).open("w", encoding="ascii", newline="") as handle:
            writer = csv.writer(handle, delimiter=" ", lineterminator="\n")
            writer.writerows(zip(seconds, values))
    metadata = {
        "source": str(PROJECT / "附件" / "附件1.xlsx"),
        "interpolation": "PCHIP sampled each second; COMSOL uses linear interpolation",
        "time_range_s": [0, 1800],
        "rows": len(seconds),
        "initial": {"temperature_c": float(temperature[0]), "moisture": float(moisture[0])
    },
        "final": {"temperature_c": float(temperature[-1]), "moisture": float(moisture[-1])
    },
    }
    (HERE / "input_metadata.json").write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8"
    )

def compare(csv_path: Path) -> None:
    """Compare COMSOL point export with the existing 1D finite-volume baseline."""
    baseline = json.loads((PROJECT / "results" / "q1" / "summary.json").read_text(encoding=
        "utf-8"))
    # COMSOL Data exports contain comment lines beginning with %. Remaining rows are
    numeric.
    data = np.loadtxt(csv_path, comments="%", delimiter=",")
    if data.ndim == 1:
        data = data[None, :]
    comsol_times = [0, 100, 300, 600, 900, 1200, 1500, 1800]

```

```

selected_radii = [0, 5, 10, 15, 20]
time_columns = [comsol_times.index(t) for t in baseline["table_times_s"]]
comsol_t = np.asarray([[data[r, 2 + 2 * j] for r in selected_radii] for j in
time_columns])
comsol_c = np.asarray([[data[r, 3 + 2 * j] for r in selected_radii] for j in
time_columns])
base_t = np.asarray(baseline["temperature_table_c"])
base_c = np.asarray(baseline["moisture_table"])
delta_t = comsol_t - base_t
delta_c = comsol_c - base_c
result = {
    "comsol_csv": str(csv_path),
    "shape": list(data.shape),
    "baseline_times_s": baseline["table_times_s"],
    "baseline_radii_cm": baseline["table_radii_cm"],
    "max_abs_temperature_difference_c": float(np.max(np.abs(delta_t))),
    "rmse_temperature_c": float(np.sqrt(np.mean(delta_t**2))),
    "max_abs_moisture_difference": float(np.max(np.abs(delta_c))),
    "rmse_moisture": float(np.sqrt(np.mean(delta_c**2))),
    "comsol_final_temperature_c": comsol_t[-1].tolist(),
    "baseline_final_temperature_c": base_t[-1].tolist(),
    "comsol_final_moisture": comsol_c[-1].tolist(),
    "baseline_final_moisture": base_c[-1].tolist(),
}
(HERE / "comparison.json").write_text(
    json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
)
with (HERE / "q1_comparison.csv").open("w", encoding="utf-8-sig", newline="") as handle
:
    writer = csv.writer(handle)
    writer.writerow(["time_s", "radius_cm", "comsol_T_C", "baseline_T_C", "delta_T_C",
        "comsol_C", "baseline_C", "delta_C"])
    for i, time_s in enumerate(baseline["table_times_s"]):
        for j, radius_cm in enumerate(baseline["table_radii_cm"]):
            writer.writerow([time_s, radius_cm, comsol_t[i, j], base_t[i, j], delta_t[i
, j],
                comsol_c[i, j], base_c[i, j], delta_c[i, j]])

if __name__ == "__main__":
    prepare()
    if len(sys.argv) == 2:
        compare(Path(sys.argv[1]))

```

2.19 Q1Automation.java

```
// File: comsol_q1/Q1Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** COMSOL reproduction of CUMCM 2026 A, question 1. */
public class Q1Automation {
    private static final String DEFAULT_WORK_DIR =
        "D:\\COMSOL\\test";

    public static Model run() throws java.io.IOException {
        return build(DEFAULT_WORK_DIR);
    }

    public static Model build(String workDir) throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(workDir);
        model.label("q1_comsol_model.mph");

        model.param().set("rad0", "0.02[m]", "Herb radius");
        model.param().set("L", "0.001[m]", "Short insulated axial slice");
        model.param().set("alphaT", "0.36/(820*2600)[m^2/s]", "Thermal diffusivity");
        model.param().set("betaT", "25/(820*2600)[m/s]", "Normalized heat transfer coefficient"
        );
        model.param().set("hm", "8e-7[m/s]");

        model.func().create("Te", "Interpolation");
        model.func("Te").set("funcname", "Te");
        model.func("Te").set("source", "file");
        model.func("Te").set("filename", workDir + "\\ambient_temperature.txt");
        model.func("Te").set("interp", "linear");
        model.func("Te").set("argunit", "s");

        model.func().create("Ce", "Interpolation");
        model.func("Ce").set("funcname", "Ce");
        model.func("Ce").set("source", "file");
        model.func("Ce").set("filename", workDir + "\\ambient_moisture.txt");
        model.func("Ce").set("interp", "linear");
        model.func("Ce").set("argunit", "s");

        model.component().create("comp1", true);
        model.component("comp1").geom().create("geom1", 2);
        model.component("comp1").geom("geom1").lengthUnit("m");
        model.component("comp1").geom("geom1").create("r1", "Rectangle");
        model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"rad0", "L"});
        model.component("comp1").geom("geom1").run();
    }
}
```

```

model.component("comp1").variable().create("var1");
model.component("comp1").variable("var1").set("Dmoist", "7e-9[m^2/s]*exp(-0.89/max(C,1e-6))");

model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{"T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*alphaT");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("g", "rad0*betaT*Te(t)");
model.component("comp1").physics("heat").feature("flux1").set("q", "rad0*betaT");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{"C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dmoist");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("g", "rad0*hm*Ce(t)");
model.component("comp1").physics("mass").feature("flux1").set("q", "rad0*hm");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").autoMeshSize(2);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "0 100 300 600 900 1200 1500 1800");
model.study("std1").feature("time").set("rtol", "1e-5");
model.study("std1").run();

double[] radii = new double[21];
double[] axial = new double[21];
for (int i = 0; i < 21; i++) {
    radii[i] = i * 0.001;
    axial[i] = 0.0005;
}

```

```

model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", radii);
model.result().dataset("cpt1").set("pointy", axial);

model.result().dataset().create("cln1", "CutLine2D");
model.result().dataset("cln1").set("genpoints", new double[][]{{0.0, 0.0005}, {0.02,
0.0005}});

model.result().create("pgT", "PlotGroup1D");
model.result("pgT").label("Temperature radial profile at 1800 s");
model.result("pgT").set("data", "cln1");
model.result("pgT").set("looplevel", new int[]{8});
model.result("pgT").create("lngr1", "LineGraph");
model.result("pgT").feature("lngr1").set("expr", "T");
model.result("pgT").feature("lngr1").set("descr", "Temperature (degC numeric)");
model.result("pgT").feature("lngr1").set("linewidth", "2");

model.result().create("pgC", "PlotGroup1D");
model.result("pgC").label("Moisture radial profile at 1800 s");
model.result("pgC").set("data", "cln1");
model.result("pgC").set("looplevel", new int[]{8});
model.result("pgC").create("lngr1", "LineGraph");
model.result("pgC").feature("lngr1").set("expr", "C");
model.result("pgC").feature("lngr1").set("descr", "Moisture (kg/kg numeric)");
model.result("pgC").feature("lngr1").set("linewidth", "2");

model.result().create("pgT2", "PlotGroup2D");
model.result("pgT2").label("Temperature field at 1800 s");
model.result("pgT2").set("looplevel", new int[]{8});
model.result("pgT2").create("surf1", "Surface");
model.result("pgT2").feature("surf1").set("expr", "T");

model.result().create("pgC2", "PlotGroup2D");
model.result("pgC2").label("Moisture field at 1800 s");
model.result("pgC2").set("looplevel", new int[]{8});
model.result("pgC2").create("surf1", "Surface");
model.result("pgC2").feature("surf1").set("expr", "C");
model.result().export().create("data1", "Data");
model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture"});
model.result().export("data1").set("filename", workDir + "\\q1_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(workDir + "\\q1_comsol_model_with_plots.mph");
return model;

```

```

}

public static void main(String[] args) throws java.io.IOException {
    String workDir = args != null && args.length > 0 ? args[0] : DEFAULT_WORK_DIR;
    build(workDir);
}
}

```

2.20 run_comsol_q1.ps1

```

# File: comsol_q1/run_comsol_q1.ps1
$ErrorActionPreference = 'Stop'
$ComsolBin = 'D:\COMSOL\install\COMSOL63\Multiphysics\bin\win64'
$WorkDir = 'D:\COMSOL\test'
$ProjectToolDir = 'D:\26mathmodel\CUMCM2026Problems\A题\comsol_q1'

Set-Location -LiteralPath $WorkDir
& 'D:\Anaconda\python.exe' "$ProjectToolDir\prepare_inputs.py"
Copy-Item -LiteralPath "$ProjectToolDir\ambient_temperature.txt" -Destination $WorkDir -
    Force
Copy-Item -LiteralPath "$ProjectToolDir\ambient_moisture.txt" -Destination $WorkDir -Force
& "$ComsolBin\comsolcompile.exe" 'Q1Automation.java'
if ($LASTEXITCODE -ne 0) { throw "COMSOL Java compilation failed: $LASTEXITCODE" }
& "$ComsolBin\comsolbatch.exe" -inputfile 'Q1Automation.class' -batchlog 'comsol_batch.log'
if ($LASTEXITCODE -ne 0) { throw "COMSOL batch run failed: $LASTEXITCODE" }
& 'D:\Anaconda\python.exe' "$ProjectToolDir\prepare_inputs.py" "$WorkDir\q1_comsol_profiles
    .csv"
Copy-Item -LiteralPath "$ProjectToolDir\comparison.json" -Destination $WorkDir -Force
Copy-Item -LiteralPath "$ProjectToolDir\q1_comparison.csv" -Destination $WorkDir -Force
Write-Host 'Q1 COMSOL simulation and comparison completed.'

```

2.21 prepare_q4_inputs.py

```

# File: comsol_q4/prepare_q4_inputs.py
from pathlib import Path
import json
import math

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator

ROOT = Path(__file__).resolve().parents[1]

```

```

OUT = Path(__file__).resolve().parent

def read_numeric_xlsx(path: Path) -> np.ndarray:
    ws = load_workbook(path, data_only=True, read_only=True).active
    rows = []
    for row in ws.iter_rows(values_only=True):
        try:
            rows.append([float(v) for v in row])
        except (TypeError, ValueError):
            continue
    return np.asarray(rows, dtype=float)

def write_table(path: Path, x: np.ndarray, y: np.ndarray) -> None:
    np.savetxt(path, np.column_stack([x, y]), fmt="%.12g", delimiter=" ")

def main() -> None:
    OUT.mkdir(parents=True, exist_ok=True)
    env = read_numeric_xlsx(ROOT / "附件" / "附件1.xlsx")
    radius = read_numeric_xlsx(ROOT / "附件" / "附件2.xlsx")

    summary = json.loads((ROOT / "results" / "q4" / "summary.json").read_text(encoding="utf-8"))
    end_s = math.ceil(float(summary["drying_time_s"]) / 600.0) * 600.0
    dense_t = np.arange(0.0, end_s + 60.0, 60.0)
    # Represent the transition from the measured warm-up series to the stated
    # 50 C / 0.05 constant-stage setpoints without smearing it over one minute.
    dense_t = np.unique(np.r_[dense_t, env[-1, 0] + 1e-6])
    te_interp = PchipInterpolator(env[:, 0], env[:, 1], extrapolate=False)
    ce_interp = PchipInterpolator(env[:, 0], env[:, 2], extrapolate=False)
    te = np.where(dense_t <= env[-1, 0], te_interp(np.minimum(dense_t, env[-1, 0])), 50.0)
    ce = np.where(dense_t <= env[-1, 0], ce_interp(np.minimum(dense_t, env[-1, 0])), 0.05)

    monotone_radius_m = np.minimum.accumulate(radius[:, 1]) / 100.0
    r_interp = PchipInterpolator(radius[:, 0], monotone_radius_m, extrapolate=False)
    rt = r_interp(dense_t)

    write_table(OUT / "ambient_temperature.txt", dense_t, te)
    write_table(OUT / "ambient_moisture.txt", dense_t, ce)
    write_table(OUT / "radius_history.txt", dense_t, rt)
    print(f"Prepared {len(dense_t)} rows through {end_s/3600:.3f} h")
    print(f"Radius: {rt[0]*100:.3f} cm -> {rt[-1]*100:.3f} cm")

if __name__ == "__main__":

```

```
main()
```

2.22 Q4Automation.java

```
// File: comsol_q4/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
        model.label("q4_shrinking_herb_comsol.mph");

        model.param().set("R0", "0.02[m]", "Initial herb radius");
        model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
        model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
        model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

        // Temperature unknown is stored as the numerical value in degC, matching the paper
        // equations.
        createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
        createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
        createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

        model.component().create("comp1", true);
        model.component("comp1").geom().create("geom1", 2);
        model.component("comp1").geom("geom1").lengthUnit("m");
        model.component("comp1").geom("geom1").create("r1", "Rectangle");
        model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
        model.component("comp1").geom("geom1").run();

        model.component("comp1").variable().create("var1");
        model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
        model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
        model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
        model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
    }
}
```

```

model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*
exp(-3850/(T+273.15))");
model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{
"T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)
");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{
"C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)
");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").autoMeshSize(3);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-4");
model.study("std1").run();

double[] x = new double[101];
double[] y = new double[101];

```

```

for (int i = 0; i < 101; i++) {
    x[i] = i * 0.0002;
    y[i] = 0.04;
}
model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");
model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

model.result().export().create("data1", "Data");
model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "Physical radius", "Herb radius"});
model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
return model;
}

private static void createInterpolation(Model model, String tag, String name,
                                         String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.23 analyze_q4_comsol.py

```
# File: comsol_q4/analyze_q4_comsol.py
from __future__ import annotations

import csv
import json
import re
from pathlib import Path

import numpy as np

HERE = Path(__file__).resolve().parent
BASELINE = HERE.parent / "results" / "q4" / "fields.npz"

def read_comsol_wide(path: Path):
    with path.open("r", encoding="utf-8-sig") as f:
        lines = [line for line in f if not line.startswith("%") or line.startswith("% X,")]
        header = next(line[2:].strip() for line in lines if line.startswith("% X,"))
        rows = [line for line in lines if not line.startswith("%")]
        names = next(csv.reader([header]))
        data = np.asarray([[float(x) for x in row] for row in csv.reader(rows)], dtype=float)
        times = []
        for j in range(2, len(names), 4):
            m = re.search(r"@ t=([0-9.Ee+-]+)", names[j])
            if not m:
                raise ValueError(names[j])
            times.append(float(m.group(1)))
        # COMSOL rounds long CSV time labels; recover exact values from the study schedule.
        # Recover the requested output times from the model's study definition.
        study = (path.parent / "Q4Automation.java").read_text(encoding="utf-8")
        schedule = re.search(r'\.set\("tlist", "range\(([^\)]+)\) ([^\)]+)\)', study)
        if schedule is None:
            raise ValueError("Cannot identify the COMSOL study output times")
        start, step, end = map(float, schedule.group(1).split(","))
        requested = np.r_[np.arange(start, end + step / 2, step),
                          [float(value) for value in schedule.group(2).split()]]
        if len(requested) != len(times) or not np.allclose(requested, times, rtol=0, atol=5):
            raise ValueError("COMSOL CSV columns do not match the study output schedule")
        return names, data, requested

def comparison_times(event_time_s):
    return np.array([21600., 43200., 86400., 129600., 172800., event_time_s])
```

```

def baseline_profile(base, key, time_s, event_time_s):
    """Use the native-grid event profile or interpolate at the requested time."""
    if abs(time_s - event_time_s) < 1e-6:
        return base["threshold_" + key]
    times = base["times_s"]
    index = int(np.searchsorted(times, time_s))
    if index < len(times) and abs(times[index] - time_s) < 1e-8:
        return base[key][index]
    if index == 0 or index == len(times):
        raise ValueError("Requested comparison time is outside the baseline trajectory")
    fraction = (time_s - times[index - 1]) / (times[index] - times[index - 1])
    return (1 - fraction) * base[key][index - 1] + fraction * base[key][index]

def main():
    names, data, times = read_comsol_wide(HERE / "q4_comsol_profiles.csv")
    x = data[:, 0]
    temp = data[:, 2::4].T
    moist = data[:, 3::4].T
    rphys = data[:, 4::4].T
    radius = data[0, 5::4]

    with np.load(BASELINE) as archive:
        base = {key: archive[key] for key in archive.files}
        base_xi = base["xi"]
        reference_summary = json.loads(BASELINE.with_name("summary.json").read_text(encoding="utf-8"))
        event_time = reference_summary["drying_time_s"]
        compare_times = comparison_times(event_time)
        records = []
        all_dt, all_dc = [], []
        for target in compare_times:
            ic = int(np.argmin(np.abs(times - target)))
            if abs(times[ic] - target) > 1e-6:
                raise ValueError("COMSOL output is missing a requested comparison time")
            bt = np.interp(x / 0.02, base_xi, baseline_profile(base, "temperature_c", target, event_time))
            bc = np.interp(x / 0.02, base_xi, baseline_profile(base, "moisture", target, event_time))
            dt = np.abs(temp[ic] - bt)
            dc = np.abs(moist[ic] - bc)
            all_dt.extend(dt)
            all_dc.extend(dc)
            records.append({
                "time_h": float(target / 3600),
                "comparison_time_s": float(target),
            })

```

```

        "comsol_output_time_s": float(times[ic]),
        "radius_cm": float(radius[ic] * 100),
        "comsol_center_moisture": float(moist[ic, 0]),
        "baseline_center_moisture": float(bc[0]),
        "comsol_surface_moisture": float(moist[ic, -1]),
        "baseline_surface_moisture": float(bc[-1]),
        "max_abs_temperature_difference_C": float(dt.max()),
        "max_abs_moisture_difference": float(dc.max()),
        "maximum_difference_xi": float(x[int(np.argmax(dc))] / 0.02),
    })

summary = {
    "comsol_version": "6.3.0.290",
    "formulation": "uniform material shrinkage in a fixed radial reference coordinate",
    "output_times": int(len(times)),
    "radial_sample_points": int(len(x)),
    "baseline_saved_nodes": int(len(base_xi)),
    "time_source": "Q4Automation.java study output schedule; CSV header times are
rounded",
    "event_comparison_scope": "Concentrations at the FVM event time; not an independent
COMSOL event search",
    "comparison": records,
    "overall_max_abs_temperature_difference_C": float(max(all_dt)),
    "overall_max_abs_moisture_difference": float(max(all_dc)),
    "event_time_h": event_time / 3600,
    "event_comsol_center_moisture": float(moist[np.argmin(abs(times-event_time)), 0]),
    "event_comsol_surface_moisture": float(moist[np.argmin(abs(times-event_time)), -1])
,
}
(HERE / "q4_comsol_validation.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
)
with (HERE / "q4_comsol_comparison.csv").open("w", newline="", encoding="utf-8-sig") as
f:
    writer = csv.DictWriter(f, fieldnames=list(records[0]))
    writer.writeheader()
    writer.writerows(records)
np.savez_compressed(HERE / "q4_comsol_fields.npz", times_s=times, x_ref_m=x,
                    radii_m=radius, rphys_m=rphys, temperature_c=temp, moisture=moist)
print(json.dumps(summary, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

2.24 plot_q4_physical.py

```
# File: comsol_q4/plot_q4_physical.py
"""Plot saved COMSOL fields with the shared manuscript typography."""
from pathlib import Path
import json
import sys

sys.dont_write_bytecode = True
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "code"))
import matplotlib as mpl
mpl.use("Agg")
from figure_style import configure_fonts, save_figure

configure_fonts()
mpl.rcParams.update({
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.linewidth": 0.6,
    "xtick.direction": "out",
    "ytick.direction": "out",
    "xtick.major.width": 0.6,
    "ytick.major.width": 0.6,
    "legend.frameon": False,
})

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, LogNorm
from matplotlib.lines import Line2D

# Academic Figure Skill Nature/Cell/Science Color Palette -- COPY VERBATIM
CATEGORICAL = ["#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666"]
CATEGORICAL_EXTENDED = [
    "#2166AC", "#B2182B", "#1B7837", "#F1A340", "#762A83", "#666666",
    "#4393C3", "#D6604D", "#5AAE61", "#B35806", "#9970AB", "#999999",
]

DIVERGING = ["#2166AC", "#F7F7F7", "#B2182B"]
SEQUENTIAL = ["#F7FBFF", "#6BAED6", "#08306B"]
ACCENT_RED = "#B2182B"
GREY = "#999999"
BLACK = "#222222"

HERE = Path(__file__).resolve().parent
BASE = HERE.parent / "results" / "q4" / "fields.npz"
```

```

from analyze_q4_comsol import baseline_profile, comparison_times
OUT = HERE / "figures"
OUT.mkdir(exist_ok=True)

# Asset confirmation: assets/figures was checked; no cylindrical physical-field asset
# exists.
# Cross-type inheritance: use the Q4 COMSOL field arrays and measured radius history
# directly.

def nearest(a, value):
    return int(np.argmin(np.abs(a - value)))

def interp_profile(xi_source, values, xi_target):
    return np.interp(xi_target, xi_source, values)

def draw_cylinder(ax, radius_cm, length_cm, xi, moisture, norm, cmap, title, initial_radius
=2.0):
    # Side surface carries the local surface value; front face exposes the radial solution.
    theta = np.linspace(0, 2*np.pi, 120)
    z = np.linspace(0, length_cm, 55)
    th, zz = np.meshgrid(theta, z)
    xx = radius_cm * np.cos(th)
    yy = radius_cm * np.sin(th)
    side_color = cmap(norm(np.full_like(xx, moisture[-1])))
    ax.plot_surface(zz, xx, yy, facecolors=side_color, shade=True,
                    linewidth=0, antialiased=True, alpha=0.72)

    rr = np.linspace(0, radius_cm, 90)
    th2, rr2 = np.meshgrid(theta, rr)
    xf = rr2 * np.cos(th2)
    yf = rr2 * np.sin(th2)
    cf = interp_profile(xi, moisture, np.clip(rr2 / radius_cm, 0, 1))
    ax.plot_surface(np.zeros_like(xf), xf, yf, facecolors=cmap(norm(cf)),
                    shade=False, linewidth=0, antialiased=True)

    # Initial outline makes the magnitude of shrinkage readable in every panel.
    ax.plot(np.zeros_like(theta), initial_radius*np.cos(theta), initial_radius*np.sin(theta)
    ),
            color=GREY, ls=(0, (2, 2)), lw=0.8, alpha=0.8)
    ax.plot([0, length_cm], [0, 0], [0, 0], color=BLACK, lw=0.45, alpha=0.6)
    ax.set_title(title, pad=1.5, fontfamily="SimSun")
    ax.set_xlim(-0.4, length_cm + 0.4)
    ax.set_ylim(-2.15, 2.15)
    ax.set_zlim(-2.15, 2.15)

```

```

ax.set_box_aspect((length_cm + 0.8, 4.3, 4.3))
ax.view_init(elev=20, azim=-63)
ax.set_axis_off()

def make_physical_plate(comsol):
    times = comsol["times_s"]
    xi = comsol["x_ref_m"] / 0.02
    moisture = comsol["moisture"]
    radii = comsol["radii_m"] * 100
    event_time = json.loads(BASE.with_name("summary.json").read_text(encoding="utf-8"))["drying_time_s"]
    chosen = [0.0, 21600.0, 86400.0, event_time]
    titles = [r"$0\,\mathrm{h}$", r"$6\,\mathrm{h}$", r"$24\,\mathrm{h}$",
              rf"${event_time/3600:.2f}\,\mathrm{h}$ (终点)"]
    cmap = LinearSegmentedColormap.from_list("moisture_blue", SEQUENTIAL, N=256)
    norm = LogNorm(vmin=0.05, vmax=2.55)

    mm = 1 / 25.4
    fig = plt.figure(figsize=(183*mm, 76*mm))
    gs = fig.add_gridspec(1, 5, width_ratios=[1, 1, 1, 1, 0.055], wspace=0.03)
    fig.subplots_adjust(left=0.015, right=0.88, bottom=0.12, top=0.74)
    for k, (target, title) in enumerate(zip(chosen, titles)):
        i = nearest(times, target)
        ax = fig.add_subplot(gs[0, k], projection="3d")
        draw_cylinder(ax, float(radii[i]), 6.0, xi, moisture[i], norm, cmap,
                      rf"{title}\n" + rf"$R={radii[i]:.3f}\,\mathrm{{cm}}$")
        ax.text2D(0.01, 0.95, chr(ord('a') + k), transform=ax.transAxes,
                  fontsize=10, fontweight="normal", color=BLACK)
    cax = fig.add_subplot(gs[0, 4])
    sm = mpl.cm.ScalarMappable(norm=norm, cmap=cmap)
    cb = fig.colorbar(sm, cax=cax)
    cb.set_label(r"水分浓度 $C\,,\mathrm{{kg/kg}}$", fontfamily="SimSun")
    cb.set_ticks([0.05, 0.1, 0.2, 0.5, 1, 2.55])
    cb.set_ticklabels(["0.05", "0.10", "0.20", "0.50", "1.0", "2.55"])
    cb.outline.set_linewidth(0.5)
    fig.suptitle("第四问: 收缩圆柱药材内部水分场\n (COMSOL 轴对称解重构)", y=0.98, fontsize=11)
    save_figure(fig, OUT / "q4_comsol_physical_moisture_field")
    plt.close(fig)

def make_validation_figure(comsol, baseline):
    ct = comsol["times_s"]
    cxi = comsol["x_ref_m"] / 0.02
    cm = comsol["moisture"]
    cr = comsol["radii_m"] * 100

```

```

bxi = baseline["xi"]
event_time = json.loads(BASE.with_name("summary.json").read_text(encoding="utf-8"))["
drying_time_s"]
chosen = [21600.0, 86400.0, event_time]
labels = ["6 h", "24 h", f"{event_time/3600:.2f} h"]
colors = [CATEGORICAL[0], CATEGORICAL[3], CATEGORICAL[1]]

mm = 1 / 25.4
fig, axes = plt.subplots(1, 3, figsize=(183*mm, 87*mm), gridspec_kw={"wspace": 0.62})
fig.subplots_adjust(left=0.105, right=0.985, bottom=0.27, top=0.77)
ax = axes[0]
for target, label, color in zip(chosen, labels, colors):
    ic = nearest(ct, target)
    r_cm = cxi * cr[ic]
    base_at_cxi = interp_profile(bxi, baseline_profile(baseline, "moisture", target,
event_time), cxi)
    ax.plot(r_cm, cm[ic], color=color, lw=1.5, label=label)
    stride = max(1, (len(cxi)-1)//10)
    pick = np.unique(np.r_[np.arange(0, len(cxi), stride), len(cxi)-1-np.array([0, 1,
2, 5])])
    ax.plot(r_cm[pick], base_at_cxi[pick], "o", ms=2.8, mfc="white",
mec=color, mew=0.7)
ax.set_xlabel(r"实际半径  $r$ , / $\backslash$ ,  $\mathrm{cm}$ ", fontfamily="SimSun")
ax.set_ylabel(r"水分浓度  $C$ , / $\backslash$ , ( $\mathrm{kg/kg}$ )", fontfamily="SimSun")
ax.set_xlim(0, 1.45)
handles, time_labels = ax.get_legend_handles_labels()
fig.legend(handles, time_labels, loc="upper center", bbox_to_anchor=(0.5, 0.92),
ncol=3, fontsize=10, title="干燥时间", title_fontsize=10)
ax.text(0.5, -0.42, "(a) 径向水分剖面", transform=ax.transAxes,
ha="center", va="top", fontsize=10)

ax = axes[1]
# Match the six times reported in the numerical comparison and paper.
parity_x, parity_y = [], []
for target in comparison_times(event_time):
    ic = nearest(ct, target)
    if abs(ct[ic] - target) > 1e-6:
        raise ValueError("Regenerate q4_comsol_fields.npz with exact study output times
")
    parity_x.extend(interp_profile(bxi, baseline_profile(baseline, "moisture", target,
event_time), cxi))
    parity_y.extend(cm[ic])
parity_x, parity_y = np.asarray(parity_x), np.asarray(parity_y)
lo, hi = 0.045, 1.82
ax.plot([lo, hi], [lo, hi], color=GREY, ls="--", lw=0.7)
ax.scatter(parity_x, parity_y, s=8, color=CATEGORICAL[0], alpha=0.7,
edgecolors="none", rasterized=True)

```

```

ax.set_xlabel("有限体积分水浓度\n(kg/kg)")
ax.set_ylabel("COMSOL 水分浓度\n(kg/kg)")
ax.set_xlim(lo, hi)
ax.set_ylim(lo, hi)
ax.set_aspect("equal", adjustable="box")
err = np.max(np.abs(parity_y - parity_x))
exponent = int(np.floor(np.log10(err)))
ax.text(0.03, 0.96, r"$\max|\Delta C|$" + "\n" +
        rf"=${err/10**exponent:.2f}\times 10^{{{exponent}}}$", transform=ax.transAxes,
        va="top", fontsize=10)
ax.text(0.5, -0.42, "(b) COMSOL 与基准模型\n逐点校核", transform=ax.transAxes,
        ha="center", va="top", fontsize=10)

ax = axes[2]
ax.plot(ct/3600, cr, color=CATEGORICAL[0], lw=1.5)
ax.fill_between(ct/3600, cr, 2.02, color=CATEGORICAL[0], alpha=0.10)
event_h = event_time/3600
ax.axvline(event_h, color=ACCENT_RED, ls=(0, (3, 2)), lw=1.0)
ax.scatter([event_h], [1.2], color=ACCENT_RED, s=18, zorder=3)
ax.annotate(f"主模型达标时刻\n{event_h:.2f} h", (event_h, 1.2), xytext=(-38, 18),
            textcoords="offset points", arrowprops=dict(arrowstyle="-", lw=0.6),
            ha="center", fontsize=10)
ax.set_xlabel(r"时间 $t\backslash,\backslash,\mathrm{h}$", fontfamily="SimSun")
ax.set_ylabel(r"药材半径 $R(t)\backslash,\backslash,\mathrm{cm}$", fontfamily="SimSun")
ax.set_xlim(0, 52)
ax.set_ylim(1.15, 2.05)
ax.text(0.5, -0.42, "(c) 药材半径收缩历程", transform=ax.transAxes,
        ha="center", va="top", fontsize=10)

fig.suptitle("COMSOL 物理场剖面、独立数值校核与收缩历程", y=0.98, fontsize=11)
save_figure(fig, OUT / "q4_comsol_profiles_validation")
plt.close(fig)
return {"max_abs_moisture_difference": float(err), "comparison_points": len(parity_x),
        "baseline_saved_nodes": len(bxi)}

def main():
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--refined', action='store_true', help='Use the completed 320-
    element, 1e-7 COMSOL validation run')
    args = parser.parse_args()
    if args.refined:
        global OUT
        from analyze_q4_comsol import read_comsol_wide
        _, data, times = read_comsol_wide(HERE / 'near_surface_check/graded320_tight/
        q4_comsol_profiles.csv')

```

```

    comsol = {'times_s': times, 'x_ref_m': data[:, 0], 'moisture': data[:, 3::4].T,
              'radii_m': data[0, 5::4]}
    OUT = HERE / 'near_surface_check/figures'
    OUT.mkdir(exist_ok=True)
    with np.load(BASE) as archive:
        baseline = {key: archive[key] for key in archive.files}
        stats = make_validation_figure(comsol, baseline)
        (OUT / 'figure_statistics.json').write_text(json.dumps(stats, indent=2), encoding='
utf-8')
        print(json.dumps(stats))
        return
    comsol = np.load(HERE / "q4_comsol_fields.npz")
    with np.load(BASE) as archive:
        baseline = {key: archive[key] for key in archive.files}
        make_physical_plate(comsol)
        make_validation_figure(comsol, baseline)
        print(json.dumps({"output": str(OUT), "figures": sorted(p.name for p in OUT.iterdir())
}, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

2.25 analyze_refinement.py

```

# File: comsol_q4/near_surface_check/analyze_refinement.py
"""Read-only comparison of independent COMSOL runs against the saved FVM field."""
from pathlib import Path
import sys, json, csv, hashlib
import numpy as np
HERE=Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parent))
from analyze_q4_comsol import read_comsol_wide, baseline_profile, comparison_times
ROOT=HERE.parents[1]
with np.load(ROOT/'results/q4/fields.npz') as z: base={k:z[k] for k in z.files}
event=json.loads((ROOT/'results/q4/summary.json').read_text(encoding='utf-8'))['
drying_time_s']
target= comparison_times(event)
fields={}; records=[]
for name, n, tol in json.loads((HERE/'manifest.json').read_text())['cases']:
    path=HERE/name/'q4_comsol_profiles.csv'
    if not path.exists(): continue
    _, raw, times=read_comsol_wide(path)
    xi=raw[:, 0]/.02; order=np.argsort(xi); xi=xi[order]
    tc=raw[order, 2::4].T; c=raw[order, 3::4].T

```

```

indexes=[int(np.argmin(abs(times-t))) for t in target]
selected=c[indexes];temp=tc[indexes]
ref=np.array([np.interp(xi,base['xi'],baseline_profile(base,'moisture',t,event)) for t in
    target])
tref=np.array([np.interp(xi,base['xi'],baseline_profile(base,'temperature_c',t,event)) for
    t in target])
err=np.abs(selected-ref);i,j=np.unravel_index(np.argmax(err),err.shape)
jcross=np.flatnonzero(c[:,0]<=.15)
crossing=None
if len(jcross):
    k=jcross[0];crossing=float(((times[k-1]+(.15-c[k-1,0]))/(c[k,0]-c[k-1,0]))*(times[k]-times[k-1]))/3600)
record={'case':name,'radial_elements':n or 'original_auto3','rtol':tol,
    'max_moisture_difference':float(err.max()),'at_time_h':float(target[i]/3600),'at_xi':float
    (xi[j]),
    'max_center_difference':float(err[:,0].max()),'max_surface_difference':float(err[:,-1].max
    ()),
    'max_outer_10pct_difference':float(err[:,xi>=.9].max()),
    'max_temperature_difference':float(np.max(abs(temp-tref))),
    'center_C_at_FVM_event':float(selected[-1,0]),'surface_C_at_FVM_event':float(selected
    [-1,-1]),
    'estimated_COMSOL_event_h':crossing,'event_estimation_scope':'linear interpolation of
    saved output; not solver event detection',
    'per_time_max_difference':[float(v) for v in err.max(axis=1)]}
fields[name]={ 'c':selected,'t':temp,'xi':xi}
records.append(record)
print(name,'max dC',record['max_moisture_difference'],'xi',record['at_xi'],'event',
    crossing,flush=True)
np.savez_compressed(HERE/'comparison_fields.npz',times_s=target,xi=xi,reference_moisture=
    ref,**{name+'_moisture':v['c'] for name,v in fields.items()})
comparisons=[]
for a,b in [( 'original_dense','original_tight'),( 'original_tight','graded80'),( 'graded80','
    graded160'),( 'graded160','graded320'),( 'graded320','graded320_tight')]:
    if a in fields and b in fields:
        comparisons.append({'a':a,'b':b,'max_moisture_difference':float(abs(fields[a]['c']-fields
            [b]['c'])).max()),'max_temperature_difference':float(abs(fields[a]['t']-fields[b]['t'])).
            max())})
with (HERE/'comparison.csv').open('w',newline='',encoding='utf-8-sig') as f:
    w=csv.DictWriter(f,fieldnames=list(records[0]));w.writeheader();w.writerows(records)
manifest=json.loads((HERE/'manifest.json').read_text())
protected={name:hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==value for name,value
    in manifest['source_sha256'].items()}
summary={'scope':'6 specified times and 1001 common reference radii; original input
    interpolation retained','cases':records,'successive_comparisons':comparisons,'
    protected_files_unchanged':protected}
(HERE/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='
    utf-8')

```

```
print(json.dumps(comparisons,indent=2));print('protected',protected)
```

2.26 write_report.py

```
# File: comsol_q4/near_surface_check/write_report.py
from pathlib import Path
import json
HERE=Path(__file__).resolve().parent
s=json.loads((HERE/'summary.json').read_text(encoding='utf-8'))
rows=s['cases'];best=next((r for r in rows if r['case']=='graded320_tight'),rows[-1])
lines=['# COMSOL 近表层网格与容差检验（独立试算）',' ',
'本目录为独立试算，未修改论文、原 COMSOL 模型或四问主计算结果。',
'', '## 检验设置', '',
'- 原 Q4Automation.java 的物性、系数型 PDE、初值、Robin 边界、半径及环境输入文件保持相同。',
'- 保持原参考矩形长度 0.08 m、轴向均匀和端面绝热；这不是含端面交换的二维验证。',
'- 原网格采用 autoMeshSize(3)。加密组使用映射四边形网格，径向 80/160/320 单元、轴向 4 单元，径向端点按 (1-cos(pi*i/N))/2 分布，在轴线与表层加密。',
'- 相对求解容差按表设置；仍为自适应时间求解，输出间隔不是固定积分步长。',
'- 与主模型在 6、12、24、36、48 h 及 51.0876993 h 比较；每时刻 1001 个共同径向位置（包括表面），主模型用全部 161 个原生节点线性插值。',
'- 所有“最大差”均针对上述六时刻/采样位置，不能解释为全时空的严格误差界。',
'', '## 与主有限体积结果比较', '',
'| 试算 | 径向单元 | 相对容差 | 最大水分绝对差 kg/kg | 时刻 h | r/R |',
'| ---|---|---|---|---|---|']
for r in rows:lines.append(f"| {r['case']} | {r['radial_elements']} | {r['rtol']} | {r['max_moisture_difference']:.8g} | {r['at_time_h']:.4f} | {r['at_xi']:.3f} |")
lines+=['', '## COMSOL 自身网格与容差变化', '',
'| 比较 | 最大水分差 kg/kg | 最大温差 °C |', '| ---|---|---|']
for r in s['successive_comparisons']:lines.append(f"| {r['a']} → {r['b']} | {r['max_moisture_difference']:.8g} | {r['max_temperature_difference']:.8g} |")
lines+=['', '## 结果解释', '',
'收紧原网格容差后，约 0.0117 kg/kg 的近表层偏差仍存在；使用表层加密网格后，偏差降至约 10-5 kg/kg 量级。这支持原 COMSOL 网格的近表层分辨能力是原偏差的主要来源。',
'', '加密组对有限体积结果的最大差不必随网格单调减小：比较基准本身仍有时间、空间离散误差和线性插值误差。因此应同时看 COMSOL 相邻网格之间的差异，不能把两法之差当成 COMSOL 的真实误差。',
'', f"最细且更紧容差组在主模型达标时刻的中心水分 {best['center_C_at_FVM_event']:.10f} kg/kg，表面水分 {best['surface_C_at_FVM_event']:.10f} kg/kg。",
'', f"按已保存输出线性插值得到的 COMSOL 达标时间估计为 {best['estimated_COMSOL_event_h']:.8f} h；此值不是内部事件求根结果，不应据此宣称秒以下的预测精度。",
'', '两法计算输入一致性不等于实验准确性验证；本次沿用原 COMSOL 的线性插值输入文件，未将其与主程序 PCHIP 之间的细微差异一并消除。',
'', '## 文件与复核', '',
'- 各子目录：Java 源码、batch.log、COMSOL 原始 CSV、包含解的 q4_shrinking_herb_comsol.mph。',
'- comparison.csv / summary.json：数值比较与逐时刻最大差。',
'- comparison_fields.npz：六时刻的共同采样场与有限体积参考。',
```

```
'- analyze_refinement.py: 重新读取原始导出并比较; write_report.py: 生成本报告。',
'- manifest.json: 运行设置与受保护文件的原始 SHA256。',
'',f"受保护文件保持不变: {all(s['protected_files_unchanged'].values())}。",
'', '网格 API 依据: [COMSOL 6.3 Distribution](https://doc.comsol.com/6.3/doc/com.comsol.help.comsol/comsol_api_mesh.49.073.html)、[Map](https://doc.comsol.com/6.3/doc/com.comsol.help.comsol/comsol_api_mesh.49.090.html)。']
(HERE/'REPORT.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
```

2.27 run_cases.ps1

```
# File: comsol_q4/near_surface_check/run_cases.ps1
$ErrorActionPreference = 'Stop'
$comsolBin = 'D:\COMSOL\install\COMSOL63\Multiphysics\bin\win64'
$experimentRoot = $PSScriptRoot
foreach ($caseName in @('original_dense', 'original_tight', 'graded80', 'graded160', 'graded320', 'graded320_tight')) {
    Push-Location -LiteralPath (Join-Path $experimentRoot $caseName)
    try {
        & "$comsolBin\comsolcompile.exe" Q4Automation.java
        if ($LASTEXITCODE -ne 0) { throw "Compilation failed: $caseName" }
        & "$comsolBin\comsolbatch.exe" -np 4 -inputfile Q4Automation.class -batchlog batch.log > launcher.log 2>&1
        if ($LASTEXITCODE -ne 0) { throw "Simulation failed: $caseName" }
    } finally { Pop-Location }
}
& 'D:\Anaconda\python.exe' -B (Join-Path $experimentRoot 'analyze_refinement.py')
if ($LASTEXITCODE -ne 0) { throw 'Comparison failed' }
& 'D:\Anaconda\python.exe' -B (Join-Path $experimentRoot 'write_report.py')
```

2.28 Q4Automation.java (original_dense)

```
// File: comsol_q4/near_surface_check/original_dense/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\original_dense";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
    }
}
```

```

model.modelPath(WORK_DIR);
model.label("q4_shrinking_herb_comsol.mph");

model.param().set("R0", "0.02[m]", "Initial herb radius");
model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

// Temperature unknown is stored as the numerical value in degC, matching the paper
// equations.
createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

model.component().create("comp1", true);
model.component("comp1").geom().create("geom1", 2);
model.component("comp1").geom("geom1").lengthUnit("m");
model.component("comp1").geom("geom1").create("r1", "Rectangle");
model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
model.component("comp1").geom("geom1").run();

model.component("comp1").variable().create("var1");
model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*exp(-3850/(T+273.15))");
model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{"T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)");

```

```

");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{
"C"});

model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").autoMeshSize(3);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-4");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {
    x[i] = i * 0.00002;
    y[i] = 0.04;
}

model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");
model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

```

```

    model.result().export().create("data1", "Data");
    model.result().export("data1").set("data", "cpt1");
    model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
    model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "Physical radius", "Herb radius"});
    model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
    model.result().export("data1").set("separator", ",");
    model.result().export("data1").run();

    model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
    return model;
}

private static void createInterpolation(Model model, String tag, String name,
                                       String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.29 Q4Automation.java (original_tight)

```

// File: comsol_q4/near_surface_check/original_tight/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\original_tight";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
    }
}

```

```

model.label("q4_shrinking_herb_comsol.mph");

model.param().set("R0", "0.02[m]", "Initial herb radius");
model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

// Temperature unknown is stored as the numerical value in degC, matching the paper
// equations.
createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

model.component().create("comp1", true);
model.component("comp1").geom().create("geom1", 2);
model.component("comp1").geom("geom1").lengthUnit("m");
model.component("comp1").geom("geom1").create("r1", "Rectangle");
model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
model.component("comp1").geom("geom1").run();

model.component("comp1").variable().create("var1");
model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*exp(-3850/(T+273.15))");
model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{"T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)");

```

```

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[] {
"C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").autoMeshSize(3);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-6");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {
    x[i] = i * 0.00002;
    y[i] = 0.04;
}
model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");
model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

model.result().export().create("data1", "Data");

```

```

model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "Physical radius", "Herb radius"});
model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
return model;
}

private static void createInterpolation(Model model, String tag, String name,
                                         String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.30 Q4Automation.java (graded80)

```

// File: comsol_q4/near_surface_check/graded80/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 * /
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\graded80";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
        model.label("q4_shrinking_herb_comsol.mph");
    }
}

```

```

model.param().set("R0", "0.02[m]", "Initial herb radius");
model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

// Temperature unknown is stored as the numerical value in degC, matching the paper
// equations.
createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

model.component().create("comp1", true);
model.component("comp1").geom().create("geom1", 2);
model.component("comp1").geom("geom1").lengthUnit("m");
model.component("comp1").geom("geom1").create("r1", "Rectangle");
model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
model.component("comp1").geom("geom1").run();

model.component("comp1").variable().create("var1");
model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*exp(-3850/(T+273.15))");
model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{"T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");

```

```

model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{
"C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").create("map1", "Map");
model.component("comp1").mesh("mesh1").feature("map1").create("rad", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").selection().set(
new int[]{2,3});
double[] locations = new double[81];
for(int i=0;i<=80;i++) locations[i]=0.5*(1-Math.cos(Math.PI*i/80));
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").set("explicit",
locations);
model.component("comp1").mesh("mesh1").feature("map1").create("ax", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").selection().set(
new int[]{1,4});
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").set("numelem",4);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-6");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {
    x[i] = i * 0.00002;
    y[i] = 0.04;
}
model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");

```

```

model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

model.result().export().create("data1", "Data");
model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "Physical radius", "Herb radius"});
model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
return model;
}

private static void createInterpolation(Model model, String tag, String name,
                                         String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.31 Q4Automation.java (graded160)

```

// File: comsol_q4/near_surface_check/graded160/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */

```

```

public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\graded160";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
        model.label("q4_shrinking_herb_comsol.mph");

        model.param().set("R0", "0.02[m]", "Initial herb radius");
        model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
        model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
        model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

        // Temperature unknown is stored as the numerical value in degC, matching the paper
        // equations.
        createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
        createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
        createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

        model.component().create("comp1", true);
        model.component("comp1").geom().create("geom1", 2);
        model.component("comp1").geom("geom1").lengthUnit("m");
        model.component("comp1").geom("geom1").create("r1", "Rectangle");
        model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
        model.component("comp1").geom("geom1").run();

        model.component("comp1").variable().create("var1");
        model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
        model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
        model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
        model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
        model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*exp(-3850/(T+273.15))");
        model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
        model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

        // Reference-coordinate radial PDE. x is the material radius at t=0.
        model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
        model.component("comp1").physics("heat").field("dimensionless").field("T");
        model.component("comp1").physics("heat").field("dimensionless").component(new String[]{"T"});
        model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
        model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
    }
}

```

```

model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{
"C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").create("map1", "Map");
model.component("comp1").mesh("mesh1").feature("map1").create("rad", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").selection().set(
new int[]{2,3});
double[] locations = new double[161];
for(int i=0;i<=160;i++) locations[i]=0.5*(1-Math.cos(Math.PI*i/160));
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").set("explicit",
locations);
model.component("comp1").mesh("mesh1").feature("map1").create("ax", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").selection().set(
new int[]{1,4});
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").set("numelem",4);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-6");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {

```

```

        x[i] = i * 0.00002;
        y[i] = 0.04;
    }
    model.result().dataset().create("cpt1", "CutPoint2D");
    model.result().dataset("cpt1").set("pointx", x);
    model.result().dataset("cpt1").set("pointy", y);

    model.result().create("pgC", "PlotGroup2D");
    model.result("pgC").label("Moisture field in shrinking herb reference domain");
    model.result("pgC").create("surf1", "Surface");
    model.result("pgC").feature("surf1").set("expr", "C");

    model.result().create("pgT", "PlotGroup2D");
    model.result("pgT").label("Temperature field in shrinking herb reference domain");
    model.result("pgT").create("surf1", "Surface");
    model.result("pgT").feature("surf1").set("expr", "T");

    model.result().export().create("data1", "Data");
    model.result().export("data1").set("data", "cpt1");
    model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
    model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "Physical radius", "Herb radius"});
    model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
    model.result().export("data1").set("separator", ",");
    model.result().export("data1").run();

    model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
    return model;
}

private static void createInterpolation(Model model, String tag, String name,
                                         String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.32 Q4Automation.java (graded320)

```
// File: comsol_q4/near_surface_check/graded320/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\graded320";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
        model.label("q4_shrinking_herb_comsol.mph");

        model.param().set("R0", "0.02[m]", "Initial herb radius");
        model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
        model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
        model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

        // Temperature unknown is stored as the numerical value in degC, matching the paper
        // equations.
        createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
        createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
        createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

        model.component().create("comp1", true);
        model.component("comp1").geom().create("geom1", 2);
        model.component("comp1").geom("geom1").lengthUnit("m");
        model.component("comp1").geom("geom1").create("r1", "Rectangle");
        model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
        model.component("comp1").geom("geom1").run();

        model.component("comp1").variable().create("var1");
        model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
        model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
        model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*Csafe/(Csafe+1)");
        model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*Csafe/(Csafe+1)");
        model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*exp(-3850/(T+273.15))");
        model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
        model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");
    }
}
```

```

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[]{
    "T"});
model.component("comp1").physics("heat").feature("cfreq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfreq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfreq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[]{
    "C"});
model.component("comp1").physics("mass").feature("cfreq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfreq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfreq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").create("map1", "Map");
model.component("comp1").mesh("mesh1").feature("map1").create("rad", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").selection().set(
    new int[]{2,3});
double[] locations = new double[321];
for(int i=0;i<=320;i++) locations[i]=0.5*(1-Math.cos(Math.PI*i/320));
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").set("explicit",
    locations);
model.component("comp1").mesh("mesh1").feature("map1").create("ax", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").selection().set(
    new int[]{1,4});
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").set("numelem",4);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");

```

```

model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-6");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {
    x[i] = i * 0.00002;
    y[i] = 0.04;
}
model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");
model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

model.result().export().create("data1", "Data");
model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "
Physical radius", "Herb radius"});
model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
return model;
}

private static void createInterpolation(Model model, String tag, String name,
String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\" + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

```

```

}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}

```

2.33 Q4Automation.java (graded320_tight)

```

// File: comsol_q4/near_surface_check/graded320_tight/Q4Automation.java
import com.comsol.model.*;
import com.comsol.model.util.*;

/** Question 4: coupled heat/moisture transport in a uniformly shrinking cylindrical herb.
 */
public class Q4Automation {
    private static final String WORK_DIR =
        "D:\\26mathmodel\\CUMCM2026Problems\\A题\\comsol_q4\\near_surface_check\\
        graded320_tight";

    public static Model build() throws java.io.IOException {
        Model model = ModelUtil.create("Model");
        model.modelPath(WORK_DIR);
        model.label("q4_shrinking_herb_comsol.mph");

        model.param().set("R0", "0.02[m]", "Initial herb radius");
        model.param().set("L", "0.08[m]", "Representative cylinder length; ends insulated");
        model.param().set("hT", "25[W/(m^2*K)]", "Convective heat transfer coefficient");
        model.param().set("hm", "8e-7[m/s]", "Convective moisture transfer coefficient");

        // Temperature unknown is stored as the numerical value in degC, matching the paper
        // equations.
        createInterpolation(model, "Te", "Te", "ambient_temperature.txt", "1");
        createInterpolation(model, "Ce", "Ce", "ambient_moisture.txt", "1");
        createInterpolation(model, "Rt", "Rt", "radius_history.txt", "m");

        model.component().create("comp1", true);
        model.component("comp1").geom().create("geom1", 2);
        model.component("comp1").geom("geom1").lengthUnit("m");
        model.component("comp1").geom("geom1").create("r1", "Rectangle");
        model.component("comp1").geom("geom1").feature("r1").set("size", new String[]{"R0", "L"});
        model.component("comp1").geom("geom1").run();

        model.component("comp1").variable().create("var1");
    }
}

```

```

model.component("comp1").variable("var1").set("Csafe", "max(C,1e-6)");
model.component("comp1").variable("var1").set("rho", "760[kg/m^3]+90[kg/m^3]*Csafe");
model.component("comp1").variable("var1").set("cp", "1850[J/(kg*K)]+2150[J/(kg*K)]*
Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("kherb", "0.12[W/(m*K)]+0.20[W/(m*K)]*
Csafe/(Csafe+1)");
model.component("comp1").variable("var1").set("Dherb", "4.2e-4[m^2/s]*exp(-0.30/Csafe)*
exp(-3850/(T+273.15))");
model.component("comp1").variable("var1").set("shrink", "R0/Rt(t)");
model.component("comp1").variable("var1").set("rphys", "x*Rt(t)/R0");

// Reference-coordinate radial PDE. x is the material radius at t=0.
model.component("comp1").physics().create("heat", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("heat").field("dimensionless").field("T");
model.component("comp1").physics("heat").field("dimensionless").component(new String[] {
"T"});
model.component("comp1").physics("heat").feature("cfeq1").set("da", "x*rho*cp");
model.component("comp1").physics("heat").feature("cfeq1").set("c", "x*kherb*shrink^2");
model.component("comp1").physics("heat").feature("cfeq1").set("f", "0");
model.component("comp1").physics("heat").feature("init1").set("T", "28");
model.component("comp1").physics("heat").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("heat").feature("flux1").selection().set(4);
model.component("comp1").physics("heat").feature("flux1").set("q", "R0*hT*shrink");
model.component("comp1").physics("heat").feature("flux1").set("g", "R0*hT*shrink*Te(t)
");

model.component("comp1").physics().create("mass", "CoefficientFormPDE", "geom1");
model.component("comp1").physics("mass").field("dimensionless").field("C");
model.component("comp1").physics("mass").field("dimensionless").component(new String[] {
"C"});
model.component("comp1").physics("mass").feature("cfeq1").set("da", "x");
model.component("comp1").physics("mass").feature("cfeq1").set("c", "x*Dherb*shrink^2");
model.component("comp1").physics("mass").feature("cfeq1").set("f", "0");
model.component("comp1").physics("mass").feature("init1").set("C", "2.55");
model.component("comp1").physics("mass").create("flux1", "FluxBoundary", 1);
model.component("comp1").physics("mass").feature("flux1").selection().set(4);
model.component("comp1").physics("mass").feature("flux1").set("q", "R0*hm*shrink");
model.component("comp1").physics("mass").feature("flux1").set("g", "R0*hm*shrink*Ce(t)
");

model.component("comp1").mesh().create("mesh1");
model.component("comp1").mesh("mesh1").create("map1", "Map");
model.component("comp1").mesh("mesh1").feature("map1").create("rad", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("rad").selection().set(
new int[] {2,3});
double[] locations = new double[321];
for(int i=0;i<=320;i++) locations[i]=0.5*(1-Math.cos(Math.PI*i/320));

```

```

model.component("comp1").mesh("mesh1").feature("map1").feature("rad").set("explicit",
locations);
model.component("comp1").mesh("mesh1").feature("map1").create("ax", "Distribution");
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").selection().set(
new int[]{1,4});
model.component("comp1").mesh("mesh1").feature("map1").feature("ax").set("numelem",4);
model.component("comp1").mesh("mesh1").run();

model.study().create("std1");
model.study("std1").create("time", "Transient");
model.study("std1").feature("time").set("tlist", "range(0,600,183600)
183915.71747843814 184200");
model.study("std1").feature("time").set("rtol", "1e-7");
model.study("std1").run();

double[] x = new double[1001];
double[] y = new double[1001];
for (int i = 0; i < 1001; i++) {
    x[i] = i * 0.00002;
    y[i] = 0.04;
}
model.result().dataset().create("cpt1", "CutPoint2D");
model.result().dataset("cpt1").set("pointx", x);
model.result().dataset("cpt1").set("pointy", y);

model.result().create("pgC", "PlotGroup2D");
model.result("pgC").label("Moisture field in shrinking herb reference domain");
model.result("pgC").create("surf1", "Surface");
model.result("pgC").feature("surf1").set("expr", "C");

model.result().create("pgT", "PlotGroup2D");
model.result("pgT").label("Temperature field in shrinking herb reference domain");
model.result("pgT").create("surf1", "Surface");
model.result("pgT").feature("surf1").set("expr", "T");

model.result().export().create("data1", "Data");
model.result().export("data1").set("data", "cpt1");
model.result().export("data1").set("expr", new String[]{"T", "C", "rphys", "Rt(t)"});
model.result().export("data1").set("descr", new String[]{"Temperature", "Moisture", "
Physical radius", "Herb radius"});
model.result().export("data1").set("filename", WORK_DIR + "\\q4_comsol_profiles.csv");
model.result().export("data1").set("separator", ",");
model.result().export("data1").run();

model.save(WORK_DIR + "\\q4_shrinking_herb_comsol.mph");
return model;
}

```

```
private static void createInterpolation(Model model, String tag, String name,
                                       String file, String fununit) {
    model.func().create(tag, "Interpolation");
    model.func(tag).set("funcname", name);
    model.func(tag).set("source", "file");
    model.func(tag).set("filename", WORK_DIR + "\\ " + file);
    model.func(tag).set("interp", "linear");
    model.func(tag).set("argunit", "s");
    model.func(tag).set("fununit", fununit);
}

public static void main(String[] args) throws java.io.IOException {
    build();
}
}
```